

เอกสารประกอบการสอน

วิชา โครงสร้างระบบคอมพิวเตอร์ (Computer System Organization)

บทที่ 6 หน่วยคำนวณทางคณิตศาสตร์และตรรกะ (Arithmetic and Logic Unit)

วัตถุประสงค์เชิงพฤติกรรม (Behavioral Objectives)

1. บอกหน้าที่การทำงานหน่วยคำนวณทางคณิตศาสตร์และตรรกะ ALU (Arithmetic and Logic Unit) ได้อย่างถูกต้อง
2. บอกการประมวลผลทางคณิตศาสตร์พื้นฐานที่ ALU (Arithmetic and Logic Unit) ดำเนินการได้อย่างถูกต้อง
3. บอกการประมวลผลทางตรรกะพื้นฐาน และการเปรียบเทียบที่ ALU (Arithmetic and Logic Unit) ดำเนินการได้อย่างถูกต้อง
4. บอกหน้าที่ของหน่วยควบคุม (Control Unit) ในการสั่งการ ALU (Arithmetic and Logic Unit) ได้อย่างถูกต้อง
5. อธิบายการทำงานของรีจิสเตอร์และแฟล็ก (Registers and Flags):ในการจัดเก็บผลลัพธ์และสถานะการประมวลผลได้อย่างถูกต้อง
6. อธิบายหลักการจัดเก็บตัวเลขจำนวนเต็มแบบไม่คิดเครื่องหมาย (Unsigned Representation) และ แบบคิดเครื่องหมาย (Sign-Magnitude Representation) ได้อย่างถูกต้อง
7. อธิบายหลักการคำนวณของ 1's Complement ของเลขฐานสองได้อย่างถูกต้อง
8. อธิบายหลักการคำนวณของ 2's Complement ของเลขฐานสองได้อย่างถูกต้อง
9. อธิบายหลักการบวกและการลบเลขฐานสอง (Binary Addition and Subtraction) ได้อย่างถูกต้อง
10. อธิบายหลักการลบเลขฐานสองโดยใช้วิธี 2's Complement ได้อย่างถูกต้อง
11. อธิบายหลักการบวกคอมพิวเตอร์ด้วย การบวกของวงจร Logic ได้อย่างถูกต้อง
12. อธิบายหลักการคูณเลขฐานสอง (Binary Multiplication) ได้อย่างถูกต้อง
13. อธิบายหลักการคูณเลขจำนวนเต็มแบบไม่คิดเครื่องหมาย (Unsigned Integer Multiplication)
14. อธิบายส่วนประกอบและการทำงานของวงจรคูณพื้นฐานของฮาร์ดแวร์สำหรับการคูณได้อย่างถูกต้อง
15. อธิบายการคูณเลขจำนวนเต็มแบบไม่คิดเครื่องหมายด้วยการทำงานฮาร์ดแวร์ได้อย่างถูกต้อง
16. อธิบายหลักการคูณเลขจำนวนเต็มแบบคิดเครื่องหมาย (Signed Integer Multiplication) ได้อย่างถูกต้อง
17. อธิบายหลักการการขยายบิตเครื่องหมาย (Sign Extension) ได้อย่างถูกต้อง
18. อธิบายการคูณเลขฐานสองแบบฮาร์ดแวร์สำหรับการคูณธรรมดาได้อย่างถูกต้อง
19. อธิบายการทำงาน Flowchart การคูณเลขฐานสองแบบฮาร์ดแวร์สำหรับการคูณธรรมดาแบบตัวเลขขนาด 4 บิต ได้อย่างถูกต้อง
20. อธิบายหลักการคำนวณคูณเลขฐานสองแบบมีเครื่องหมาย (Signed Binary Numbers) แบบอัลกอริทึม (Booth's Algorithm) ได้อย่างถูกต้อง

21. อธิบายการคำนวณเลขฐานสองด้วยแนวคิดหลัก แบบอัลกอริทึม (Booth's Algorithm) ด้วยการคูณเลขฐานสองแบบมีเครื่องหมาย (2's Complement) ได้อย่างถูกต้อง
22. อธิบายการทำงาน Flowchart การคูณเลขฐานสอง แบบอัลกอริทึม (Booth's Algorithm) ด้วยการคูณเลขฐานสองแบบมีเครื่องหมาย (2's Complement) ได้อย่างถูกต้อง
23. อธิบายหลักการหารเลขฐานสอง (Binary Division) แบบเลขจำนวนเต็ม (Integer Division) ได้อย่างถูกต้อง
24. อธิบายการหารเลขฐานสอง (Binary Division) แบบไม่ลงตัว เศษเหลือ (Remainder) ได้อย่างถูกต้อง
25. อธิบายหลักการหารเลขฐานสองด้วยแบบอัลกอริทึม (Restoring Division Algorithm) ได้อย่างถูกต้อง
26. อธิบายส่วนประกอบหลักของวงจรหารเลขฐานสอง (Hardware for Binary Division) ได้อย่างถูกต้อง
27. อธิบายการทำงานวงจรหารเลขฐานสอง (Hardware Restoring Division Algorithm) ได้อย่างถูกต้อง
28. อธิบายขั้นตอนการทำงาน Flowchart การหารเลขฐานสองแบบอัลกอริทึม (Restoring Division Algorithm) ได้อย่างถูกต้อง
29. บอกการจัดเก็บและเลขจำนวนจริงตามมาตรฐาน IEEE754 (Floating Point Representation) ได้อย่างถูกต้อง
30. อธิบายหลักการจัดเก็บเลขจำนวนจริงตามมาตรฐาน IEEE 754 (Floating Point Representation) ได้อย่างถูกต้อง
31. อธิบายหลักการจัดเก็บรูปแบบการจัดเก็บเลขทศนิยมตามมาตรฐาน IEEE 754 แบบ (Single-precision และ แบบ Double-precision) ได้อย่างถูกต้อง
32. บอกหน้าที่ Sign bit (S) ของส่วนประกอบของเลขทศนิยมตามมาตรฐาน IEEE 754 ได้อย่างถูกต้อง
33. บอกหน้าที่ Exponents (E) ของส่วนประกอบของเลขทศนิยมตามมาตรฐาน IEEE 754 ได้อย่างถูกต้อง
34. อธิบายหลักการ Normal Form (รูปแบบปกติ) ของ Mantissa (M) ตามส่วนประกอบของเลขทศนิยมตามมาตรฐาน IEEE 754 ได้อย่างถูกต้อง

1. การทำงานของหน่วยคำนวณทางคณิตศาสตร์และตรรกะ (Arithmetic and Logic Unit - ALU)



รูปที่ 6-1 ภาพจำลองกลไกการทำงานภายในของ CPU ผ่านการเปรียบเทียบกับการประมวลผลข้อมูลเชิงตรรกะและรีจิสเตอร์

(ประมวลผล) ให้เป็นอาหารจานใหม่ (ผลลัพธ์)

1.2 การประมวลผลทางตรรกะ: นอกจากคำนวณแล้ว ALU ยังทำหน้าที่ "ตัดสินใจ" และ "จัดการกับเงื่อนไข" ด้วยการดำเนินการทางตรรกะ เช่น

AND: (และ): ถ้าทุกเงื่อนไขเป็นจริง ผลลัพธ์จึงจะเป็นจริง (เช่น ถ้ามีข้าวสาร และ เนื้อหมู ถึงจะทำข้าวผัดหมูได้)

OR: (หรือ): ถ้ามีเงื่อนไขใดเงื่อนไขหนึ่งเป็นจริง ผลลัพธ์ก็จะเป็นจริง (เช่น ถ้ามีไก่ หรือ มีปลา ก็ทำอาหารได้หนึ่งอย่าง)

NOT: (ไม่) กลับค่าจากจริงเป็นเท็จ และเท็จเป็นจริง (เช่น ไม่ใช่ไก่ ก็อาจจะเป็นปลา)

COMPLEMENT: (ส่วนเติมเต็ม) คล้ายกับ NOT แต่เป็นการกลับบิตทั้งหมดจาก 0 เป็น 1 และ 1 เป็น 0

การเปรียบเทียบ: ALU สามารถเปรียบเทียบค่าสองค่าได้ เช่น มากกว่า (>), น้อยกว่า (<), เท่ากับ (=), ไม่เท่ากับ (!=) เพื่อใช้ในการควบคุมการทำงานของโปรแกรม เช่น ถ้าคะแนนมากกว่า 50 ให้ "ผ่าน"

บทบาทหน้าที่ของหน่วยควบคุม (Control Unit - CU): CU เปรียบเสมือน "ผู้จัดการวงออเคสตรา"

หรือ "หัวหน้าพ่อครัว" ที่คอยส่งสัญญาณและสั่งการให้ ALU ทำงานตามลำดับและประเภทการ

ประมวลผลที่ถูกต้อง ไม่ว่าจะเป็นคำสั่งบวก ลบ หรือการเปรียบเทียบ

ALU คืออะไร ?: ALU เปรียบเสมือน

"สมอง" ที่ทำหน้าที่หลักในการคำนวณและตัดสินใจเชิงตรรกะของคอมพิวเตอร์ มัน

ไม่ใช่แค่เครื่องคิดเลขธรรมดา แต่เป็น

"เครื่องคำนวณและผู้ตัดสินใจ" ที่จัดการกับข้อมูลระดับบิต

1.1 การประมวลผลทางคณิตศาสตร์: คือ

หน้าที่พื้นฐานที่สุดที่ ALU ทำ ไม่ว่าจะเป็น

การบวก (Addition), ลบ (Subtraction),

คูณ (Multiplication), และหาร

(Division) ซึ่งทั้งหมดนี้จะดำเนินการกับ

ตัวเลขในรูปแบบฐานสอง

ตัวอย่างเปรียบเทียบ: ลองนึกภาพ ALU

เป็นพ่อครัวในครัวที่เชี่ยวชาญการทำอาหาร

พ่อครัวคนนี้รับวัตถุดิบ (ข้อมูล) มาปรุง

2.การทำงานของรีจิสเตอร์และแฟล็ก (Registers and Flags):



รูปที่ 6-2 ภาพจำลององค์ประกอบหลักภายในหน่วยประมวลผลกลาง: หน่วยคำนวณและตรรกะ (ALU), รีจิสเตอร์ (Registers) และแฟล็กแสดงสถานะ (Status Flags)

รีจิสเตอร์ (Registers): เป็นหน่วยความจำขนาดเล็กพิเศษที่อยู่ภายใน CPU ใกล้กับ ALU มากๆ ทำหน้าที่เหมือน "โต๊ะทำงาน" ของ ALU ที่ใช้เก็บข้อมูลชั่วคราวที่กำลังประมวลผล และเก็บผลลัพธ์ที่ได้ทันที มีความเร็วสูงที่สุด

แฟล็ก (Flags): เป็นบิตสถานะที่เก็บข้อมูลเพิ่มเติมเกี่ยวกับผลลัพธ์ของการดำเนินการล่าสุดของ ALU เปรียบเสมือน "โน้ตย่อเล็กๆ" ที่ ALU เขียนไว้ เช่น

Zero Flag (Z): บอกว่าผลลัพธ์เป็นศูนย์หรือไม่ (ถ้าผลลัพธ์เป็น 0 แฟล็กนี้จะถูกตั้งค่า)

Carry Flag (C): บอกว่ามีการทด (carry) เกิดขึ้นจากการบวก หรือมีการยืม (borrow) เกิดขึ้นจากการลบหรือไม่

Overflow Flag (V): บอกว่าผลลัพธ์ที่ได้มีขนาดเกินกว่าที่รีจิสเตอร์จะเก็บได้หรือไม่

ตัวอย่างประเภทตัวแปร

ชนิดของตัวแปร	ขนาด (bits)	ขอบเขต	ข้อมูลที่เก็บ
char	8	-128 ถึง 127	ข้อมูลชนิดอักขระ ใ้เนื้อที่ 1 byte
unsigned char	8	0 ถึง 255	ข้อมูลชนิดอักขระ ไม่คิดเครื่องหมาย
int	16	-32,768 ถึง 32,767	ข้อมูลชนิดจำนวนเต็ม ใ้เนื้อที่ 2 byte
unsigned int	16	0 ถึง 65,535	ข้อมูลชนิดจำนวนเต็ม ไม่คิดเครื่องหมาย
short	8	-128 ถึง 127	ข้อมูลชนิดจำนวนเต็มแบบสั้น ใ้เนื้อที่ 1 byte
unsigned short	8	0 ถึง 255	ข้อมูลชนิดจำนวนเต็มแบบสั้น ไม่คิดเครื่องหมาย
long	32	-2,147,483,648 ถึง 2,147,483,649	ข้อมูลชนิดจำนวนเต็มแบบยาว ใ้เนื้อที่ 4 byte
unsigned long	32	0 ถึง 4,294,967,296	ข้อมูลชนิดจำนวนเต็มแบบยาว ไม่คิดเครื่องหมาย
float	32	$3.4 \times 10e(-38)$ ถึง $3.4 \times 10e(38)$	ข้อมูลชนิดเลขทศนิยม ใ้เนื้อที่ 4 byte
double	64	$3.4 \times 10e(-308)$ ถึง $3.4 \times 10e(308)$	ข้อมูลชนิดเลขทศนิยม ใ้เนื้อที่ 8 byte
long double	128	$3.4 \times 10e(-4032)$ ถึง $1.1 \times 10e(4032)$	ข้อมูลชนิดเลขทศนิยม ใ้เนื้อที่ 16 byte

รูปที่ 6-3 ตารางเปรียบเทียบขนาดบิต ขอบเขตค่าตัวเลข และลักษณะการจัดเก็บข้อมูลของตัวแปรชนิดต่างๆ ในสถาปัตยกรรมคอมพิวเตอร์

2.1 การแทนค่าตัวเลขจำนวนเต็ม (Integer Representation)



Unsigned Representation: เป็น

วิธีการจัดเก็บตัวเลขจำนวนเต็มบวก

เท่านั้น ไม่มีบิตสำหรับเครื่องหมาย

ทุกบิตใช้แทนค่าตัวเลขอย่างเดียว

ตัวอย่าง: หากใช้ 8 บิต, เลข 5 ใน

Unsigned คือ 00000101₂ (0-

255) เปรียบเสมือนการนับ "จำนวน

ของ" สิ่งของที่ไม่ต้องสนใจว่ามีติดลบ

หรือไม่

Sign-Magnitude

Representation: การจัดเก็บ

ตัวเลขจำนวนเต็มที่มีเครื่องหมาย

โดยบิตซ้ายสุด (Most Significant

Bit - MSB) จะถูกใช้เป็น "บิต

รูปที่ 6-4 กลไกการแทนค่าจำนวนเต็มแบบไม่คิดเครื่องหมาย (Unsigned)

และแบบใช้บิตเครื่องหมาย (Sign-Magnitude)

เครื่องหมาย" (Sign Bit) โดย 0 แทนค่าบวก และ 1 แทนค่าลบ ส่วนบิตที่เหลือใช้แทนขนาดของตัวเลข

ตัวอย่าง: หากใช้ 8 บิต:

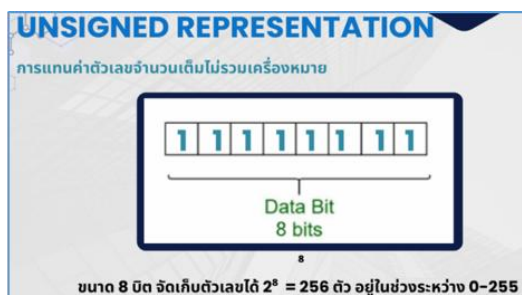
+5 บวกห้า คือ 00000101₂,

ส่วน -5 ลบห้า คือ 10000101₂

- เก็บเป็นเลขฐาน 2 - เก็บเป็นตัวเลขแบบคิดเครื่องหมาย (Sign-Magnitude Representation) - เก็บเป็นตัวเลขแบบไม่คิดเครื่องหมาย (Unsigned Representation)	Signed value 1 1 1 1 1 1 1 1 = -1 Unsigned value 1 1 1 1 1 1 1 1 = 255
---	--

รูปที่ 6-5 การเปรียบเทียบผลลัพธ์การตีความชุดข้อมูลเลขฐานสองขนาด 8 บิต

ระหว่างระบบคิดเครื่องหมาย (Signed) และระบบไม่คิดเครื่องหมาย (Unsigned)



รูปที่ 6-6 โครงสร้างการจัดเก็บข้อมูลแบบไม่

คิดเครื่องหมายขนาด 8 บิต (Unsigned)

ข้อจำกัด: มีปัญหาเรื่องค่าศูนย์สองแบบ (+0 และ -0) และการดำเนินการทางคณิตศาสตร์ซับซ้อน



รูปที่ 6-7 โครงสร้างการจัดเก็บข้อมูลแบบคิด

เครื่องหมาย (Sign-Magnitude) ขนาด 8 บิต

2.2 การทำ 1's Complement และ 2's Complement เพื่อใช้งานกับเลขฐาน 2

2.2.1 เลข 1's Complement: เป็นวิธีที่ใช้ในการหาค่าลบของเลขฐานสอง โดยการ "กลับบิต" ทั้งหมดของเลขนั้น จาก 0 เป็น 1 และจาก 1 เป็น 0

ตัวอย่าง: หากใช้ 8 บิต :เลข 5 คือ 00000101_2 ดังนั้น -

เลข 5 ในรูปแบบ 1's Complement คือ 11111010_2 (กลับบิตทั้งหมด)

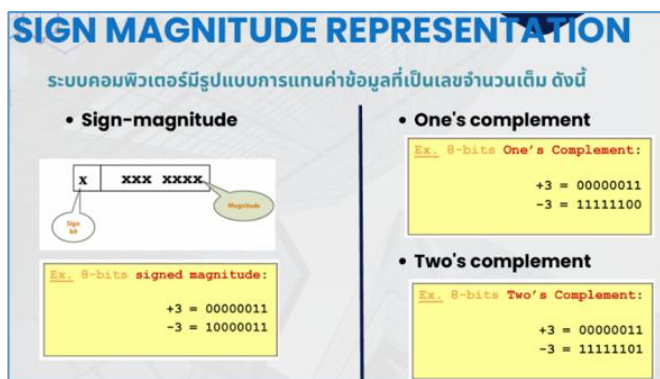
ข้อจำกัด: ยังคงมีปัญหาเรื่องค่าศูนย์สองแบบ

2.2.2 เลข 2's Complement: เป็นวิธีที่นิยมและใช้งานมากที่สุดในการแทนค่าลบของเลขฐานสองในระบบคอมพิวเตอร์ เนื่องจากแก้ปัญหาเรื่องศูนย์สองแบบและทำให้การดำเนินการบวก/ลบง่ายขึ้น (ใช้เพียงวงจรบวก) วิธีการคือ หา 1's Complement ก่อน แล้วบวก 1

ตัวอย่าง: หากใช้ 8 บิต: 5 คือ 00000101_2

1's Complement ของ 5 คือ 11111010_2

2's Complement ของ 5 (-5) คือ $11111010_2 + 1_2 = 11111011_2$



ตัวอย่างเปรียบเทียบ: 2's Complement

เปรียบเสมือน "ระบบบัญชี" ที่ทำให้การบันทึก
ทั้งรายรับ (บวก) และรายจ่าย (ลบ) สามารถ
ใช้หลักการคำนวณเดียวกันได้ ทำให้ง่ายต่อ
การจัดการ

รูปที่ 6-8 การเปรียบเทียบรูปแบบการแทนค่าจำนวนเต็ม
ลบในระบบเลขฐานสอง: Sign-Magnitude,
One's Complement และ Two's Complement

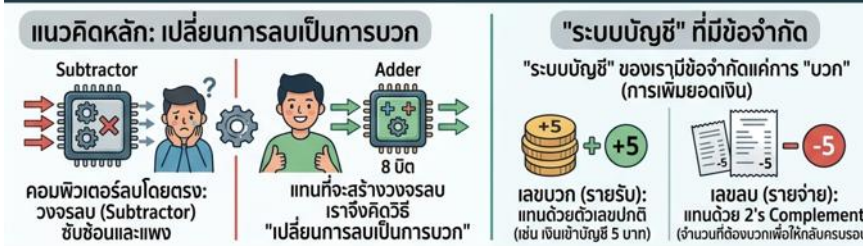
อธิบายแนวคิดเรื่อง 2's Complement ในเชิงเปรียบเทียบ "ระบบบัญชี" ให้เข้าใจง่ายขึ้นดังนี้

แนวคิดของ 2's Complement

โดยพื้นฐานแล้ว 2's Complement คือวิธีการแปลงเลขฐานสองบวกให้กลายเป็นเลขฐานสองลบ โดยมีหลักการสำคัญคือ การใช้การบวกเพื่อทำการลบ

ในโลกของคอมพิวเตอร์ การสร้างวงจรสำหรับการบวก (Adder) นั้นง่ายและประหยัดกว่าการสร้างวงจรสำหรับการลบ (Subtractor) แยกต่างหากมาก ดังนั้น แทนที่จะสร้างวงจรลบขึ้นมาใหม่ เราจึงคิดวิธี "เปลี่ยนการลบเป็นการบวก" แทน โดยใช้หลักการของ 2's Complement

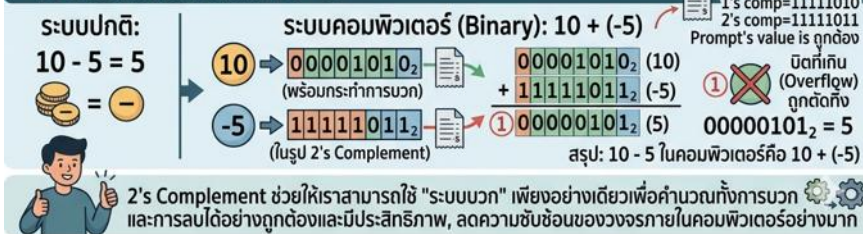
อธิบายแนวคิดเรื่อง 2's Complement ในเชิงเปรียบเทียบ "ระบบบัญชี" ให้เข้าใจง่าย



3. วงรอบ (Cycle) และการคำนวณ Complement



4. ตัวอย่างการคำนวณ: 10 - 5



รูปที่ 6-9 รูปแบบการแทนค่าจำนวนเต็มลบ: Sign-Magnitude, One's Complement และ Two's Complement

2.3 เปรียบเทียบกับ "ระบบบัญชี"

ลองจินตนาการว่า "ระบบบัญชี" ของเรามีข้อจำกัดแค่การ "บวก" (การเพิ่มยอดเงิน) เท่านั้น แต่เราต้องการบันทึก "รายจ่าย" (การหักยอดเงิน) ด้วย

เลขบวก (รายรับ): แทนด้วยตัวเลขปกติ เช่น เงินเข้าบัญชี 5 บาท คือ 5

เลขลบ (รายจ่าย): แทนด้วย 2's Complement ซึ่งก็คือ "จำนวนที่ต้องบวกเพิ่มเพื่อให้กลับไปครบวงรอบ" และ ในระบบเลขฐานสอง ถ้าเราใช้ 8 บิต วงรอบของมันคือ 256 ตัว (จาก 0 ถึง 255) การลบ 5: แทนที่จะ "ลบ" เราจะ "บวก"

ด้วยเลข 2's Complement ของ 5 ซึ่งก็คือ $256 - 5 = 251$

ดังนั้น ในระบบบัญชีของเรา การ "จ่าย" เงิน 5 บาท ไม่ได้บันทึกว่า -5 แต่บันทึกเป็นการ "เพิ่ม" ยอดเงิน 251 บาทแทน ซึ่งในระบบ 8 บิต เมื่อนับเลย 255 ไป ตัวเลขก็จะกลับมาเริ่มที่ 0 ใหม่พอดี

ตัวอย่าง: การคำนวณ $10 - 5$ ในระบบปกติ เราคำนวณ $10 - 5 = 5$

ในระบบคอมพิวเตอร์ที่ใช้ 2's Complement เราคำนวณ $10 + (-5)$: ดังนั้น

เลข 10 คือ 00001010_2 และ กระทำการบวกกับ -5 (ในรูป 2's Complement) คือ 11111011_2

เมื่อนำมาบวกกัน:

$$00001010_2 (10) + 11111011_2 (-5) = \text{เท่ากับ } (1)00000101_2 (5)$$

บิตที่เกินออกมา (ตัวเลข 1 ในวงเล็บ) จะถูกตัดทิ้ง ทำให้ผลลัพธ์ที่ได้คือ 00000101_2 ซึ่งก็คือ 5 นั่นเอง

สรุปคือ 2's Complement ช่วยให้เราสามารถใช้ "ระบบบวก" เพียงอย่างเดียว เพื่อคำนวณทั้งการบวกและการลบได้อย่างถูกต้องและมีประสิทธิภาพ ลดความซับซ้อนของวงจรภายในคอมพิวเตอร์ได้อย่างมาก

3. หลักการบวกและการลบเลขฐานสอง (Binary Addition and Subtraction)

การบวกเลขฐานสอง: คล้ายกับการบวกเลขฐานสิบ แต่มีกฎง่ายๆ เพียงแค่ $0+0=0$, $0+1=1$, $1+0=1$,

$1+1=0$ (ทด 1) ตัวอย่าง: 0101_2 (5) + 0011_2 (3) - เท่ากับ - 1000_2 (8)

* การลบเลขฐานสองโดยใช้วิธี 2's Complement: นี่คือการมาตรฐานที่ ALU ใช้ในการลบ หลักการ

คือ "เปลี่ยนการลบเป็นการบวก" โดยการบวกตัวตั้งกับ 2's Complement ของตัวลบ

* ตัวอย่าง: 5 - 3 (ใน 4 บิต) เลข 5 คือ 0101_2

กระบวนการทำให้ เลข 3 ในรูปแบบ 2's Complement:

แปลงเป็นเลขฐาน 2 เพราะฉะนั้นเลข 3 คือ 0011_2

กระบวนการที่ 1 ทำ 1's Complement คือ 1100_2

กระบวนการที่ 2 ทำ 2's Complement คือ 1101_2

นำมาบวกกัน: 0101_2 (5) + 1101_2 (-3)

ตอบคือ $1\ 0010_2$ (ตัดตัวทศออก)

เหลือ 0010_2 (แปลงเป็นเลขฐาน 10 ซึ่งคือ 2)

Binary Addition

Fundamental Rules

$0+0=0$
 $0+1=1$
 $1+0=1$
 $1+1=0$ (carry 1)



Example:

$01012\ (5)$
 $+ 00012\ (3)$

 $-10002\ (8)$

2's Complement Subtraction

1's Complement: 111002

$00112\ (3)$

$00112\ 11002$

2's Complement: 11012 (-3)

$+ 11012\ (-3)$

$01012\ (5)$

$+ 11012\ (-3)$

$-1\ 00102\ (2)$

Analogy for



Instead of walking backward 3 steps, you turn around and walk forward 3 steps

รูปที่ 6-10 อธิบายแนวคิด 2's Complement ผ่านการเปรียบเทียบกับระบบบัญชีและการหมุนเวียนของวงรอบข้อมูล

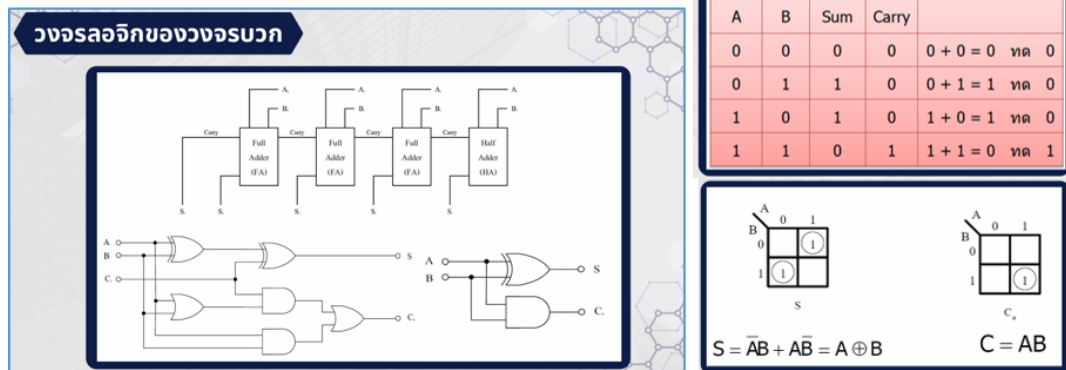
ตัวอย่างเปรียบเทียบ: การลบด้วย 2's Complement เหมือนกับการที่ "แทนที่จะเดินถอยหลังไป 3 ก้าว เราเปลี่ยนเป็นการหันหลังกลับแล้วเดินไปข้างหน้า 3 ก้าว" ซึ่งให้ผลลัพธ์เดียวกันแต่เราใช้การเดินไปข้างหน้าอย่างเดียว

การบวกเลขสองจำนวน		การบวกเลขสองจำนวนกับค่าลบ (มีบิตเครื่องหมาย)	
บวกค่าเลขบวกสองค่าเข้าด้วยกัน เช่น การบวกค่า +9 กับ +4	บวกค่าเลขลบสองค่าเข้าด้วยกัน เช่น การบวกค่า -9 กับ -4	การบวกกันระหว่างเลข +9 กับ -4	การบวกกันระหว่างเลข -9 กับ +4
แบบไม่มีบิตเครื่องหมาย	แบบมีบิตเครื่องหมาย	มีบิตเครื่องหมาย	มีบิตเครื่องหมาย
$\begin{array}{r} (+9) \ 01001 \\ (+4) \ 00100 \\ \hline (+13) \ 01101 \end{array}$	$\begin{array}{r} (+9) \ 01001 \\ (+4) \ 00100 \\ \hline \ 01101 \end{array}$	$\begin{array}{r} (+9) \ 01001 \\ (-4) \ 11100 \\ \hline \ 0101 \end{array}$	$\begin{array}{r} (-9) \ 10111 \\ (+4) \ 00100 \\ \hline \ 1011 \end{array}$
- ถ้ามีตัวทด ให้ตัดตัวทศออก ผลลัพธ์จะมีค่าเป็นบวก (บิตเครื่องหมายเป็นศูนย์) - คำตอบ = +5	- ถ้ามีตัวทด ให้ตัดตัวทศออก ผลลัพธ์จะมีค่าเป็นลบ (บิตเครื่องหมายเป็นหนึ่ง) - คำตอบ = -5	- ถ้ามีตัวทด ให้ตัดตัวทศออก ผลลัพธ์จะมีค่าเป็นบวก (บิตเครื่องหมายเป็นศูนย์) - คำตอบ = +5	- ถ้ามีตัวทด ให้ตัดตัวทศออก ผลลัพธ์จะมีค่าเป็นลบ (บิตเครื่องหมายเป็นหนึ่ง) - คำตอบ = -5
การลบเลขสองจำนวน (มีบิตเครื่องหมาย)		เลขเดียวกันเครื่องหมายต่างกัน (มีบิตเครื่องหมาย)	
การบวกกันระหว่างเลข -9 กับ -4		การบวกกันระหว่างเลข -9 กับ 9	
$\begin{array}{r} (-9) \ 10111 \\ (-4) \ 11100 \\ \hline \ 11011 \end{array}$		$\begin{array}{r} (-9) \ 10111 \\ (+9) \ 01001 \\ \hline \ 0000 \end{array}$	
- ถ้ามีตัวทด ให้ตัดตัวทศออก ผลลัพธ์จะมีค่าเป็นลบ (บิตเครื่องหมายเป็นหนึ่ง) - คำตอบ = -13		- ถ้ามีตัวทด ให้ตัดตัวทศออก ผลลัพธ์จะมีค่าเป็นบวก (บิตเครื่องหมายเป็นศูนย์) - คำตอบ = 0	

รูปที่ 6-11 ตัวอย่างการดำเนินการบวกเลขฐานสอง พร้อมการจัดการบิตเครื่องหมายและบิตทด

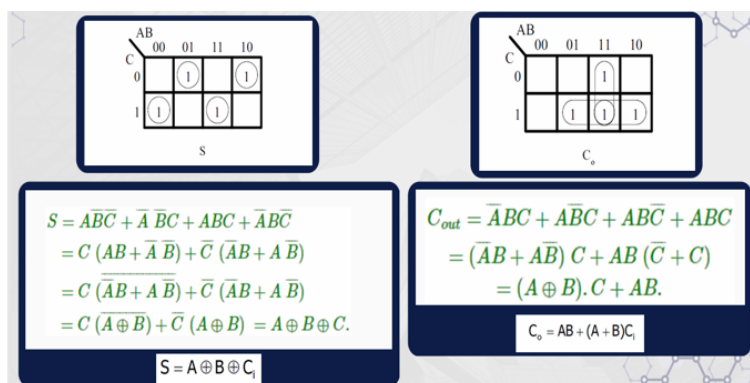
3.1 การบวกคอมพิวเตอร์ด้วย การบวกของวงจร Logic

วงจรลอจิกของวงจรบวก



รูปที่ 6-12 วิวัฒนาการจากการคำนวณในสมองสู่การออกแบบวงจรไฟฟ้า

3.1.1 ตารางความจริงและกฎการบวก (The Rulebook) "กฎเหล็ก" ของการบวกเลขฐานสอง 1 บิต:

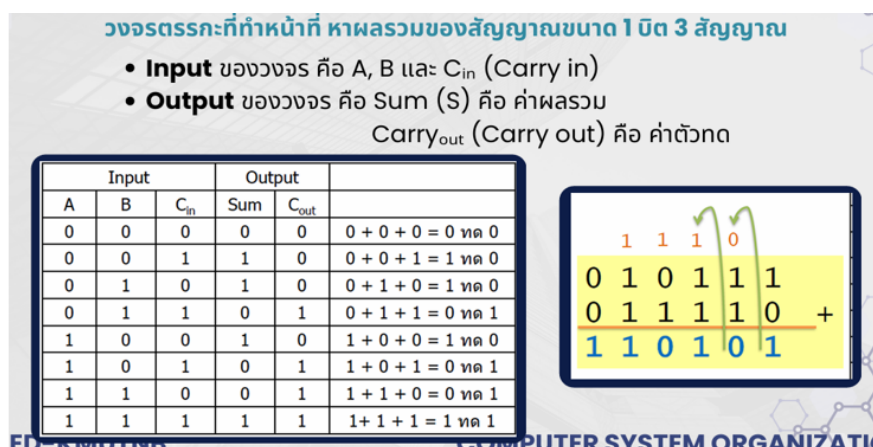


รูปที่ 6-13 กระบวนการบวกเลขฐานสอง: จากแนวคิดเลขฐานสิบสู่การทำงานของวงจร Full Adder และ 4-bit Ripple Carry Adder

- ตารางความจริง (Truth Table): บอกเราว่าถ้าเอา A มาบวกกับ B จะได้ผลลัพธ์เป็น Sum (ผลบวก) และ Carry (ตัวทด) อย่างไร
 - 0 + 0 = 0 ทด 0
 - 1 + 1 = 0 ทด 1 (เพราะ 1+1 ในเลขฐานสองคือ 10₂)

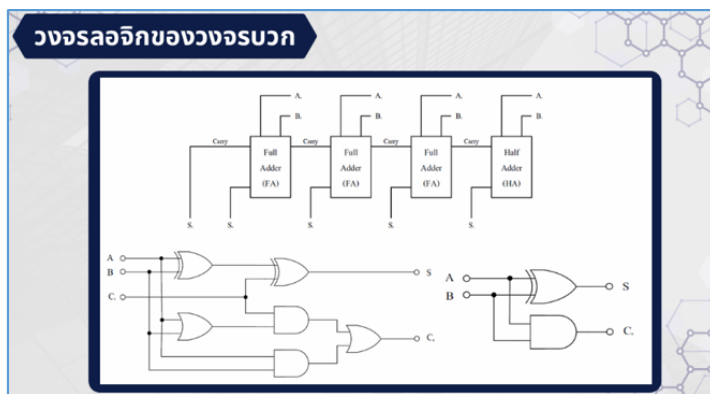
- สมการลอจิก: จากตารางนี้ เราจะได้สูตรทางไฟฟ้าคือ:

- Sum (S): ใช้เกต XOR ($A \oplus B$) เพราะมันจะได้ค่าเป็น 1 ก็ต่อเมื่อ input ต่างกันเท่านั้น
- Carry (C): ใช้เกต AND (AB) เพราะมันจะเกิดตัวทอก็ต่อเมื่อทั้ง A และ B เป็น 1 พร้อมกัน



รูปที่ 6-14 ตารางความจริงของวงจรบวกเต็มรูปแบบ (Full Adder Truth Table) และการบวกเลขฐานสองแบบ 6 บิต พร้อมการแสดงบิตทด

3.3 การสร้างบล็อกคำนวณ: Half Adder และ Full Adder คือการเอาสูตรข้างบนมาสร้างเป็นวงจรจริง



รูปที่ 6-15 การใช้ผังคาร์โนห์ (Karnaugh Map) เพื่อการลดรูปสมการ
ลอจิกของผลรวม (Sum) และตัวทด (Carry Out) ในวงจรบวกเต็มรูปแบบ

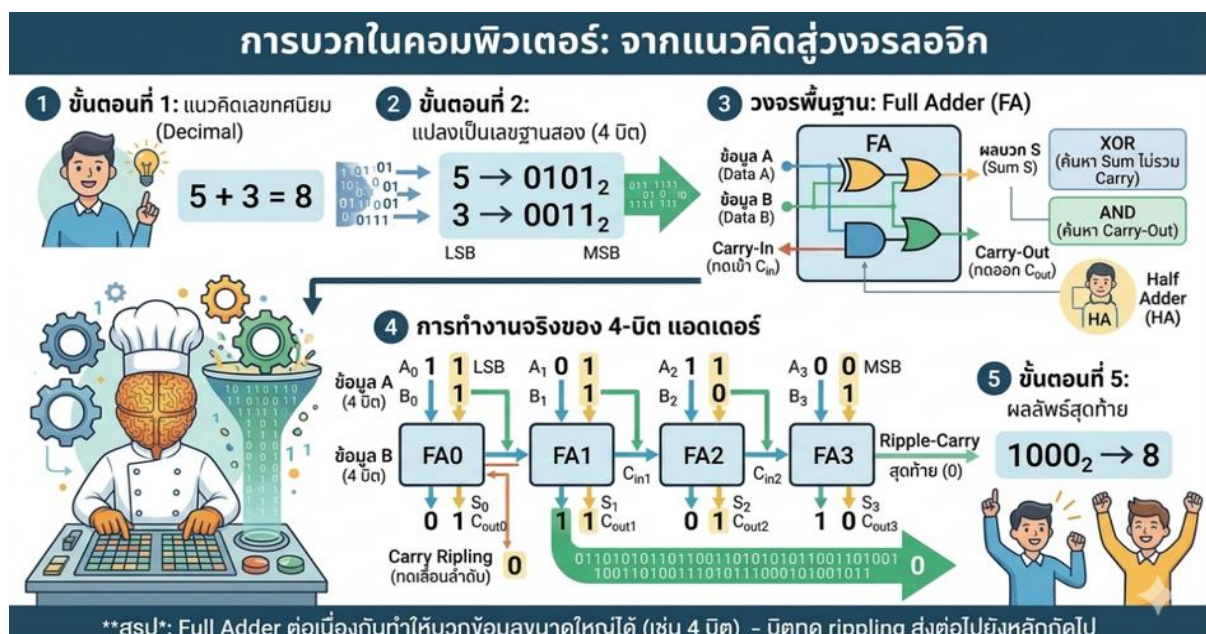
- ภายใน FA ประกอบด้วย XOR, AND และ OR เพื่อจัดการกับข้อมูลทั้ง 3 ตัว (A, B, C_{in}) ให้กลายเป็นผลลัพธ์ที่ถูกต้อง

3.4 การทำงานร่วมกันแบบ 4-บิต (The Machine in Action)

เมื่อเราต้องการบวกเลขที่มากกว่า 1 หลัก (เช่น $5 + 3$ ในรูป 6-12) คอมพิวเตอร์จะต้องวงจรเหล่านี้เข้าด้วยกันเป็นทอด ๆ เรียกว่า **Ripple Carry Adder**:

ขั้นตอนการคำนวณ (ตัวอย่าง $5 + 3$):

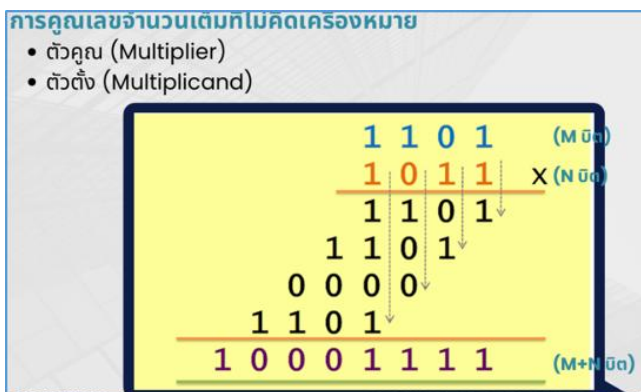
1. แปลงเลข: 5 กลายเป็น 0101_2 และ 3 กลายเป็น 0011_2
2. หลักที่ 0 (ขวาสุด): ใช้ HA หรือ FA ตัวแรกบวก $1 + 1$ ได้ 0 ทด 1 ส่งไปหลักถัดไป
3. หลักที่ 1 ถึง 3: ตัวทดจะถูกส่งต่อ (Ripple) ไปยังวงจร FA ตัวถัดไปเรื่อย ๆ เหมือนโดมิโน
4. ผลลัพธ์สุดท้าย: เมื่อบวกครบทุกบิต จะได้คำตอบออกมาเป็น 1000_2 ซึ่งก็คือเลข 8 ในฐานสิบ



รูปที่ 6-16 การพัฒนางจรลอจิกของระบบบวกเลขฐานสอง: การต่อขยายวงจร Full Adder แบบขนาน และโครงสร้างเกิดภายในของ Half/Full Adder

4. หลักการคูณเลขฐานสอง (Binary Multiplication)

การคูณเลขจำนวนเต็มแบบไม่มีเครื่องหมายในระบบเลขฐานสอง มีหลักการคล้ายกับการคูณยาวในระบบเลขฐานสิบโดยมีขั้นตอนดังนี้:



รูปที่ 6-17 ตัวอย่างการคูณเลขจำนวนเต็มแบบไม่มีเครื่องหมาย (Unsigned Integer Multiplication) และผลลัพธ์ในรูปแบบบิตขยาย (M+N bits)

ระบบเลขฐานสิบโดยมีขั้นตอนดังนี้:

1. นำตัวคูณแต่ละบิตไปคูณกับตัวตั้งทีละบิต: เริ่มจากบิตขวาสุดของตัวคูณ คูณกับตัวตั้งทั้งหมด
2. เลื่อนผลลัพธ์ย่อยไปทางซ้าย: ในแต่ละครั้งที่คูณด้วยบิตถัดไปของตัวคูณ (เลื่อนจากขวาไปซ้าย) ผลลัพธ์ย่อยที่ได้จะต้องเลื่อนไปทางซ้าย 1 ตำแหน่งเสมอ เหมือนกับการคูณยาวในฐานสิบ
3. นำผลลัพธ์ย่อยทั้งหมดมาบวกกัน: เมื่อได้ผลลัพธ์ย่อยครบทุกบิตของตัวคูณแล้วให้นำผลลัพธ์ย่อยเหล่านั้นมาบวกกันตามหลักการบวกเลขฐานสอง

4.1 การคูณเลขฐานสอง (Unsigned Integer Multiplication)

โจทย์กำหนดให้หาผลลัพธ์การคูณ ของ 13×11

ขั้นตอนที่ 1: แปลงเลขฐานสิบเป็นเลขฐานสองโดย เลข $13_{10} = 1101_2$ และ เลข $11_{10} = 1011_2$

ขั้นตอนที่ 2: ทำการคูณแบบคูณยาว

เริ่มจากบิตขวาสุดของตัวคูณ ($(11_{10}) = (1011_2)$) ไปคูณกับตัวตั้ง ($(13_{10}) = (1101_2)$) ทีละบิต และนำผลลัพธ์ย่อยมาบวกกัน

2.1 นำบิตแรก (ขวาสุด) ของตัวคูณ คือ 1 มาคูณกับตัวตั้ง 1101

- $1101 \times 1 = 1101$ (นี่คือผลลัพธ์ย่อยบรรทัดแรก)

2.2 นำบิตที่สองของตัวคูณ คือ 1 มาคูณกับตัวตั้ง 1101

- $1101 \times 1 = 1101$
- จากนั้น เลื่อนผลลัพธ์ย่อยนี้ไปทางซ้าย 1 ตำแหน่ง
- เราจะได้ 11010

2.3 นำบิตที่สามของตัวคูณ คือ 0 มาคูณกับตัวตั้ง 1101

- $1101 \times 0 = 0000$
- เลื่อนผลลัพธ์ย่อยนี้ไปทางซ้าย 2 ตำแหน่ง
- เราจะได้ 000000

2.4 นำบิตที่สี่ (ซ้ายสุด) ของตัวคูณ คือ 1 มาคูณกับตัวตั้ง 1101

- $1101 \times 1 = 1101$
- เลื่อนผลลัพธ์ย่อยนี้ไปทางซ้าย 3 ตำแหน่ง
- เราจะได้ 1101000

ขั้นตอนที่ 3: นำผลลัพธ์ย่อยทั้งหมดมาบวกกัน

ในการบวกเลขฐานสอง เราจะบวกทีละหลัก

เหมือนกับการบวกเลขฐานสิบ โดยที่ผลลัพธ์ของการบวกในแต่ละหลักจะเป็นไปตามกฎดังนี้:

- $0 + 0 = 0$ และ $0 + 1 = 1$ และ $1 + 0 = 1$
- $1 + 1 = 10$ (ได้ 0 ทดไป 1) และ $1 + 1 + 1 = 11$ (ได้ 1 ทดไป 1)

คำตอบที่ได้มาคือ $1101 + 11010 + 000000 + 1101000$ และกำหนดให้ความกว้างของเลขทุกชุดเป็น 8 บิต เพื่อความถูกต้องและเป็นระบบในการคำนวณ อาจารย์ขอปรับตัวเลขทั้งหมดให้อยู่ในรูปแบบ 8 บิต ก่อน ดังนี้:

- 1101_2 จะเป็น 00001101_2
- 11010_2 จะเป็น 00011010_2
- 00000_2 จะเป็น 0000000_2
- 1101000_2 จะเป็น 01101000_2

$$\begin{array}{r} 00001101 \quad (13_{10}) \\ 00011010 \quad (26_{10}) \\ 00000000 \quad (0_{10}) \\ + 01101000 \quad (104_{10}) \\ \hline 10001111 \end{array}$$

เริ่มการบวกจากตัวเลขที่มีจำนวนบิตน้อยที่สุดไปหาตัวเลขที่มีจำนวนบิตมากที่สุด หรือจะบวกพร้อมกันทั้งหมดก็ได้ หลังจากนั้นนำผลลัพธ์ย่อยที่ได้จากขั้นตอนที่ 3 มาบวกกันทั้งหมด:

(ขั้นตอนแปลงผลลัพธ์กลับเป็นเลขฐานสิบเพื่อตรวจสอบ)

$$10001111_2 = (1 * 2^7) + (0 * 2^6) + (0 * 2^5) + (0 * 2^4) + (1 * 2^3) + (1 * 2^2) + (1 * 2^1) + (1 * 2^0)$$

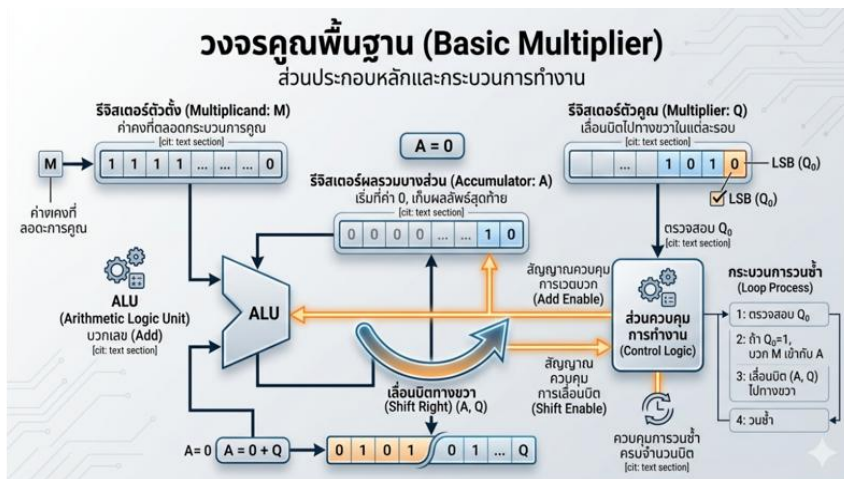
$$143_{10} = 128 + 0 + 0 + 0 + 8 + 4 + 2 + 1$$

ตรวจสอบ: $13 * 11 = 143$

ดังนั้น ผลลัพธ์ของการคูณเลขฐานสอง 1101_2 ด้วย 1011_2 คือ 10001111_2 ซึ่งเท่ากับ 143 ในระบบเลขฐานสิบ

การคูณเลขฐานสองในระดับฮาร์ดแวร์ จะใช้วงจรที่ทำงานวนซ้ำ (iterative process) โดยอาศัยหลักการของการบวก (Add) และการเลื่อนบิต (Shift) ซึ่งเป็นพื้นฐานเดียวกันกับการคูณยาวที่เราได้เรียนรู้ไปแล้วนั่นเอง

4.2 ส่วนประกอบหลักของฮาร์ดแวร์สำหรับการคูณ



วงจรคูณพื้นฐาน (Basic Multiplier) ประกอบไปด้วย รีจิสเตอร์หลักๆ ดังนี้:

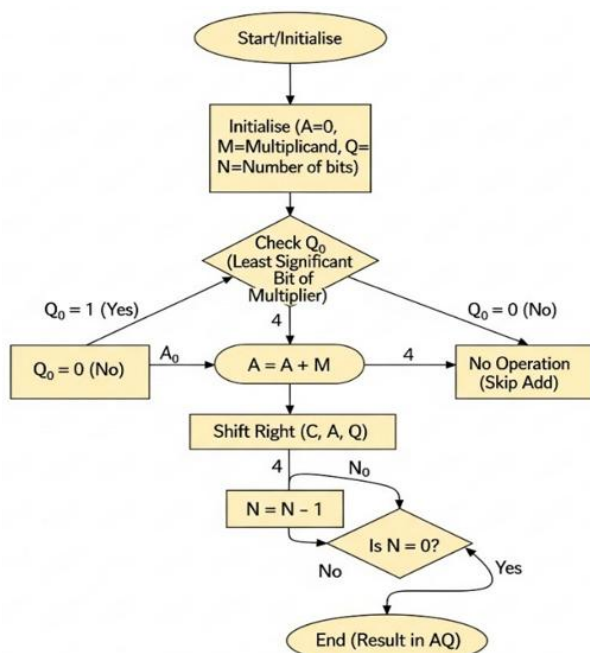
1. Multiplicand (M):
 รีจิสเตอร์สำหรับเก็บ ตัวตั้ง
 (ค่าคงที่ตลอดกระบวนการคูณ)
2. Multiplier (Q): รีจิสเตอร์
 สำหรับเก็บ ตัวคูณ ซึ่งจะมีการ

รูปที่ 6-18 แผนผังโครงสร้างและกระบวนการทำงานของฮาร์ดแวร์วงจรคุณพื้นฐาน เลื่อนบิตไปทางขวาในแต่ละรอบ

3. **Accumulator (A):** รีจิสเตอร์สำหรับเก็บ *ผลรวมบางส่วน* (Partial Product) และผลลัพธ์สุดท้าย เริ่มต้นด้วยค่า 0 นอกจากนี้ ยังมีส่วนประกอบอื่นๆ ที่สำคัญ เช่น: ALU (Arithmetic Logic Unit): ทำหน้าที่ในการบวกเลข (Add)

4.Control Logic: ส่วนควบคุมการทำงานวงจร กำหนดว่าจะบวกหรือเลื่อนบิต และควบคุมการวนซ้ำ

4.3 หลักการทำงานของวงจรคุณพื้นฐาน (Flowchart) : คำอธิบาย Flowchart และหลักการทำงาน:



4.3.1 Start / Initialise (เริ่มต้น / กำหนดค่าเริ่มต้น):

- กำหนดค่าเริ่มต้นให้รีจิสเตอร์ A (Accumulator) เป็น 0
 - กำหนดค่าตัวตั้ง (Multiplicand) ไปยังรีจิสเตอร์ M
 - กำหนดค่าตัวคูณ (Multiplier) ไปยังรีจิสเตอร์ Q
 - กำหนดค่า N ซึ่งคือจำนวนบิตของตัวคูณ (หรือตัวตั้งก็ได้)
- เพื่อใช้ในการนับรอบการทำงาน

Check Q_0 (ตรวจสอบบิต Q_0):

ในแต่ละรอบการทำงาน (Iteration) ระบบจะตรวจสอบบิต
 ขวาสุด (Least Significant Bit หรือ LSB) ของรีจิสเตอร์ Q
 (ตัวคน)

ถ้า $Q_0 = 1$: หมายความว่าบิตปัจจุบันของคุณเป็น '1'
ซึ่งตามหลักการคุณยาว เราจะต้องนำตัวตั้ง (M) มาบวกเข้า
กับผลรวมบางส่วนในรีจิสเตอร์ A ($A = A + M$)

ถ้า $Q_0 = 0$: หมายความว่าปัจจุบันของคุณเป็น '0' เราไม่จำเป็นต้องบวกตัวตั้ง เพราะคุณด้วย '0' ได้ '0' จึงข้ามขั้นตอนนี้ไป

รูปที่ 6-19 ผังงานแสดงอัลกอริทึมการคูณเลขจำนวนเต็มแบบไม่มีเครื่องหมายด้วยวิธีการบวกและเลื่อนบิต

4.3.2 Shift Right (C, A, Q) (เลื่อนบิตไปทางขวา):

- หลังจากขั้นตอนการบวก (หรือไม่บวก) แล้ว จะทำการ **Arithmetic Shift Right** (เลื่อนบิตทางขวาแบบรักษาสัญญาณ) พร้อมกันทั้งรีจิสเตอร์ A และ Q โดยบิตที่หลุดจาก Q ทางขวาจะหายไป และบิตที่หลุดจาก A ทางขวา จะย้ายไปเป็นบิตซ้ายสุดของ Q
- โดยทั่วไป, C (Carry bit) อาจจะถูกใช้เพื่อรองรับบิตที่เลื่อนหลุดจาก A ไปยัง Q เพื่อให้ผลลัพธ์ไม่คลาดเคลื่อน แต่ในวงจรพื้นฐานบางแบบอาจรวม C เข้าไปใน A ด้วย
- การเลื่อนบิตนี้เป็นสิ่งสำคัญที่จำลองการเลื่อนผลลัพธ์ย่อยไปทางซ้ายในการคูณยาวด้วยมือ แต่ในฮาร์ดแวร์ เราจะเลื่อน *ตัวคูณ* ไปทางขวา และ *ผลรวมบางส่วน* (A) กับ *ส่วนที่เหลือของตัวคูณ* (Q) ไปทางขวาพร้อมกัน ซึ่งให้ผลลัพธ์เทียบเท่ากัน

4.3.3 $N = N - 1$ (ลดจำนวนรอบ):

- ลดค่าตัวนับรอบ (N) ลง 1

4.3.4 Is N = 0? (ตรวจสอบว่าครบจำนวนรอบหรือไม่):

- ถ้า N ยังไม่เป็น 0 ให้กลับไปทำขั้นตอนที่ 2 (Check Q_0) อีกครั้ง
- ถ้า N เป็น 0 หมายความว่าได้ดำเนินการครบทุกบิตของตัวคูณแล้ว

4.3.5 End (จบการทำงาน):

- เมื่อการทำงานสิ้นสุดลง ผลลัพธ์ของการคูณจะอยู่ที่การรวมกันของรีจิสเตอร์ A และ Q (AQ) โดย A จะเป็นส่วนที่อยู่ซ้ายมือ (บิตที่มีนัยสำคัญสูงกว่า) และ Q จะเป็นส่วนที่อยู่ขวามือ (บิตที่มีนัยสำคัญน้อยกว่า)

4.4 การคูณเลขจำนวนเต็มแบบไม่คิดเครื่องหมายด้วยการทำงานฮาร์ดแวร์ (ยกตัวอย่างหาผลคูณของ $3 \times 2 =$ เท่าไหร่) และกำหนดการแปลงเลขฐานสิบเป็นฐานสอง (กำหนดขนาด 4 บิต):

การคูณเลขฐานสองด้วยฮาร์ดแวร์ไม่ได้ใช้การคูณตรง ๆ แต่ใช้วิธีวนซ้ำ (Iterative Process) ที่อาศัยแค่การ บวก (Add) และการ เลื่อนบิต (Shift) โดยมี ส่วนประกอบหลักเริ่มต้นที่ใช้ในวงจร:

- **A (Accumulator Register):** รีจิสเตอร์สำหรับสะสมผลบวก มีการกำหนดค่าเริ่มต้นเป็น 0000
- **Q (Multiplier Register):** รีจิสเตอร์ที่เก็บค่าตัวคูณ มีการกำหนดค่าเริ่มต้นเป็น 0010
- **M (Multiplicand Register):** รีจิสเตอร์ที่เก็บค่าตัวตั้ง มีการกำหนดค่าเริ่มต้นเป็น 0011
- **N (Counter):** ตัวนับจำนวนรอบการทำงาน โดยจะนับลดลงทีละรอบ
- **Multiplicand (M) = $3_{10} = 0011_2$, Multiplier (Q) = $2_{10} = 0010_2$ และจำนวน $N = 4$ (จำนวนบิต)**

สถานะเริ่มต้น: $A = 0000$ $Q = 0010$ $M = 0011$
 $N = 4$ (เท่ากับจำนวนบิตของเลขฐาน 2)

รอบ (Iteration)	N	Q ₀ (LSB of Q)	การกระทำ (Action)	A (Accumulator)	Q (Multiplier)	ผลลัพธ์ AQ (ถ้า สิ้นสุด)
เริ่มต้น	4	-	Initialise	0000	0010	
1	3	0	No Add (Q ₀ =0)	0000	0010	
			Shift Right (A,Q)	0000	0001	
2	2	1	A = A + M (0000+0011)	0011	0001	
			Shift Right (A,Q)	0001	1000	
3	1	0	No Add (Q ₀ =0)	0001	1000	
			Shift Right (A,Q)	0000	1100	
4	0	0	No Add (Q ₀ =0)	0000	1100	
			Shift Right (A,Q)	0000	0110	
สิ้นสุด	0	-				00000110 (AQ)

ตารางที่ 6-1 การจำลองขั้นตอนการทำงานของอัลกอริทึมการคูณเลขจำนวนเต็มแบบไม่มีเครื่องหมาย

ผลลัพธ์สุดท้าย: เมื่อ $N = 0$ ผลลัพธ์ที่ได้คือ $AQ = 00000110_2$

แปลงกลับเป็นเลขฐานสิบ:

$$00000110_2 = (0 \times 2^7) + (0 \times 2^6) + (0 \times 2^5) + (0 \times 2^4) + (0 \times 2^3) + (1 \times 2^2) + (1 \times 2^1) + (0 \times 2^0)$$

$$6_{10} = 0 + 0 + 0 + 0 + 0 + 4 + 2 + 0$$

ซึ่งถูกต้อง เพราะ $3 \times 2 = 6$

สรุป การทำงานของฮาร์ดแวร์สำหรับการคูณเลขฐานสองนั้นเป็นการนำหลักการคูณยาวมาประยุกต์ใช้
โดยอาศัยการวนซ้ำของการบวกและเลื่อนบิต ซึ่งเป็นกระบวนการพื้นฐานที่หน่วย ALU และ Control
Unit ใน CPU สามารถจัดการได้

4.4.1 อธิบายแนวคิดหลักของการทำงานตามตาราง

การคูณเลขฐานสองด้วยฮาร์ดแวร์ในระดับวงจรนั้นไม่ได้ใช้การคูณตรงๆ เหมือนที่เราทำบนกระดาษ แต่ใช้วิธีที่ชาญฉลาดกว่า นั่นคือการประยุกต์ใช้ การบวก (Add) และ การเลื่อนบิต (Shift) ซ้ำๆ กันไป กระบวนการนี้จะใช้รีจิสเตอร์หลัก 3 ตัว:

- A (Accumulator): รีจิสเตอร์สำหรับเก็บผลบวกบางส่วน
- Q (Multiplier): รีจิสเตอร์สำหรับเก็บตัวคูณ
- M (Multiplicand): รีจิสเตอร์สำหรับเก็บตัวตั้ง

วงจรจะตรวจสอบบิตขวาสุดของ Q (Q_0) ในแต่ละรอบการทำงาน เพื่อตัดสินใจว่าจะบวกค่า M เข้าไปใน A หรือไม่ จากนั้นจึงเลื่อนบิต A และ Q ไปทางขวาพร้อมกัน

อธิบายตารางการทำงานแบบละเอียด: (การคูณกันของ 3×2 ดังนั้น สถานะเริ่มต้น) :

- M (ตัวตั้ง) = $3_{10} = 0011_2$
- Q (ตัวคูณ) = $2_{10} = 0010_2$
- A (Accumulator) = 0000_2
- N (จำนวนรอบ) = 4 (เท่ากับจำนวนบิต)

รอบที่ 1:

1. ตรวจสอบ Q_0 : บิตขวาสุดของ Q คือ 0
2. การกระทำ: เนื่องจาก $Q_0 = 0$ จึง ไม่ต้องบวก (No Add) ค่า M เข้าไปใน A
 - A ยังคงเป็น 0000_2
 - Q ยังคงเป็น 0010_2
3. เลื่อนบิต: เลื่อนค่าในรีจิสเตอร์ A และ Q ไปทางขวาพร้อมกัน 1 ตำแหน่ง
 - ค่าใน A (0000) ถูกเลื่อนไปทางขวา ทำให้ A ยังคงเป็น 0000_2
 - ค่าใน Q (0010) ถูกเลื่อนไปทางขวา ทำให้ Q กลายเป็น 0001_2
4. นับรอบ: N ลดลงเหลือ 3

รอบที่ 2:

1. ตรวจสอบ Q_0 : บิตขวาสุดของ Q คือ 1
2. การกระทำ: เนื่องจาก $Q_0 = 1$ จึง บวก (Add) ค่า M (0011) เข้าไปใน A (0000)
 - $A = 0000_2 + 0011_2 = 0011_2$
 - Q ยังคงเป็น 0001_2
3. เลื่อนบิต: เลื่อนค่าในรีจิสเตอร์ A และ Q ไปทางขวาพร้อมกัน

- ค่าใน A (0011) ถูกเลื่อนไปทางขวา ทำให้ A กลายเป็น 0001_2
- ค่าใน Q (0001) ถูกเลื่อนไปทางขวา และบิตซ้ายสุดของ Q จะรับค่าจากบิตขวาสุดของ A
(1) ทำให้ Q กลายเป็น 1000_2

4. นับรอบ: N ลดลงเหลือ 2

รอบที่ 3:

1. ตรวจสอบ Q_0 : บิตขวาสุดของ Q คือ 0
2. การกระทำ: เนื่องจาก $Q_0 = 0$ จึง **ไม่ต้องบวก** (No Add)
 - A ยังคงเป็น 0001_2
 - Q ยังคงเป็น 1000_2
3. เลื่อนบิต: เลื่อนค่าในรีจิสเตอร์ A และ Q ไปทางขวาพร้อมกัน
 - ค่าใน A (0001) ถูกเลื่อนไปทางขวา ทำให้ A กลายเป็น 0000_2
 - ค่าใน Q (1000) ถูกเลื่อนไปทางขวา และบิตซ้ายสุดของ Q จะรับค่าจากบิตขวาสุดของ A
(1) ทำให้ Q กลายเป็น 1100_2

4. นับรอบ: N ลดลงเหลือ 1

รอบที่ 4:

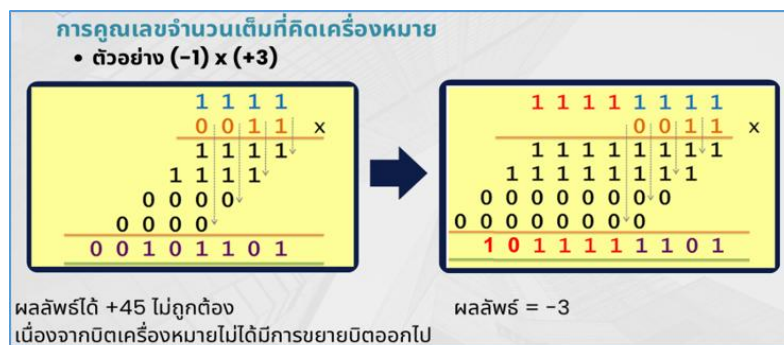
1. ตรวจสอบ Q_0 : บิตขวาสุดของ Q คือ 0
2. การกระทำ: เนื่องจาก $Q_0 = 0$ จึง **ไม่ต้องบวก** (No Add)
 - A ยังคงเป็น 0000_2
 - Q ยังคงเป็น 1100_2
3. เลื่อนบิต: เลื่อนค่าในรีจิสเตอร์ A และ Q ไปทางขวาพร้อมกัน
 - ค่าใน A (0000) ถูกเลื่อนไปทางขวา ทำให้ A ยังคงเป็น 0000_2
 - ค่าใน Q (1100) ถูกเลื่อนไปทางขวา และบิตซ้ายสุดของ Q จะรับค่าจากบิตขวาสุดของ A
(0) ทำให้ Q กลายเป็น 0110_2

4. นับรอบ: N ลดลงเหลือ 0

ผลลัพธ์สุดท้าย เมื่อ $N = 0$ กระบวนการจะสิ้นสุด และผลลัพธ์สุดท้ายจะอยู่ที่การรวมกันของรีจิสเตอร์ A และ Q หรือ AQ ซึ่งก็คือ $0000 + 0110 = 00000110_2$

เมื่อแปลงกลับเป็นเลขฐานสิบ จะได้ 6 ซึ่งถูกต้องตรงกับ 3×2 จะเห็นได้ว่าการทำงานในระดับฮาร์ดแวร์นั้นไม่ใช่การคูณที่ซับซ้อน แต่เป็นการวนซ้ำของกระบวนการง่ายๆ เพียงสองอย่างคือการบวกและการเลื่อนบิต

4.5 หลักการคูณเลขจำนวนเต็มที่มีเครื่องหมาย (Signed Integer Multiplication)



รูปที่ 6-20 ข้อจำกัดของการคูณเลขแบบคิดเครื่องหมายและการ
แก้ปัญหาด้วยการขยายบิตเครื่องหมาย

การคูณการคูณเลขจำนวนเต็มแบบมีเครื่องหมาย (-1×3) โดยใช้หลักการของ 2's Complement ซึ่งการคูณแบบนี้ยังคงใช้พื้นฐานเดียวกันกับการคูณยาวแบบปกติ แต่จะมีการแปลงตัวเลขลบให้อยู่ใน

รูป 2's Complement ก่อนการคำนวณ

ขั้นตอนการคูณ $(-1) \times 3$ ดังนี้

ขั้นตอนที่ 1: แปลงเลขฐานสิบเป็นเลขฐานสองแบบ 2's Complement

3_{10} เป็นเลขบวก ในระบบ 4 บิต (เพื่อความกระชับ) คือ 0011_2

-1_{10} เป็นเลขลบ ต้องแปลงเป็น 2's Complement:

1 คือ 0001_2

1's Complement ของ 0001_2 คือ 1110_2 (กลับบิตตรงข้าม)

2's Complement ของ 0001_2 คือ $1110_2 + 1_2 = 1111_2$ (บวก 1 เพิ่มเข้าไป)

ดังนั้น โจทย์ของเราคือ $1111_2 \times 0011_2$

ขั้นตอนที่ 2: ทำการคูณยาวตามปกติ

เราจะนำตัวคูณ (Multiplier) คือ 0011_2 ไปคูณกับตัวตั้ง (Multiplicand) คือ 1111_2

ขั้นตอนการคูณคือ การคำนวณจะดำเนินไปตามขั้นตอนการคูณยาว ดังนี้

1. นำบิตแรก (ขวาสุด) ของตัวคูณ (1) ไปคูณกับตัวตั้ง (0011)

○ $0011_2 \times 1 = 0011_2$

○ ได้ผลลัพธ์ย่อยบรรทัดแรกคือ 0011_2

2. นำบิตที่สองของตัวคูณ (1) ไปคูณกับตัวตั้ง (0011)

○ $0011_2 \times 1 = 0011_2$

○ จากนั้น เลื่อนผลลัพธ์ย่อยนี้ไปทางซ้าย 1 ตำแหน่ง

○ เราจะได้ 00110_2

3. นำบิตที่สามของตัวคุณ (1) ไปคูณกับตัวตั้ง (0011)

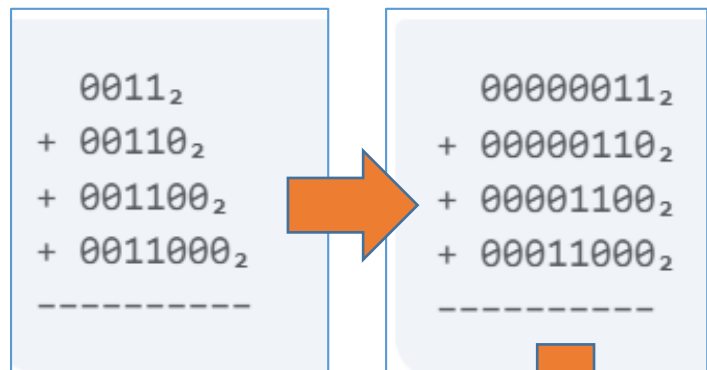
- $0011_2 \times 1 = 0011_2$
- เลื่อนผลลัพธ์ย่อยนี้ไปทางซ้าย 2 ตำแหน่ง
- เราจะได้ 001100_2

4. นำบิตที่สี่ (ซ้ายสุด) ของตัวคุณ (1) ไปคูณกับตัวตั้ง (0011)

- $0011_2 \times 1 = 0011_2$
- เลื่อนผลลัพธ์ย่อยนี้ไปทางซ้าย 3 ตำแหน่ง
- เราจะได้ 0011000_2

5. นำผลลัพธ์ย่อยทั้งหมดมาบวกกัน

- 0011_2
- $+ 00110_2$
- $+ 001100_2$
- $+ 0011000_2$



เราจะเริ่มบวกจากหลักขวาสุด (หลักที่ 0) ไปทางซ้ายทีละหลัก

คำอธิบายการบวกแต่ละหลัก :

หลักที่ 1 (บิตขวาสุด): $1+0+0+0=1_2$

- ผลลัพธ์คือ 1 และไม่มีตัวทด

หลักที่ 2: $1+1+0+0=10_2$

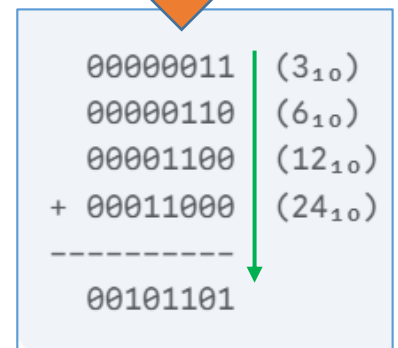
- ผลลัพธ์คือ 0 ในหลักนี้ และ ทด 1 ไปยังหลักถัดไป (หลักที่ 3)

• หลักที่ 3 $0+1+1+0 = 11_2$

- แต่การคำนวณที่ถูกต้องต้อง รวมค่าที่ทดมาจากหลักที่ 2 ด้วย
- ดังนั้นการคำนวณ: $(1 \text{ ที่ทดมา}) + 0 + 1 + 1 + 0 = 11_2$
- ผลลัพธ์ที่ได้คือ 1 ในหลักนี้ และ ทด 1 ไปยังหลักถัดไป (หลักที่ 4)

• หลักที่ 4: $0 + 0 + 1 + 1 = 11_2$

- แต่การคำนวณที่ถูกต้องต้อง รวมค่าที่ทดมาจากหลักที่ 3 ด้วย
- ดังนั้นการคำนวณ: $(1 \text{ ที่ทดมา}) + 0 + 0 + 1 + 1 = 11_2$
- ผลลัพธ์คือ 1 ในหลักนี้ และ ทด 1 ไปยังหลักถัดไป



• หลักที่ 5:

- การคำนวณ: $(1 \text{ ที่ทดมา}) + 0 + 0 + 0 + 1 = 10_2$
- ผลลัพธ์คือ **0** ในหลักนี้ และ **ทด 1** ไปยังหลักถัดไป

• หลักที่ 6:

- การคำนวณ: $(1 \text{ ที่ทดมา}) + 0 + 0 + 0 + 0 = 1_2$
- ผลลัพธ์คือ **1** และไม่มีตัวทด

• หลักที่ 7:

- การคำนวณ: $0+0+0+0=0$
- ผลลัพธ์คือ **0**

• หลักที่ 8 (บิตซ้ายสุด):

- การคำนวณ: $0+0+0+0=0$
- ผลลัพธ์คือ **0**

00000011	(3 ₁₀)
00000110	(6 ₁₀)
00001100	(12 ₁₀)
+ 00011000	(24 ₁₀)

00101101	

เมื่อเรานำผลลัพธ์ที่ถูกต้องในแต่ละหลักมารวมกัน จะได้ผลลัพธ์สุดท้ายคือ 00101101₂ ซึ่งเมื่อแปลงเป็นเลขฐาน 10 จะเท่ากับ 45₁₀ ซึ่งตรงกับการบวกเลขฐาน 10 ของ 3+6+12+24 จึงเท่ากับ 45

อ้าว... แต่ในตัวอย่างที่อาจารย์ให้มา ผลลัพธ์กลับเป็น 1111101

การคูณเลขจำนวนเต็มที่มีเครื่องหมาย

• ตัวอย่าง $(-1) \times (+3)$

1 1 1 1

0 0 1 1 x

1 1 1 1

0 0 0 0

0 0 0 0

0 0 1 0 1 1 0 1

ผลลัพธ์ได้ +45 ไม่ถูกต้อง

เนื่องจากบิตเครื่องหมายไม่ได้มีการขยายบิตออกไป

➔

1 1 1 1 1 1 1 1

0 0 1 1 x

1 1 1 1 1 1 1 1

1 1 1 1 1 1 1 1

0 0 0 0 0 0 0 0

0 0 0 0 0 0 0 0

1 0 1 1 1 1 1 0 1

ผลลัพธ์ = -3

รูปที่ 6-20 ข้อจำกัดของการคูณเลขแบบคิดเครื่องหมายและการแก้ปัญหาด้วยการขยายบิตเครื่องหมาย

ตรงนี้เป็นจุดสำคัญ เพราะการคูณเลขแบบมีเครื่องหมาย **ผลลัพธ์ย่อยก็ต้องมีการพิจารณาเครื่องหมายด้วย** โดยเฉพาะเมื่อเราทำ **Sign Extension** หรือการขยายบิตเครื่องหมายไปทางซ้าย การคูณที่ถูกต้องตามหลักการของภาพก่อนหน้า คือ การขยายบิตซ้ายสุดของตัวตั้งและตัวคูณ (ซึ่งก็คือบิตเครื่องหมาย) ให้เป็น 8 บิต ก่อนที่จะทำการคูณและบวก

4.5.1 หลักการการขยายบิตเครื่องหมาย (Sign Extension):

ในระบบเลข 2's Complement บิตซ้ายสุด (most significant bit) คือบิตเครื่องหมาย (0 สำหรับบวก และ 1 สำหรับลบ) ก่อนการคูณแบบยาว เมื่อขยายขนาดของตัวเลข (จาก 4 บิตเป็น 8 บิต) เราต้อง **คัดลอก** บิตเครื่องหมาย ไปทางซ้ายจนครบตามจำนวนบิตที่ต้องการ เพื่อให้ค่าของตัวเลขไม่เปลี่ยนแปลง

- **ตัวตั้ง (Multiplicand) = $3_{10} = 0011_2$** : เป็นเลขบวก บิตเครื่องหมายคือ 0 เมื่อขยายเป็น 8 บิต จึงเติม 0 ไปทางซ้าย ได้เป็น **00000011_2**
- **ตัวคูณ (Multiplier) = $-1_{10} = 1111_2$** : เป็นเลขลบ บิตเครื่องหมายคือ 1 เมื่อขยายเป็น 8 บิต จึงเติม 1 ไปทางซ้าย ได้เป็น **11111111_2**

4.5.2 ขั้นตอนการคูณยาวแบบมีเครื่องหมาย

เราจะทำการคูณ 00000011_2 (ตัวตั้ง) กับ 11111111_2 (ตัวคูณ) และบวกผลลัพธ์ย่อย โดยผลลัพธ์สุดท้ายจะมีขนาดเป็น 16 บิต (8 + 8)

บิตของตัวคูณ	ผลลัพธ์ย่อย (00000011 x บิตตัวคูณ)	ผลลัพธ์ย่อย (8 บิต) ที่ต้องบวก
บิตที่ 0 (1)	00000011 ₂	...00000011 ₂
บิตที่ 1 (1)	00000011 ₂	...00000110 ₂
บิตที่ 2 (1)	00000011 ₂	...00001100 ₂
บิตที่ 3 (1)	00000011 ₂	...00011000 ₂
บิตที่ 4 (1)	00000011 ₂	...00110000 ₂
บิตที่ 5 (1)	00000011 ₂	...01100000 ₂
บิตที่ 6 (1)	00000011 ₂	...11000000 ₂
บิตที่ 7 (1)	00000011 ₂	...10000000 ₂ (เลข 2's Complement ของ -3)

ตารางที่ 6-2 การแสดงขั้นตอนการคำนวณผลลัพธ์ย่อยและการบวกเลขฐานสองแบบคิดเครื่องหมาย

ผลบวกของเลขฐานสองชุดนี้คือ 11111101_2 ซึ่งเท่ากับ -3 ในระบบเลขฐานสิบแบบ 8 บิต

ขั้นตอนการบวกอย่างละเอียด การบวกเลขชุดนี้สามารถทำได้โดยการบวกทีละคู่ หรือจะตั้งบวกเป็นแถว เหมือนการบวกยาวในเลขฐานสิบก็ได้ เพื่อให้เข้าใจง่ายการบวกจากหลักขวาสุดไปทางซ้าย โดยพิจารณาตัวทด (carry) ในแต่ละหลัก

การบวกทีละหลัก

หลักที่ 0 (ขวาสุด): $1 + 0 + 0 + 0 + 0 + 0 + 0 + 0 = 1$

หลักที่ 1: $1 + 1 + 0 + 0 + 0 + 0 + 0 + 0 = 10_2 \rightarrow$ ผลลัพธ์ 0, ทด 1

หลักที่ 2: $1 + 1 + 0 + 0 + 0 + 0 + 0 + 0$ (รวมตัวทด 1) = $11_2 \rightarrow$ ผลลัพธ์ 1, ทด 1

หลักที่ 3: $1 + 1 + 0 + 0 + 0 + 0 + 0 + 0$ (รวมตัวทด 1) = $11_2 \rightarrow$ ผลลัพธ์ 1, ทด 1

หลักที่ 4: $0 + 1 + 0 + 0 + 0 + 0 + 0 + 0$ (รวมตัวทด 1) = $10_2 \rightarrow$ ผลลัพธ์ 0, ทด 1

หลักที่ 5: $0 + 1 + 1 + 0 + 0 + 0 + 0 + 0$ (รวมตัวทด 1) = $11_2 \rightarrow$ ผลลัพธ์ 1, ทด 1

หลักที่ 6: $0 + 1 + 1 + 0 + 0 + 0 + 0 + 0$ (รวมตัวทด 1) = $11_2 \rightarrow$ ผลลัพธ์ 1, ทด 1

หลักที่ 7 (ซ้ายสุด): $0 + 1 + 1 + 0 + 0 + 0 + 0 + 0$ (รวมตัวทด 1) = $11_2 \rightarrow$ ผลลัพธ์ 1, ทด 1

0	0	0	0	0	0	1	1 ₂
+	0	0	0	0	0	1	1 0 ₂
+	0	0	0	0	1	1	0 0 ₂
+	0	0	0	1	1	0	0 0 ₂
+	0	0	1	1	0	0	0 0 ₂
+	0	1	1	0	0	0	0 0 ₂
+	1	1	0	0	0	0	0 0 ₂
+	1	0	0	0	0	0	0 0 ₂

1	1	1	1	1	1	0	1 ₂

ผลลัพธ์ที่ได้จากการบวกแต่ละหลักเรียงกันคือ 11111101₂ ซึ่งเป็นผลลัพธ์ที่ต้องการ

ที่มาของผลลัพธ์ 11111101₂:

ถ้าเป็นการคูณในระดับฮาร์ดแวร์จริงจะทำในลักษณะการบวกสะสม (Accumulation) ที่ละรอบ

1. รอบที่ 1: นำ 00000011 มาบวก (เพราะบิตตัวคูณเป็น 1)

○ $A = 00000011$

2. เลื่อนบิต: เลื่อน A และ Q ไปทางขวา

3. รอบที่ 2: นำ 00000011 มาบวก (เพราะบิตตัวคูณเป็น 1)

○ $A = A + M = 00000011 + 00000011$

4. ทำซ้ำไปเรื่อย ๆ

เพราะฉะนั้น ตัวเลขสุดท้าย 11111101₂ นี้คือ 2's

Complement ของ -3 ซึ่งเป็นผลลัพธ์ที่ต้องการของการคูณ $3 \times (-1)$

00000011 ₂	(มาจากบิต 0)
+	00000110 ₂ (มาจากบิต 1)
+	00001100 ₂ (มาจากบิต 2)
+	00011000 ₂ (มาจากบิต 3)
+	00110000 ₂ (มาจากบิต 4)
+	01100000 ₂ (มาจากบิต 5)
+	11000000 ₂ (มาจากบิต 6)
+	10000000 ₂ (มาจากบิต 7)

11111101 ₂	

ตัวอย่างการบวกเลขฐานสองโดยเน้นที่การบวกทีละคู่ เพื่อให้เห็นภาพการคำนวณที่ชัดเจนยิ่งขึ้น

4.5.3 การบวกเลขฐานสองแบบค่าบวก: ตัวอย่าง $5 + 3 = 8$

เราจะใช้ตัวอย่าง $5 + 3 = 8$ ซึ่งเป็นพื้นฐานที่เข้าใจง่าย เพื่อแสดงขั้นตอนการบวกเลขฐานสองแบบมีเครื่องหมาย

ขั้นตอนที่ 1: แปลงเลขฐานสิบเป็นเลขฐานสอง

• 5₁₀ (เลขบวก) ในระบบ 8 บิต คือ 00000101₂

• 3₁₀ (เลขบวก) ในระบบ 8 บิต คือ 00000011₂

ขั้นตอนที่ 2: ตั้งบวกและคำนวณ นำตัวเลขทั้งสองมาตั้งบวกทีละหลัก โดยเริ่มจากหลักขวาสุดไปทางซ้าย

คำอธิบายการบวกทีละหลัก:

- หลักที่ 0 (ขวาสุด): $1 + 1 = 10 \rightarrow$ ผลลัพธ์คือ 0 ทด 1 ไปหลักถัดไป
- หลักที่ 1: $0 + 1 + 1$ (รวมตัวทด) $= 10 \rightarrow$ ผลลัพธ์คือ 0 ทด 1 ไปหลักถัดไป
- หลักที่ 2: $1 + 0 + 1$ (รวมตัวทด) $= 10 \rightarrow$ ผลลัพธ์คือ 0 ทด 1 ไปหลักถัดไป
- หลักที่ 3: $0 + 0 + 1$ (รวมตัวทด) $= 1 \rightarrow$ ผลลัพธ์คือ 1
- หลักที่ 4 - 7: $0 + 0 = 0 \rightarrow$ ผลลัพธ์คือ 0 (4 หลัก)

เมื่อนำผลลัพธ์ที่ได้มาเรียงต่อกัน จะได้ **00001000₂** ซึ่งเป็น

เลขฐานสองของ 8 ในระบบฐานสิบ และเป็นคำตอบที่ถูกต้อง

$$\begin{array}{r} 00000101 \quad (5) \\ + 00000011 \quad (3) \\ \hline 00001000 \quad (8) \end{array}$$

4.5.4 การบวกเลขฐานสองแบบค่าบวกและลบ: ตัวอย่าง $5 - 3 = 2$

คำนวณ $5 - 3$ ในระบบคอมพิวเตอร์จะถูกเปลี่ยนเป็นการบวกเลขฐานสองแบบมีเครื่องหมาย โดยใช้ 2's Complement

หลักการ: เปลี่ยนการลบเป็นการบวก

คอมพิวเตอร์ส่วนใหญ่ไม่ได้มีวงจรสำหรับ "ลบ" โดยเฉพาะ แต่จะใช้หลักการเปลี่ยนการลบเป็นการบวก

แทน:

- $A - B$ จะถูกคำนวณเป็น $A + (-B)$ ดังนั้น $5 - 3$ จะกลายเป็น $5 + (-3)$

ขั้นตอนการคำนวณ $5 + (-3)$

ขั้นตอนที่ 1: แปลงเลขฐานสิบเป็นเลขฐานสอง

- **5₁₀** (ตัวตั้ง): ในระบบเลขฐานสอง 8 บิต คือ **00000101₂**
- **-3₁₀** (ตัวลบ): ต้องแปลงเป็น 2's Complement ก่อน
 - หาค่าบวก 3 คือ **00000011₂**
 - ทำ 1's Complement (สลับบิต): **11111100₂**
 - บวก 1: **11111100₂ + 1₂ = 11111101₂**
 - ดังนั้น -3 ในรูป 2's Complement คือ **11111101₂**

ขั้นตอนที่ 2: ตั้งบวกและคำนวณ

นำค่าที่แปลงได้มาตั้งบวกกันตามหลักการบวกเลขฐานสอง

คำอธิบายการบวกทีละหลัก:

- หลักที่ 0 (ขวาสุด): $1 + 1 = 10 \rightarrow$ ผลลัพธ์ 0 ทด 1
- หลักที่ 1: $0 + 0$ (รวมตัวทด 1) $= 1 \rightarrow$ ผลลัพธ์ 1
- หลักที่ 2: $1 + 1 = 10 \rightarrow$ ผลลัพธ์ 0 ทด 1
- หลักที่ 3: $0 + 1$ (รวมตัวทด 1) $= 10 \rightarrow$ ผลลัพธ์ 0 ทด 1
- หลักที่ 4-7: $0 + 1$ (รวมตัวทด 1) $= 10 \rightarrow$ ผลลัพธ์ 0 ทด 1 (ทำซ้ำ 4 ครั้ง)

$$\begin{array}{r} 00000101 \quad (5) \\ + 11111101 \quad (-3) \\ \hline (1)0000010 \quad (2) \end{array}$$

บิตที่เกินออกมา (1 ในหลักที่ 8) จะถูกทิ้งไป เนื่องจากเรากำหนดในระบบ 8 บิต

ขั้นตอนที่ 3: ตรวจสอบผลลัพธ์

ผลลัพธ์ที่ได้คือ 00000010₂ ซึ่งเป็นเลขฐานสองของ 2 ในระบบฐานสิบ และถูกต้องตรงกับ $5 - 3 = 2$

4.5.5 การบวกเลขฐานสองแบบค่าติดลบ: ตัวอย่าง: $(-5) + (-3) = -8$

ในกรณีนี้ ตัวเลขทั้งสองเป็นลบ เราจะใช้ 2's Complement ในการคำนวณ

ขั้นตอนที่ 1: แปลงเลขฐานสิบเป็น 2's Complement

- -5₁₀:
 - ค่าบวก 5 คือ 00000101₂
 - ทำ 1's Complement: 11111010₂
 - บวก 1: 11111011₂
- -3₁₀:
 - ค่าบวก 3 คือ 00000011₂
 - ทำ 1's Complement: 11111100₂
 - บวก 1: 11111101₂

ขั้นตอนที่ 2: ตั้งบวกและคำนวณ

คำอธิบายการบวกทีละหลัก:

- หลักที่ 0: $1 + 1 = 10 \rightarrow$ ผลลัพธ์คือ 0 ทด 1
- หลักที่ 1: $1 + 0 + 1$ (รวมตัวทด) $= 10 \rightarrow$ ผลลัพธ์คือ 0 ทด 1
- หลักที่ 2: $0 + 1 + 1$ (รวมตัวทด) $= 10 \rightarrow$ ผลลัพธ์คือ 0 ทด 1
- หลักที่ 3: $1 + 1 + 1$ (รวมตัวทด) $= 11 \rightarrow$ ผลลัพธ์คือ 1 ทด 1
- หลักที่ 4: $1 + 1 + 1$ (รวมตัวทด) $= 11 \rightarrow$ ผลลัพธ์คือ 1 ทด 1
- หลักที่ 5: $1 + 1 + 1$ (รวมตัวทด) $= 11 \rightarrow$ ผลลัพธ์คือ 1 ทด 1
- หลักที่ 6: $1 + 1 + 1$ (รวมตัวทด) $= 11 \rightarrow$ ผลลัพธ์คือ 1 ทด 1
- หลักที่ 7: $1 + 1 + 1$ (รวมตัวทด) $= 11 \rightarrow$ ผลลัพธ์คือ 1 ทด 1

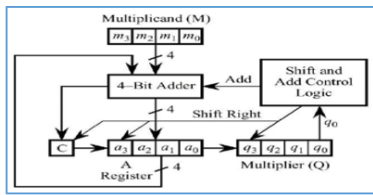
1 1 1 1 1 1 1 0	<-- ตัวทด (Carry)
1 1 1 1 1 0 1 1	(-5)
+ 1 1 1 1 1 1 0 1	(-3)

(1)1 1 1 1 1 0 0 0	(-8)

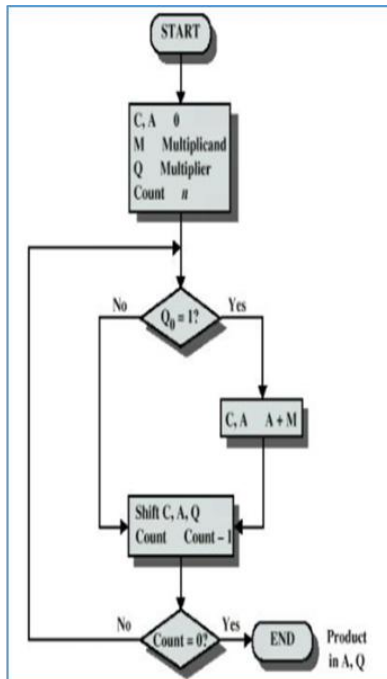
บิตที่เกินออกมา (1) ในหลักที่ 8 จะถูกทิ้งไป เนื่องจากเรากำหนดในระบบ 8 บิต ผลลัพธ์ที่ได้คือ

11111000₂ ซึ่งเมื่อแปลงกลับเป็นเลขฐานสิบแบบ 2's Complement จะได้ -8 ซึ่งถูกต้อง

4.5.6 การบวกเลขฐานสองแบบฮาร์ดแวร์สำหรับการคูณธรรมดา



รูปที่ 6-21 ขั้นตอนการทำงานการคูณเลขฐานสองแบบไม่มีเครื่องหมายขนาด 4 บิต



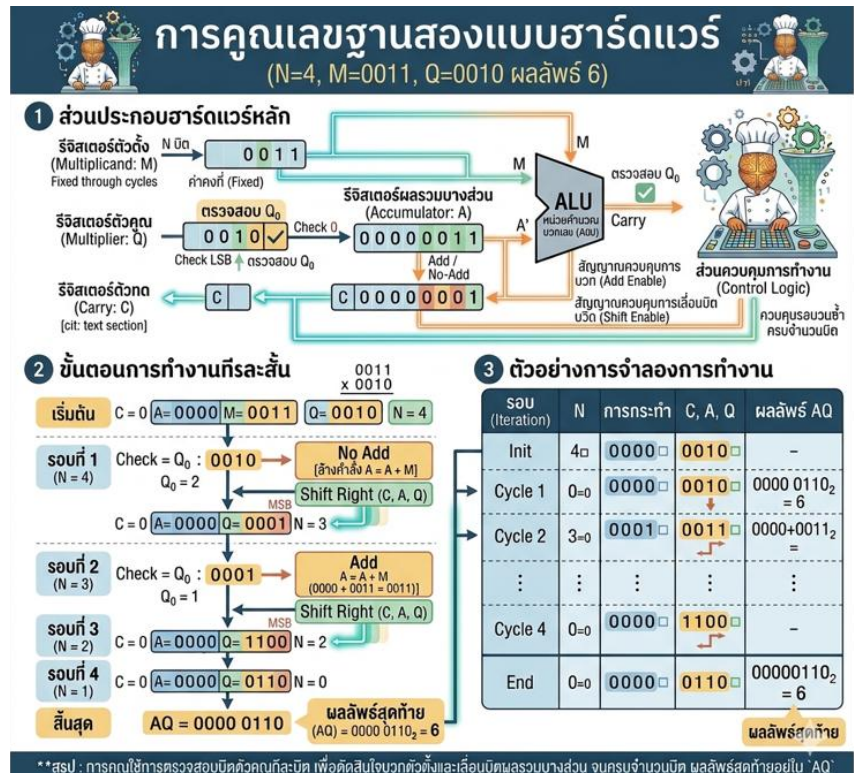
รูปที่ 6-22 ผังงานแสดงกระบวนการคูณเลขฐานสองแบบไม่มีเครื่องหมาย

การคูณด้วยตัวเลขขนาด 4 บิต

ตัวตั้ง (Multiplicand) จัดเก็บลงในรีจิสเตอร์ M

ตัวคูณ (Multiplier) จัดเก็บลงในรีจิสเตอร์ Q

รีจิสเตอร์ A และ C จะถูกกำหนดค่าเริ่มต้นเป็นศูนย์



รูปที่ 6-23 แผนผังโครงสร้างฮาร์ดแวร์วงจรหารเลขฐานสองแบบไม่มีเครื่องหมาย

1. กำหนด

กำหนดให้ C, A = 0,

M คือ ตัวตั้ง ,

Q คือ ตัวคูณ,

N คือ จำนวนบิต ของตัวตั้ง (รอบการทำงาน)

2. เช็คค่า Q₀

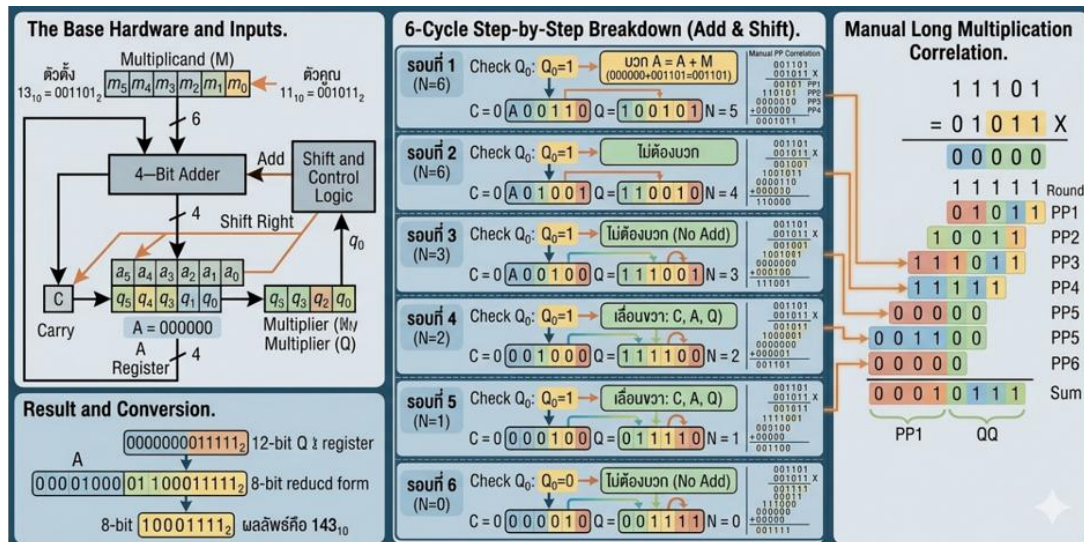
ถ้า Q₀ เท่ากับ 0 ให้ Shift Right (C,A,Q) และลบค่า N ไป 1

ถ้า Q₀ เท่ากับ 1 ให้ บวกค่า M เข้าที่ A คือ A=A+M และทำการ Shift Right (C,A,Q) จากนั้นลบค่า N ไป 1

3. ทำงานจนกว่า N=0 ให้จบการทำงาน

4.5.7 ฮาร์ดแวร์สำหรับการคูณธรรมดา (หาผลลัพธ์การคูณของ 13×11 (การคูณด้วยตัวเลขขนาด 4 บิต))

อธิบายขั้นตอนการทำงานของฮาร์ดแวร์สำหรับการคูณแบบธรรมดา (Unsigned Binary Multiplication) โดยใช้ตัวอย่าง 13×11 การคูณแบบหลักการเดียวกับการคูณยาวด้วยมือ แต่ในระดับฮาร์ดแวร์จะใช้การวนซ้ำของกระบวนการ "บวก" (Add) และ "เลื่อนบิต" (Shift) เพื่อหาผลลัพธ์จึงจะใช้รีจิสเตอร์หลัก 3 ตัว



รูปที่ 6-24 การวิเคราะห์ความสัมพันธ์ระหว่างโครงสร้างฮาร์ดแวร์และขั้นตอนการประมวลผล

การคูณเลขฐานสองขนาด 6 บิต (13 คูณ 11)

- M (Multiplicand): เก็บตัวตั้ง (13 หรือ 1101₂) Q (Multiplier): เก็บตัวคูณ (11 หรือ 1011₂)
- A (Accumulator): เก็บผลบวกสะสม เริ่มต้นด้วย 0 C (Carry): บิตทด (Carry Bit)

1. กำหนดค่าเริ่มต้น

- M = 001101₂ (13 ใน 6 บิต) Q = 001011₂ (11 ใน 6 บิต)
- A = 000000₂ C = 0 N = 6 (จำนวนบิต)

2. รอบที่ 1: ตรวจสอบ Q₀ Q₀ (บิตขวาสุดของ Q) คือ 1

- การกระทำ: เนื่องจาก Q₀ = 1
- ให้ทำการ บวกค่า M เข้า A (A = A + M)
 - A = 000000₂ + 001101₂ = 001101₂
- จากนั้น: ทำการ Shift Right ทั้ง C, A, และ Q
 - C จะรับบิตที่หลุดจาก A
 - A จะรับบิตที่หลุดจาก Q และบิตซ้ายสุดจะถูก Shift จาก C (ในกรณีนี้ 0)
 - Q จะถูก Shift ไปทางขวาผลลัพธ์หลัง Shift: C = 0, A = 000110₂, Q = 100101₂
- N ลดลงเหลือ 5

3. รอบที่ 2: ตรวจสอบ Q_0 Q_0 (บิตขวาสุดของ Q) คือ 1

- การกระทำ: เนื่องจาก $Q_0 = 1$ ให้ทำการ บวกค่า M เข้า A ($A = A + M$)
 - $A = 000110_2 + 001101_2 = 010011_2$
- จากนั้น: ทำการ Shift Right ทั้ง C, A, และ Q
 - ผลลัพธ์หลัง Shift: $C = 0$, $A = 001001_2$, $Q = 110010_2$
- N ลดลงเหลือ 4

4. รอบที่ 3: ตรวจสอบ Q_0 Q_0 (บิตขวาสุดของ Q) คือ 0

- การกระทำ: เนื่องจาก $Q_0 = 0$ ให้ทำการ No Add (ไม่ต้องบวก)
- จากนั้น: ทำการ Shift Right ทั้ง C, A, และ Q
 - ผลลัพธ์หลัง Shift: $C = 0$, $A = 000100_2$, $Q = 111001_2$
- N ลดลงเหลือ 3

5. รอบที่ 4: ตรวจสอบ Q_0 Q_0 (บิตขวาสุดของ Q) คือ 1

- การกระทำ: เนื่องจาก $Q_0 = 1$ ให้ทำการ บวกค่า M เข้า A ($A = A + M$)
 - $A = 000100_2 + 001101_2 = 010001_2$
- จากนั้น: ทำการ Shift Right ทั้ง C, A, และ Q
 - ผลลัพธ์หลัง Shift: $C = 0$, $A = 001000_2$, $Q = 111100_2$
- N ลดลงเหลือ 2

6. รอบที่ 5: ตรวจสอบ Q_0 Q_0 (บิตขวาสุดของ Q) คือ 0

- การกระทำ: เนื่องจาก $Q_0 = 0$ ให้ทำการ No Add
- จากนั้น: ทำการ Shift Right ทั้ง C, A, และ Q
 - ผลลัพธ์หลัง Shift: $C = 0$, $A = 000100_2$, $Q = 011110_2$
- N ลดลงเหลือ 1

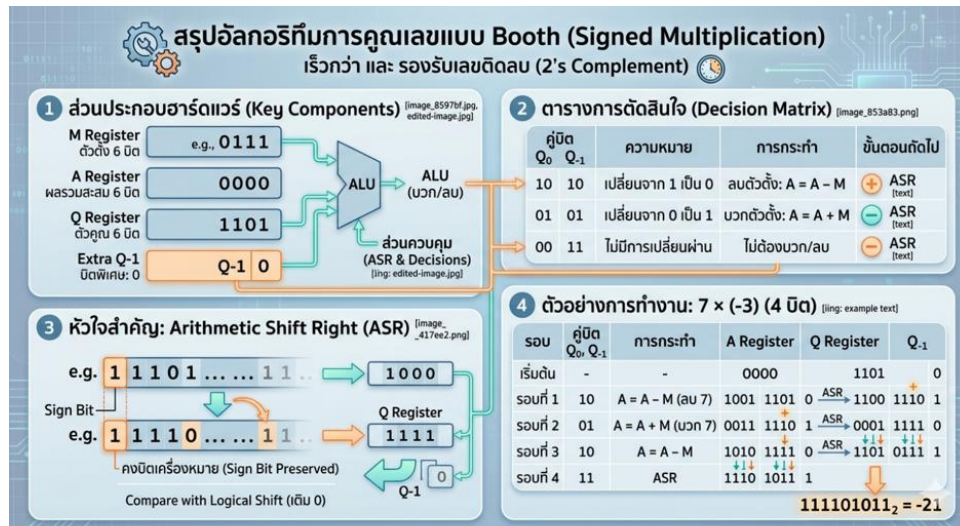
7. รอบที่ 6: ตรวจสอบ Q_0 Q_0 (บิตขวาสุดของ Q) คือ 0

- การกระทำ: เนื่องจาก $Q_0 = 0$ ให้ทำการ No Add
- จากนั้น: ทำการ Shift Right ทั้ง C, A, และ Q
 - ผลลัพธ์หลัง Shift: $C = 0$, $A = 000010_2$, $Q = 001111_2$
- N ลดลงเหลือ 0

8. สิ้นสุดการทำงาน เมื่อ $N = 0$ การทำงานสิ้นสุดลง ผลลัพธ์สุดท้ายจะอยู่ที่การรวมกันของรีจิสเตอร์ A และ Q ซึ่งได้ 000010001111_2 หรือ 10001111_2 เมื่อลดขนาดลงเหลือ 8 บิต (ตามตัวอย่างก่อนหน้า) ซึ่งเท่ากับ 143 ในเลขฐานสิบ

4.6 หลักการคำนวณเลขฐานสองแบบมีเครื่องหมาย (Signed Binary Numbers) แบบอัลกอริทึม

(Booth's Algorithm) Booth's Algorithm เป็นอัลกอริทึมที่ใช้ในการคูณเลขฐานสอง โดยอยู่ในรูปแบบ 2's Complement จุดเด่นของมันคือจะช่วยลดจำนวนครั้งของการบวกและลบเมื่อตัวคูณมีบิต 0 หรือ 1 เรียงติดกันจำนวนมาก ทำให้การทำงานเร็วกว่าการคูณแบบพื้นฐาน



รูปที่ 6-25 สรุปโครงสร้างฮาร์ดแวร์และตรรกะการตัดสินใจของอัลกอริทึมของบูธสำหรับการคูณเลขฐานสองแบบมีเครื่องหมาย

แผนภาพนี้ถูกออกแบบมาเป็น **ผังข้อมูลแบบโต้ตอบที่ละรอบ (Interactive Trace-Map Infographic)** โดยแบ่งออกเป็น 4 ส่วนหลักดังนี้:

- ส่วนประกอบฮาร์ดแวร์ (Hardware Architecture):** อยู่ทางด้านซ้ายบน แสดงให้เห็นรีจิสเตอร์หลักทั้ง 4 ตัว (C, A, Q, M) เชื่อมต่อกันอย่างไรในระบบฮาร์ดแวร์ และมีการทำลิงค์สี (Color-coding) เพื่อแสดงว่าข้อมูลแต่ละประเภทสัมพันธ์กับส่วนใดของผังข้อมูลกลาง
- ขั้นตอนการประมวลผล (Operational Flow Trace):** เป็นส่วนที่ใหญ่ที่สุดอยู่ตรงกลาง แสดงการทำงานทีละ รอบ (รอบที่ 1-6) โดยในแต่ละรอบจะมีผังย่อยแสดง:
 - การตัดสินใจ:** ดูค่า Q_0 เพื่อเลือกเส้นทางระหว่าง "บวก $A = A + M$ " (เมื่อ $Q_0=1$) หรือ "ไม่ต้องบวก" (เมื่อ $Q_0=0$)
 - สถานะรีจิสเตอร์:** แสดงค่าในรีจิสเตอร์ C, A, Q ทั้งก่อนและหลังการเลื่อนบิต (Shift Right) โดยใช้ค่าที่ให้อย่างครบถ้วน
- การเปรียบเทียบกับวิธีคูณด้วยมือ:** อยู่ทางด้านขวา แสดงตารางการคูณ 13×11 แบบเต็มรูปแบบ และมีการทำลิงค์สี (Color-coding) เพื่อแสดงว่าผลคูณย่อย (Partial Products) ในแต่ละบรรทัดของการคูณด้วยมือสัมพันธ์กับการบวกในรอบใดของฮาร์ดแวร์
- ผลลัพธ์และการแปลง:** อยู่ทางด้านซ้ายล่าง แสดงให้เห็นสถานะสุดท้ายของรีจิสเตอร์ A และ Q ที่มาต่อกันได้ผลรวม 12 บิต และเมื่อลดขนาดเหลือ 8 บิต จะได้ผลลัพธ์ 143_{10} ซึ่งตรงกัน

แผนภาพนี้จะช่วยให้เข้าใจได้ชัดเจนว่า แม้อุปกรณ์คอมพิวเตอร์จะไม่ได้คูณเลขเหมือนมนุษย์ แต่การวนซ้ำของกระบวนการบวกและเลื่อนบิตนั้นให้ผลลัพธ์ที่ถูกต้องตามหลักการคณิตศาสตร์ทุกประการ

หลักการทำงานของ Booth's Algorithm จะพิจารณา บิตของตัวคูณทีละคู่ ซึ่งประกอบด้วย บิตปัจจุบันของตัวคูณ (Q_n) และ บิตทางขวาของตัวคูณที่สมมติขึ้นมา (Q_{n-1}) ที่เริ่มต้นด้วยค่า 0 เสมอ แล้วจึงทำตามเงื่อนไขดังนี้:

ถ้าบิตคู่เป็น "00" หรือ "11": ไม่จำเป็นต้องทำการบวกหรือลบ

- ให้ทำการ Arithmetic Shift Right (เลื่อนบิตทางขวาแบบรักษาสัญญาณ) ไป 1 ตำแหน่ง พร้อมกันทั้งรีจิสเตอร์ A (Accumulator) และ Q (Multiplier) รวมถึง Q_{n-1} ด้วย

ถ้าบิตคู่เป็น "01": หมายความว่ากำลังเปลี่ยนจากลำดับ 0 เป็น 1 (สมมติ 00...0011...1)

- ให้ทำการ บวกตัวตั้ง (M) เข้าไปใน Accumulator ($A = A + M$)
- จากนั้นจึงทำการ Arithmetic Shift Right ไป 1 ตำแหน่งพร้อมกันทั้ง A, Q, และ Q_{n-1}

ถ้าบิตคู่เป็น "10": หมายความว่ากำลังเปลี่ยนจากลำดับ 1 เป็น 0 (สมมติ 11...1100...0)

- ให้ทำการ ลบตัวตั้ง (M) ออกจาก Accumulator ($A = A - M$) ซึ่งการลบในระบบดิจิทัลคือการ บวกด้วย 2's Complement ของ M ($-M$) นั่นเอง ($A = A + (-M)$)
- จากนั้นจึงทำการ Arithmetic Shift Right ไป 1 ตำแหน่งพร้อมกันทั้ง A, Q, และ Q_{n-1}

จำนวนรอบการทำงาน จะเท่ากับจำนวนบิตของตัวคูณ

4.6.1 ส่วนประกอบหลักของฮาร์ดแวร์สำหรับ Booth's Algorithm: คล้ายกับการคูณแบบพื้นฐาน แต่มีการจัดการบิต Q_{n-1} เพิ่มเติม:

- Multiplicand (M): รีจิสเตอร์สำหรับเก็บ ตัวตั้ง (ค่าคงที่ตลอดกระบวนการคูณ)
- Multiplier (Q): รีจิสเตอร์สำหรับเก็บ ตัวคูณ ซึ่งจะมีการเลื่อนบิตไปทางขวาในแต่ละรอบ
- Q_{n-1} : บิตพิเศษ ที่ใช้เก็บค่าบิตขวาสุดของ Q ที่ถูก Shift ออกไปในรอบก่อนหน้า เริ่มต้นด้วยค่า 0
- Accumulator (A): รีจิสเตอร์สำหรับเก็บ ผลรวมบางส่วน (Partial Product) และผลลัพธ์สุดท้าย เริ่มต้นด้วยค่า 0
- ALU (Arithmetic Logic Unit): ทำหน้าที่ในการบวก (+M) หรือลบ (-M หรือ +(-M))
- Control Logic: ส่วนควบคุมการทำงานของวงจร กำหนดว่าจะบวก, ลบ, หรือเลื่อนบิต และควบคุมการวนซ้ำ

1. กำหนด

$A, Q_{-1} = 0$, M คือ ตัวตั้ง, Q คือ ตัวคูณ, N คือ จำนวนบิตของตัวตั้ง (รอบการทำงาน)

2. เช็คว่า $Q_0 Q_{-1}$

ถ้าเท่ากับ 00, 11

ให้ Arithmetic Shift Right (A, Q, Q_{-1}) และ ลบค่า N ไป 1

ถ้าเท่ากับ 01

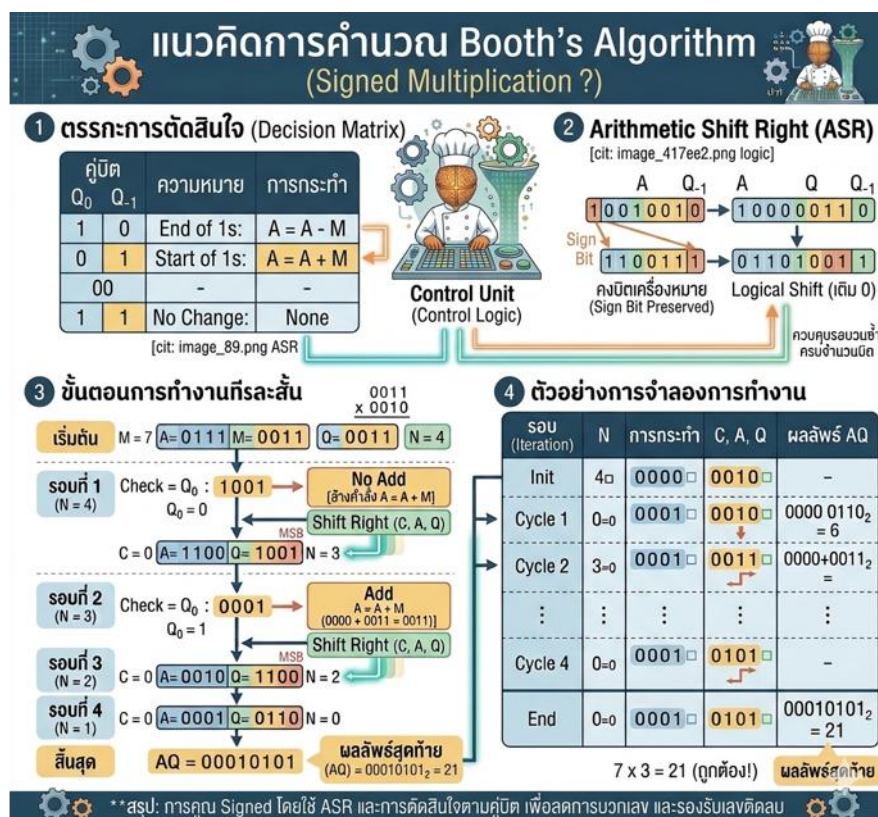
ให้ บวกค่า M เข้าที่ A คือ $A = A + M$ และทำการ Arithmetic Shift Right (A, Q, Q_{-1})
จากนั้นลบค่า N ไป 1

ถ้าเท่ากับ 10

ให้ ลบค่า M จาก A คือ $A = A - M$ และทำการ Arithmetic Shift Right (A, Q, Q_{-1}) จากนั้น
ลบค่า N ไป 1

3. ทำงานจนกว่า $N=0$ ให้จบการทำงาน

4.6.2 การคำนวณด้วยแนวคิดหลักของ Booth's Algorithm (ตัวอย่างการคูณ: $7 \times 3 =$ เท่าไหร่)



รูปที่ 6-26 รายละเอียดขั้นตอนการประมวลผลและตรรกะการ

คำนวณของอัลกอริทึมของบูธสำหรับการคูณเลขฐานสอง (7 คูณ 3)

แผนภาพเป็น ผังข้อมูลเชิงแนวคิดและเชิงปริมาณแบบบูรณาการ (Integrated Conceptual and Quantitative Trace-Map Infographic) โดยแบ่งออกเป็น 4 ส่วนหลักดังนี้ครับ:

1. แนวคิดหลัก (Core Logic) [ด้านซ้ายบน]: แสดง ตรรกะการตัดสินใจ (Decision Matrix) เป็นตารางอธิบายเงื่อนไขบิตคู่ 10 (ลบ), 01 (บวก), และ 00/11 (ไม่บวก/ลบ) และระบุว่าต้องทำ ASR เสมอหลังการดำเนินการ

2. หัวใจสำคัญ (Arithmetic Shift Right - ASR) [ด้านขวาบน]: แสดงแผนภาพอธิบายความหมายของ ASR โดยเน้นการ คงบิตเครื่องหมาย (Sign Bit Preserved) เปรียบเทียบกับ Logical Shift ทั่วไป เพื่อรักษาความเป็นเลขมีเครื่องหมาย
3. ขั้นตอนการทำงานที่ละขั้น (Iterative Operational Flow Trace) [ส่วนกลาง]: เป็นส่วนที่แสดงการทำงานจริงแบบที่ละรอบ (รอบที่ 1-4) โดยจำลองค่าในรีจิสเตอร์ C, A, Q, Q_{-1} ทั้งก่อนและหลังการดำเนินการและการเลื่อนบิตอย่างครบถ้วนตามโจทย์
4. ผลลัพธ์และสรุป (Result & Verification) [ด้านซ้ายล่าง]: แสดงการนำค่าสุดท้ายของ A และ Q มาต่อกัน ได้ผลรวม 8 บิต 00010101_2 พร้อมแสดงสูตรการแปลงกลับเป็นเลขฐานสิบและตรวจสอบผลลัพธ์กับโจทย์ 7 คูณ 3 = 21 (ถูกต้อง!)

การทำงาน Booth's Algorithm เป็นวิธีการคูณเลขฐานสองแบบมีเครื่องหมาย (2's Complement) ที่มีประสิทธิภาพสูงกว่าการคูณแบบพื้นฐาน โดยใช้การพิจารณาบิตของตัวคูณทีละคู่ เพื่อลดจำนวนการบวก/ลบให้น้อยลง โดยเฉพาะอย่างยิ่งเมื่อมีบิต 0 หรือ 1 ติดกันหลายบิต โดย การทำงานจะอาศัย

หลักการ: 00 หรือ 11: ไม่ต้องบวก/ลบ 01: บวกด้วยตัวตั้ง (M)

10: ลบด้วยตัวตั้ง (-M หรือบวกด้วย 2's Complement ของ M)

จากนั้นจึง Arithmetic Shift Right (เลื่อนบิตไปทางขวาแบบรักษาสัญญาณ) วงจรจะใช้รีจิสเตอร์ 3 ตัว:

- A (Accumulator): เก็บผลบวกสะสม Q (Multiplier): เก็บตัวคูณ
- M (Multiplicand): เก็บตัวตั้ง Q_{-1} (Q_{n-1}): บิตพิเศษสำหรับพิจารณาคู่บิต

1. กำหนดค่าเริ่มต้น (ตัวอย่าง: 7×3)

- ตัวตั้ง (M) = $7_{10} = 0111_2$
- ตัวคูณ (Q) = $3_{10} = 0011_2$
- จำนวนบิต (N) = 4
- $-M$ (2's Complement ของ M) = $-7 = 1001_2$
- A = 0000_2
- $Q_{-1} = 0$

2. รอบที่ 1: ตรวจสอบ Q_0 และ Q_{-1}

- Q_0 (บิตขวาสุดของ Q) = 1
- $Q_{-1} = 0$
- คู่บิต คือ 10
- การกระทำ: เนื่องจากเป็น 10 ให้ ลบ M ออกจาก A ($A = A - M$ หรือ $A = A + (-M)$)
 - $A = 0000_2 + 1001_2 = 1001_2$

- จากนั้น: ทำการ Arithmetic Shift Right A, Q และ Q_{-1}
 - A (1001) เลื่อนขวา บิตซ้ายสุด (1) ถูกคัดลอก ได้ 1100_1
 - Q (0011) เลื่อนขวา โดยรับบิตขวาสุดของ A (1) มาเป็นบิตซ้ายสุด ได้ 1001_2
 - Q_{-1} รับบิตขวาสุดของ Q (1) ได้ 1

- N ลดลงเหลือ 3

3. รอบที่ 2: ตรวจสอบ Q_0 และ Q_{-1}

- Q_0 (บิตขวาสุดของ Q) = 1
- $Q_{-1} = 1$
- คู่บิต คือ 11
- การกระทำ: เนื่องจากเป็น 11 ให้ ไม่ต้องบวก/ลบ
- จากนั้น: ทำการ Arithmetic Shift Right A, Q และ Q_{-1}
 - A (1100) เลื่อนขวา บิตซ้ายสุด (1) ถูกคัดลอก ได้ 1110_1
 - Q (1001) เลื่อนขวา โดยรับบิตขวาสุดของ A (0) มาเป็นบิตซ้ายสุด ได้ 0100_2
 - Q_{-1} รับบิตขวาสุดของ Q (1) ได้ 1
- N ลดลงเหลือ 2

4. รอบที่ 3: ตรวจสอบ Q_0 และ Q_{-1}

- Q_0 (บิตขวาสุดของ Q) = 0
- $Q_{-1} = 1$
- คู่บิต คือ 01
- การกระทำ: เนื่องจากเป็น 01 ให้ บวก M เข้า A ($A = A + M$)
 - $A = 1110_2 + 0111_2 = 0101_2$ (บิตที่เกินถูกทิ้ง)
- จากนั้น: ทำการ Arithmetic Shift Right A, Q และ Q_{-1}
 - A (0101) เลื่อนขวา บิตซ้ายสุด (0) ถูกคัดลอก ได้ 0010_1
 - Q (0100) เลื่อนขวา โดยรับบิตขวาสุดของ A (1) มาเป็นบิตซ้ายสุด ได้ 1010_2
 - Q_{-1} รับบิตขวาสุดของ Q (0) ได้ 0
- N ลดลงเหลือ 1

5. รอบที่ 4: ตรวจสอบ Q_0 และ Q_{-1}

- Q_0 (บิตขวาสุดของ Q) = 0
- $Q_{-1} = 0$

- คู่บิต คือ 00
- การกระทำ: เนื่องจากเป็น 00 ให้ ไม่ต้องบวก/ลบ
- จากนั้น: ทำการ Arithmetic Shift Right A, Q และ Q_{-1}
 - A (0010) เลื่อนขวา บิตซ้ายสุด (0) ถูกคัดลอก ได้ 0001_1
 - Q (1010) เลื่อนขวา โดยรับบิตขวาสุดของ A (0) มาเป็นบิตซ้ายสุด ได้ 0101_2
 - Q_{-1} รับบิตขวาสุดของ Q (0) ได้ 0
- N ลดลงเหลือ 0

6. สิ้นสุดการทำงาน

เมื่อ $N = 0$ การทำงานสิ้นสุดลง ผลลัพธ์สุดท้ายจะอยู่ที่การรวมกันของรีจิสเตอร์ A และ Q ซึ่งได้ 00010101_2 หรือ 00010101_2 ซึ่งแปลงกลับเป็นเลขฐานสิบได้เท่ากับ 21 และถูกต้องตรงกับ 7×3

ตัวอย่างการทําก่อนกลับบ้าน

1. จงแสดงวิธีการคูณเลขฐานสองขนาด 4 บิต (แบบไม่คิดเครื่องหมาย) โดยกำหนดให้ตัวตั้งของการคูณ = 8_{10} และ ตัวคูณ = 4_{10} โดยใช้วิธีการคูณแบบธรรมดา

ตัวอย่าง 1.1 หลักการคูณเลขฐานสองแบบไม่คิดเครื่องหมาย (แบบธรรมดา)

- การคูณเลขฐานสองแบบไม่คิดเครื่องหมายมีหลักการพื้นฐานที่คล้ายกับการคูณยาวด้วยมือในระบบเลขฐานสิบที่เราคุ้นเคยกันดี โดยใช้กฎการคูณที่เรียบง่ายเพียง 4 ข้อ
- $0 \times 0 = 0$
- $0 \times 1 = 0$
- $1 \times 0 = 0$
- $1 \times 1 = 1$

ขั้นตอนการคูณ: 8×4

1. แปลงเลขฐานสิบเป็นเลขฐานสอง

ขั้นแรก เราต้องแปลงตัวตั้ง (Multiplicand) และตัวคูณ (Multiplier) จากเลขฐานสิบเป็นเลขฐานสองขนาด 4 บิตก่อน

- ตัวตั้ง (Multiplicand): 8_{10} เท่ากับ 1000_2
- ตัวคูณ (Multiplier): 4_{10} เท่ากับ 0100_2

ดังนั้น โจทย์ของเราจะกลายเป็น $1000_2 \times 0100_2$

2. ทำการคูณแบบคูณยาว

เราจะนำบิตของตัวคูณ (0100_2) มาคูณกับตัวตั้ง (1000_2) ทีละบิต โดยเริ่มจากบิตขวาสุดไปทางซ้าย

- นำบิตที่ 0 (ขวาสุด) ของตัวคุณ คือ 0 ไปคูณกับตัวตั้ง 1000_2
 - $1000_2 \times 0 = 0000_2$
 - ได้ผลลัพธ์ย่อยบรรทัดแรกคือ 0000
- นำบิตที่ 1 ของตัวคุณ คือ 0 ไปคูณกับตัวตั้ง 1000_2
 - $1000_2 \times 0 = 0000_2$
 - จากนั้น เลื่อนผลลัพธ์ย่อยนี้ไปทางซ้าย 1 ตำแหน่ง
 - เราจะได้ 00000
- นำบิตที่ 2 ของตัวคุณ คือ 1 ไปคูณกับตัวตั้ง 1000_2
 - $1000_2 \times 1 = 1000_2$
 - เลื่อนผลลัพธ์ย่อยนี้ไปทางซ้าย 2 ตำแหน่ง
 - เราจะได้ 100000
- นำบิตที่ 3 (ซ้ายสุด) ของตัวคุณ คือ 0 ไปคูณกับตัวตั้ง 1000_2
 - $1000_2 \times 0 = 0000_2$
 - เลื่อนผลลัพธ์ย่อยนี้ไปทางซ้าย 3 ตำแหน่ง
 - เราจะได้ 0000000

3. นำผลลัพธ์ย่อยทั้งหมดมาบวกกัน

นำผลลัพธ์ย่อยที่ได้จากขั้นตอนที่ 2 มาบวกกันทั้งหมด

$$\begin{array}{r}
 0000_2 \quad (1000 \times 0) \\
 + 00000_2 \quad (1000 \times 0, \text{ เลื่อน } 1) \\
 + 100000_2 \quad (1000 \times 1, \text{ เลื่อน } 2) \\
 + 0000000_2 \quad (1000 \times 0, \text{ เลื่อน } 3) \\
 \hline
 100000_2
 \end{array}$$

เพื่อความถูกต้องในการบวก เราจะเพิ่ม 0 ให้

$$\begin{array}{r}
 00000000_2 \\
 + 00000000_2 \\
 + 00100000_2 \\
 + 00000000_2 \\
 \hline
 00100000_2
 \end{array}$$

ทุกบรรทัดมีจำนวนบิตเท่ากันก่อนบวก

ผลลัพธ์สุดท้ายที่ได้คือ 00100000_2 หรือ 100000_2 เมื่อตัด 0 ที่ไม่จำเป็นออก

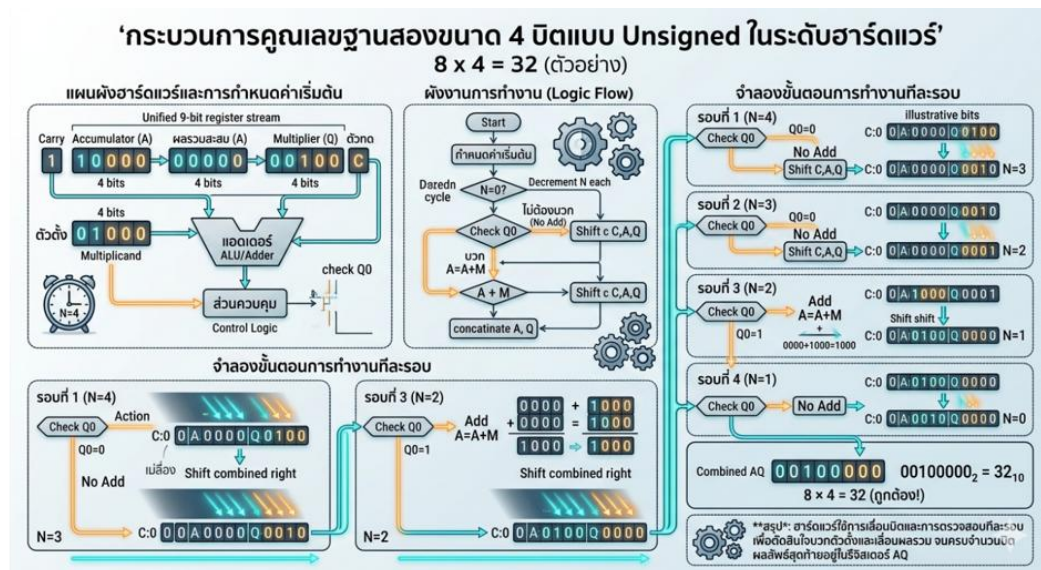
4. ตรวจสอบผลลัพธ์

แปลงผลลัพธ์ 100000_2 กลับเป็นเลขฐานสิบ

- $(1 \times 2^5) + (0 \times 2^4) + (0 \times 2^3) + (0 \times 2^2) + (0 \times 2^1) + (0 \times 2^0)$
- $32 + 0 + 0 + 0 + 0 + 0 = 32_{10}$ (การคำนวณ $8 \times 4 = 32$ ซึ่งถูกต้องตรงกับผลลัพธ์ที่ได้)

ตัวอย่าง 1.2 หลักการคูณเลขฐานสองแบบไม่คิดเครื่องหมาย (แบบในระดับฮาร์ดแวร์)

การคูณแบบนี้อาศัยหลักการเดียวกับการคูณยาวด้วยมือ แต่ในระดับฮาร์ดแวร์จะใช้วิธีการวนซ้ำของกระบวนการ "บวก" (Add) และ "เลื่อนบิต" (Shift) เพื่อให้ได้ผลลัพธ์สุดท้าย



รูปที่ 6-27 กระบวนการและขั้นตอนการจำลองทำงานของฮาร์ดแวร์คูณเลขฐานสองแบบไม่คิดเครื่องหมาย (Unsigned Binary Multiplication) ด้วยเทคนิคการบวกและเลื่อนบิต (Add-and-Shift) สำหรับตัวอย่าง 8 คูณ 4

อธิบายกระบวนการทำงานของฮาร์ดแวร์สำหรับการคูณเลขฐานสองขนาด 4 บิตแบบ Unsigned สำหรับตัวอย่าง 8 คูณ 4 = 32 แบบละเอียด โดยแบ่งเนื้อหาออกเป็น 3 ส่วนหลักดังนี้ครับ:

- แผนผังฮาร์ดแวร์และการกำหนดค่าเริ่มต้น (บนซ้าย):** แสดงส่วนประกอบหลักในระดับฮาร์ดแวร์ ประกอบด้วยรีจิสเตอร์ C (Carry), A (Accumulator), Q (Multiplier) และ M (Multiplicand) พร้อมด้วยแอดเดอร์ (ALU) และส่วนควบคุม (Control Logic) ที่ทำงานร่วมกัน นอกจากนี้ยังมีการกำหนดค่าเริ่มต้นสำหรับ 8 คูณ 4 ให้ชัดเจน
- ผังงานการทำงาน (Logic Flow) (กลางบน):** เป็นผังงานสรุปตรรกะการตัดสินใจในระดับวงจร ว่าในแต่ละรอบฮาร์ดแวร์จะดูเพียงบิตขวาสุดของ Q (Q_0) เท่านั้น ถ้าเป็น 1 จะทำการบวก $A = A + M$ และเลื่อนบิตไปทางขวาพร้อมกัน (Shift Right) แต่ถ้าเป็น 0 จะเลื่อนบิตเพียงอย่างเดียวจนครบจำนวนรอบ (N)
- จำลองขั้นตอนการทำงานทีละรอบ (Main Body Flow):** เป็นส่วนที่แสดงการทำงานจริงแบบจำลองทีละรอบ (รอบที่ 1-4) โดยในแต่ละรอบจะมีผังย่อยแสดง:
 - การตัดสินใจ:** ตรวจสอบ Q_0 เพื่อเลือกเส้นทางระหว่าง "ไม่ต้องบวก" (เมื่อ $Q_0=0$) หรือ "บวก $A = A + M$ " (เมื่อ $Q_0=1$)

- สถานะรีจิสเตอร์: แสดงสถานะของบิตในรีจิสเตอร์ C, A, Q ทั้งก่อนและหลังการเลื่อนบิต (Shift Right) พร้อมด้วยการทำเน้นสี (Color-coding) เพื่อแสดงว่าบิตตัวทที่หลุดจาก A (LSB ของ A) เลื่อนไปเป็นบิตหน้าสุดของ Q (MSB ของ Q) ตามกระบวนการเลื่อนบิตที่ถูกต้อง
 - ผลลัพธ์รอบถัดไป: แสดงสถานะใหม่ของ C, A, Q และค่า N ที่ลดลงครับ
4. ผลลัพธ์สุดท้าย: อยู่ด้านล่าง แสดงสถานะสุดท้ายของรีจิสเตอร์ A และ Q ที่มาต่อกันได้ผลรวม 8 บิต 00100000_2 พร้อมแสดงการแปลงกลับเป็นเลขฐานสิบตรวจสอบผลลัพธ์กับ 8 คูณ 4 = 32 (ถูกต้อง!) วงจรจะใช้รีจิสเตอร์หลัก 3 ตัว:

- M (Multiplicand): เก็บตัวตั้ง
- Q (Multiplier): เก็บตัวคูณ
- A (Accumulator): เก็บผลบวกสะสม ซึ่งจะเริ่มด้วยค่า 0

นอกจากนี้ยังมี C (Carry) บิตสำหรับเก็บตัวท และ N สำหรับนับรอบการทำงาน

1. กำหนดค่าเริ่มต้น (ขั้นตอนการทำงานของตัวอย่าง 8×4)

ก่อนเริ่มการคำนวณ เราต้องแปลงเลขฐานสิบเป็นเลขฐานสองขนาด 4 บิต และกำหนดค่าเริ่มต้นให้กับรีจิสเตอร์ต่างๆ

- ตัวตั้ง (M) = $8_{10} = 1000_2$ ตัวคูณ (Q) = $4_{10} = 0100_2$ รีจิสเตอร์ A = 0000_2
- รีจิสเตอร์ C = 0 จำนวนบิต (N) = 4

2. รอบที่ 1: ตรวจสอบ Q_0 ถ้า Q_0 (บิตขวาสุดของ Q) คือ 0

- การกระทำ: เนื่องจาก $Q_0 = 0$ จึง **ไม่ต้องบวก** ค่า M เข้า A
- จากนั้น: ทำการ Shift Right ทั้ง A และ Q ไปทางขวา 1 ตำแหน่ง โดยบิตขวาสุดของ A จะเลื่อนไปเป็นบิตซ้ายสุดของ Q และบิตขวาสุดของ Q จะถูกทิ้งไป
 - A จาก 0000 เลื่อนไปขวาได้ 0000
 - Q จาก 0100 เลื่อนไปขวาได้ 0010
- N ลดลงเหลือ 3

3. รอบที่ 2: ตรวจสอบ Q_0 ถ้า Q_0 (บิตขวาสุดของ Q) คือ 0

- การกระทำ: เนื่องจาก $Q_0 = 0$ จึง **ไม่ต้องบวก**
- จากนั้น: ทำการ Shift Right ทั้ง A และ Q
 - A จาก 0000 เลื่อนไปขวาได้ 0000
 - Q จาก 0010 เลื่อนไปขวาได้ 0001
- N ลดลงเหลือ 2

4. รอบที่ 3: ตรวจสอบ Q_0 ถ้า Q_0 (บิตขวาสุดของ Q) คือ 1

- การกระทำ: เนื่องจาก $Q_0 = 1$ ให้ บวกค่า M (1000_2) เข้า A ($A = A + M$)
 - $A = 0000_2 + 1000_2 = 1000_2$
- จากนั้น: ทำการ Shift Right ทั้ง A และ Q
 - A จาก 1000 เลื่อนไปขวาได้ 0100
 - Q จาก 0001 เลื่อนไปขวาโดยรับบิตจาก A ได้ 1000
- N ลดลงเหลือ 1

5. รอบที่ 4: ตรวจสอบ Q_0 ถ้า Q_0 (บิตขวาสุดของ Q) คือ 0

- การกระทำ: เนื่องจาก $Q_0 = 0$ จึง ไม่ต้องบวก
- จากนั้น: ทำการ Shift Right ทั้ง A และ Q
 - A จาก 0100 เลื่อนไปขวาได้ 0010
 - Q จาก 1000 เลื่อนไปขวาโดยรับบิตจาก A ได้ 0100
- N ลดลงเหลือ 0

ผลลัพธ์สุดท้าย

เมื่อ $N = 0$ การทำงานสิ้นสุดลง ผลลัพธ์สุดท้ายจะอยู่ที่การรวมกันของรีจิสเตอร์ A และ Q ซึ่งคือ

00100100_2

เมื่อนำ 00100100_2 แปลงกลับเป็นเลขฐานสิบจะได้ 32 ซึ่งถูกต้องตรงกับ 8×4

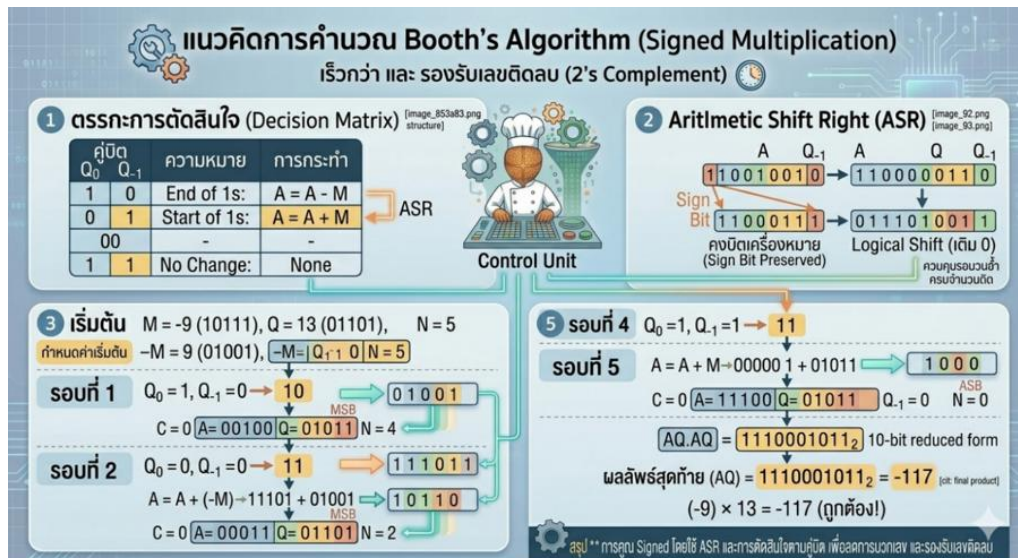
2.จงแสดงวิธีการคูณเลขฐานสองขนาด 5 บิต (แบบคิดเครื่องหมาย) โดยกำหนดให้ตัวตั้งของการคูณ = -9_{10} และตัวคูณ = 13_{10} โดยใช้วิธีการคูณแบบ Booth's Algorithm

ตัวอย่างที่ 2.1 หลักการทำงานของ Booth's Algorithm

Booth's Algorithm อาศัยหลักการบวกและลบตัวตั้งตามรูปแบบของบิตในตัวคูณ แทนที่จะเป็นการบวกซ้ำๆ ทุกบิตเหมือนการคูณแบบธรรมดา โดยมีหลักการพิจารณาบิตของตัวคูณทีละคู่ (บิตปัจจุบันและบิตก่อนหน้า) เพื่อตัดสินใจว่าจะดำเนินการอย่างไร

- บิตคู่ 00 หรือ 11: ไม่ต้องทำอะไร (No Add/Subtract)
- บิตคู่ 01: บวกตัวตั้ง (M)
- บิตคู่ 10: ลบตัวตั้ง (M) หรือบวกด้วย 2's Complement ของ M (-M)

เมื่อทำตามเงื่อนไขแล้ว จะทำการเลื่อนบิตทางขวาแบบรักษาสัญญาณ (Arithmetic Shift Right) ในรีจิสเตอร์ที่เกี่ยวข้อง



รูปที่ 6-28 แผนภาพแสดงตรรกะการทำงานและขั้นตอนการจำลองการคูณเลขฐานสอง

แบบมีเครื่องหมาย ด้วยอัลกอริทึมของบูธ (Booth's Algorithm) ตัวอย่าง -9 คูณ 13

2.2 อธิบายกระบวนการทำงานของ Booth's Algorithm สำหรับการคูณ $(-9) \times 13$ แบบละเอียด โดยแบ่งเนื้อหาออกเป็น 4 ส่วนหลักดังนี้:

- แนวคิดการคำนวณ Booth's Algorithm (บนขวา):** แสดงผังงาน (Flowchart) สรุปตรรกะการตัดสินใจในระดับวงจร ว่าฮาร์ดแวร์จะตรวจสอบคู่บิต Q_0, Q_{-1} เพื่อเลือกเส้นทางระหว่าง "ไม่ต้องบวก/ลบ" (No Change) หรือ "ลบ M" (End of 1s) หรือ "บวก M" (Start of 1s) จากนั้นจึงทำการเลื่อนบิตแบบ Arithmetic Shift Right (ASR) พร้อมกัน จนครบจำนวนรอบ (N)
- หัวใจสำคัญ: Arithmetic Shift Right (ASR) (บนซ้าย):** แสดงแผนภาพจำลองการเลื่อนบิตแบบ Arithmetic Shift Right ที่เน้นการ "คงบิตเครื่องหมาย (Sign Bit Preserved)" โดยแสดงให้เห็นว่าบิตหน้าสุดถูกคัดลอกเมื่อเลื่อนขวา ซึ่งแตกต่างจากการเลื่อนแบบ Logical Shift ทั่วไป เพื่อรักษาความเป็นเลขมีเครื่องหมาย (Signed Numbers)
- ขั้นตอนการทำงานทีละขั้น (Iterative Operational Flow Trace) [ส่วนกลาง]:** เป็นส่วนที่แสดงการทำงานจริงแบบทีละรอบ (รอบที่ 1-5) โดยจำลองค่าในรีจิสเตอร์ C, A, Q, Q_{-1} ทั้งก่อนและหลังการดำเนินการและการเลื่อนบิตอย่างครบถ้วนตามโจทย์ มีการทำเน้นสี (Color-coding) เพื่อแสดงว่าบิตตัวใดเลื่อนไปเป็นบิตหน้าสุดของ Q ตามกระบวนการเลื่อนบิตที่ถูกต้อง
- สรุปผลลัพธ์ (Result Panel):** อยู่ทางด้านซ้าย แสดงให้เห็นค่าสุดท้ายของรีจิสเตอร์ A และ Q ที่มาต่อกันได้ผลรวม 10 บิต 1110001011_2 พร้อมแสดงสูตรการแปลงกลับเป็นเลขฐานสิบและตรวจสอบผลลัพธ์กับโจทย์ $(-9) \times 13 = -117$ (ถูกต้อง!)

2.3 อธิบายการทำงานดังนี้

1. กำหนดค่าเริ่มต้น (ขั้นตอนการคูณ: $(-9) \times 13$)

เราต้องแปลงตัวตั้งและตัวคูณเป็นเลขฐานสองขนาด 5 บิตแบบมีเครื่องหมายก่อน:

- ตัวตั้ง (Multiplicand) = -9_{10}
 - ค่าบวก $9_{10} = 01001_2$
 - 2's Complement ของ 9 = $10111_2 \rightarrow M = 10111_2$
 - ค่าลบของ M = $-M = 9_{10} = 01001_2$
- ตัวคูณ (Multiplier) = 13_{10}
 - ค่าบวก $13_{10} = Q = 01101_2$
- กำหนดค่ารีจิสเตอร์เริ่มต้น
 - A (Accumulator) = 00000_2 Q_{-1} (บิตพิเศษ) = 0 N (จำนวนรอบ) = 5

2. รอบที่ 1: ตรวจสอบ Q_0 และ Q_{-1}

- $Q_0 = 1, Q_{-1} = 0 \rightarrow$ คู่บิต 10
- การกระทำ: ลบ M ออกจาก A ($A = A - M$ หรือ $A = A + (-M)$)
 - $A = 00000_2 + 01001_2 = 01001_2$
- Shift: ทำ Arithmetic Shift Right ทั้ง A, Q, และ Q_{-1}
 - A กลายเป็น 00100_2
 - Q กลายเป็น 10110_2 (รับบิต 1 จาก A มาแทนบิตที่เลื่อนออกไป)
 - Q_{-1} กลายเป็น 1 (รับบิต 0 จาก Q ที่ถูกเลื่อนออกไป)
- N ลดลงเหลือ 4

3. รอบที่ 2: ตรวจสอบ Q_0 และ Q_{-1}

- $Q_0 = 0, Q_{-1} = 1 \rightarrow$ คู่บิต 01
- การกระทำ: บวก M เข้า A ($A = A + M$)
 - $A = 00100_2 + 10111_2 = 11011_2$
- Shift: ทำ Arithmetic Shift Right ทั้ง A, Q, และ Q_{-1}
 - A กลายเป็น 11101_2
 - Q กลายเป็น 11011_2
 - Q_{-1} กลายเป็น 0
- N ลดลงเหลือ 3

4. รอบที่ 3: ตรวจสอบ Q_0 และ Q_{-1}

- $Q_0 = 1, Q_{-1} = 0 \rightarrow$ คู่บิต 10
- การกระทำ: ลบ M ออกจาก A ($A = A + (-M)$)
 - $A = 11101_2 + 01001_2 = 00110_2$
- Shift: ทำ Arithmetic Shift Right ทั้ง A, Q, และ Q_{-1}
 - A กลายเป็น 00011₂
 - Q กลายเป็น 01101₂
 - Q_{-1} กลายเป็น 1
- N ลดลงเหลือ 2

5. รอบที่ 4: ตรวจสอบ Q_0 และ Q_{-1}

- $Q_0 = 1, Q_{-1} = 1 \rightarrow$ คู่บิต 11
- การกระทำ: ไม่ต้องบวก/ลบ
- Shift: ทำ Arithmetic Shift Right ทั้ง A, Q, และ Q_{-1}
 - A กลายเป็น 00001₂
 - Q กลายเป็น 10110₂
 - Q_{-1} กลายเป็น 1
- N ลดลงเหลือ 1

6. รอบที่ 5: ตรวจสอบ Q_0 และ Q_{-1}

- $Q_0 = 0, Q_{-1} = 1 \rightarrow$ คู่บิต 01
- การกระทำ: บวก M เข้า A ($A = A + M$)
 - $A = 00001_2 + 10111_2 = 11000_2$
- Shift: ทำ Arithmetic Shift Right ทั้ง A, Q, และ Q_{-1}
 - A กลายเป็น 11100₂
 - Q กลายเป็น 01011₂
 - Q_{-1} กลายเป็น 0
- N ลดลงเหลือ 0

7. สรุปผลลัพธ์ เมื่อ $N = 0$ กระบวนการจะสิ้นสุดลง ผลลัพธ์สุดท้ายจะอยู่ที่การรวมกันของรีจิสเตอร์ A และ Q ซึ่งได้ 1110001011₂

- ผลลัพธ์ที่ได้คือ 1110001011₂ ซึ่งเป็นเลขฐานสองขนาด 10 บิต เมื่อแปลงกลับเป็นเลขฐานสิบ จะได้ -117 ซึ่งถูกต้องกับ $-9 * 13 = -117$

ตารางสรุปการทำงานของ Booth's Algorithm: $(-9) \times 13$

รอบ (N)	Q ₀ Q ₋₁	การกระทำ (Action)	A	Q	Q ₋₁
เริ่มต้น	-	Initialise	00000	01101	0
1 (4)	10	$A = A + (-M) \rightarrow A = 00000 + 01001 = 01001$ Shift Right (A, Q, Q ₋₁)	00100	10110	1
2 (3)	01	$A = A + M \rightarrow A = 00100 + 10111 = 11011$ Shift Right (A, Q, Q ₋₁)	11101	11011	0
3 (2)	10	$A = A + (-M) \rightarrow A = 11101 + 01001 = 00110$ Shift Right (A, Q, Q ₋₁)	00011	01101	1
4 (1)	11	No Add/Subtract Shift Right (A, Q, Q ₋₁)	00001	10110	1
5 (0)	01	$A = A + M \rightarrow A = 00001 + 10111 = 11000$ Shift Right (A, Q, Q ₋₁)	11100	01011	0
สิ้นสุด	-	Final Result	A (11100) + Q (01011) = 1110001011		

ตารางที่ 6-3 ตารางสรุปการทำงานของ Booth's Algorithm: $(-9) \times 13$

5. หลักการหารเลขฐานสอง (Binary Division) แบบเลขจำนวนเต็ม (Integer Division)

สำหรับการหารเลขฐานสอง (Binary Division) แบบหลักการหารเลขจำนวนเต็ม (Integer Division) นั้น มีความซับซ้อนกว่าการคูณ แต่ยังคงใช้หลักการเดียวกันกับการหารยาวในเลขฐานสิบที่เราคุ้นเคยกันดี โดยมี ส่วนประกอบหลักคือ:

- ตัวตั้ง (Dividend)
- ตัวหาร (Divisor)
- ผลหาร (Quotient)
- เศษ (Remainder)

การหารเลขฐานสอง (Binary Division) ในคอมพิวเตอร์นั้นใช้หลักการเดียวกันกับการหารยาวที่เราคุ้นเคยกันดี แต่มีความเรียบง่ายกว่ามากเพราะใช้แค่เลข 0 และ 1 เท่านั้น

หลักการพื้นฐาน

หลักการสำคัญของการหารเลขฐานสองคือการ **"เปรียบเทียบและลบ"** อย่างเป็นระบบ

1. **เปรียบเทียบ:** เริ่มจากบิตแรกทางซ้ายของตัวตั้ง (Dividend) แล้วนำมาเปรียบเทียบกับตัวหาร (Divisor)
 - กำหนดผลหาร:
 - ถ้าตัวตั้งมีค่า มากกว่าหรือเท่ากับ ตัวหาร: ผลหารในตำแหน่งนั้นคือ 1
 - ถ้าตัวตั้งมีค่า น้อยกว่า ตัวหาร: ผลหารในตำแหน่งนั้นคือ 0
2. **การลบ:** ถ้าผลหารเป็น 1 เราจะทำการลบตัวตั้งด้วยตัวหารเพื่อหาเศษเหลือ
3. **เลื่อนบิต:** นำบิตถัดไปจากตัวตั้งเดิมมาต่อท้ายเศษที่เหลือเพื่อสร้างตัวตั้งใหม่ แล้วทำซ้ำขั้นตอนเดิมไปเรื่อยๆ จนกว่าจะครบทุกหลัก



รูปที่ 6-29 ขั้นตอนการจำลองการหารเลขฐานสองแบบไม่คิดเครื่องหมาย (Unsigned Binary Division) ด้วยวิธีการหารยาว

กระบวนการ "เปรียบเทียบ-ลบ-เลื่อน" ของการหารในระบบคอมพิวเตอร์มี 4 ส่วนสำคัญ:

- **ตรรกะการเปรียบเทียบและการตัดสินใจ (Decision Matrix):** อธิบายเงื่อนไขพื้นฐานว่าถ้าตัวตั้งมากกว่าหรือเท่ากับตัวหาร ให้ใส่ผลหารเป็น 1 และทำการลบเพื่อหาเศษ แต่ถ้าตัวตั้งน้อยกว่าให้ใส่ผลหารเป็น 0
- **การจำลองขั้นตอนการหารที่ละรอบ (Operational Trace):** แสดงการทำงานตั้งแต่รอบที่ 1 ถึง 4 ตามจำนวนบิตของตัวตั้ง โดยระบุสถานะของบิตในแต่ละขั้นตอนอย่างละเอียด
 - รอบที่ 1 และ 2: ตัวตั้งน้อยกว่าตัวหาร จึงได้ผลหาร (Q) เป็น 0
 - รอบที่ 3 และ 4: ตัวตั้งมากกว่าตัวหาร จึงได้ผลหาร (Q) เป็น 1 และมีการลบตัวหารออกจากตัวตั้งเพื่อหาเศษเหลือ
- **ส่วนควบคุมการทำงาน (Control Unit):** แสดงภาพจำลองการทำงานของหน่วยควบคุมที่จัดการการดึงบิตถัดไปลงมา (Bring Down) และการตัดสินใจในระดับบิต
- **ผลลัพธ์สุดท้ายและการตรวจคำตอบ:** สรุปค่าผลหารที่ได้คือ 0011_2 (หรือ 3) และเศษเหลือคือ 10_2 (หรือ 2) พร้อมการตรวจสอบความถูกต้องด้วยสูตร $(3 * 3) + 2 = 11$

ตัวอย่าง: การหาร 10110_2 ด้วย 10_2 ($22_{10} \div 2_{10}$) ใช้หลักการหารยาวเพื่อแสดงกระบวนการทีละขั้นตอน

1. เตรียมการ:

- ตัวตั้ง (Dividend): 10110_2
- ตัวหาร (Divisor): 10_2
- สร้างผลหาร (Quotient): เว้นที่ว่างไว้สำหรับคำตอบ

2. ขั้นตอนการหาร: หลักการหารยาวเลขฐานสอง เพื่อให้เห็นภาพที่ชัดเจนขึ้นในแต่ละขั้นตอน

ขั้นตอน	การดำเนินการ	ตัวตั้ง (Current Dividend)	เปรียบเทียบกับตัวหาร (10_2)	ผลหาร (Quotient) ที่ได้	เศษที่เหลือ (Remainder)
1	พิจารณาบิตแรก (1)	1	$1 < 10$	0	1
2	ดึงบิตถัดไป (0)	10	$10 \geq 10$	1	$10 - 10 = 0$
3	ดึงบิตถัดไป (1)	01	$01 < 10$	0	1
4	ดึงบิตถัดไป (1)	11	$11 \geq 10$	1	$11 - 10 = 1$
5	ดึงบิตถัดไป (0)	10	$10 \geq 10$	1	$10 - 10 = 0$

ตารางที่ 6-4 ขั้นตอนการคำนวณการหารยาวเลขฐานสอง (Binary Long Division Analysis)

แสดงความสัมพันธ์ระหว่างตัวตั้ง ผลหาร และเศษเหลือในแต่ละจังหวะการประมวลผล

ผลลัพธ์สุดท้าย:

- ผลหาร (Quotient): 01011_2 หรือ 1011_2 (11_{10}) เศษเหลือ (Remainder): 0_2 (0_{10})

คำอธิบายแบบละเอียดทีละขั้นตอน

ขั้นตอนที่ 1

- การดำเนินการ: เราเริ่มต้นด้วยการพิจารณาบิตแรกของตัวตั้ง (10110_2) ซึ่งก็คือบิตแรกทางซ้ายสุด 1
- ตัวตั้ง: ณ จุดนี้ ตัวตั้งของเราคือ 1_2
- เปรียบเทียบ: นำ 1_2 ไปเปรียบเทียบกับตัวหาร (10_2)
ซึ่งมีค่าเท่ากับ 2_{10} เราพบว่า $1 < 10$ (1_{10} น้อยกว่า 2_{10})
ถ้าตัวตั้งมีค่า มากกว่าหรือเท่ากับ ตัวหาร: ผลหารในตำแหน่งนั้นคือ 1
ถ้าตัวตั้งมีค่า น้อยกว่า ตัวหาร: ผลหารในตำแหน่งนั้นคือ 0
- ผลหาร: เนื่องจากตัวตั้งมีค่าน้อยกว่าตัวหาร ผลหารในตำแหน่งนี้จึงเป็น 0
- เศษเหลือ: เศษที่เหลือคือตัวตั้งเดิม คือ 1

ขั้นตอนที่ 2

- การดำเนินการ: ดึงบิตถัดไปจากตัวตั้งเดิม (10110_2) นั่นคือ 0 มาต่อท้ายเศษที่เหลือจากขั้นตอนที่ 1
กลายเป็น 10_2 ถ้าตัวตั้งมีค่า มากกว่าหรือเท่ากับ ตัวหาร: ผลหารในตำแหน่งนั้นคือ 1
- ตัวตั้ง: ณ จุดนี้ ตัวตั้งของเราคือ 10_2 ถ้าตัวตั้งมีค่า น้อยกว่า ตัวหาร: ผลหารในตำแหน่งนั้นคือ 0
- เปรียบเทียบ: นำ 10_2 ไปเปรียบเทียบกับตัวหาร (10_2) เราพบว่า $10_2 \geq 10_2$ (2_{10} เท่ากับ 2_{10})
- ผลหาร: เนื่องจากตัวตั้งมีค่าเท่ากับตัวหาร ผลหารในตำแหน่งนี้จึงเป็น 1
- เศษเหลือ: เราทำการลบ $10_2 - 10_2$ ได้ 0

ขั้นตอนที่ 3

- การดำเนินการ: ดึงบิตถัดไปจากตัวตั้งเดิม (10110_2) นั่นคือ 1 มาต่อท้ายเศษที่เหลือจากขั้นตอนที่ 2
กลายเป็น 01_2
- ตัวตั้ง: ณ จุดนี้ ตัวตั้งของเราคือ 01_2
- เปรียบเทียบ: นำ 01_2 ไปเปรียบเทียบกับตัวหาร (10_2) เราพบว่า $01_2 < 10_2$ (1_{10} น้อยกว่า 2_{10})
- ผลหาร: ผลหารในตำแหน่งนี้จึงเป็น 0 ถ้าตัวตั้งมีค่า มากกว่าหรือเท่ากับ ตัวหาร: ผลหารในตำแหน่งนั้นคือ 1
- เศษเหลือ: เศษที่เหลือคือตัวตั้งเดิม คือ 01 ถ้าตัวตั้งมีค่า น้อยกว่า ตัวหาร: ผลหารในตำแหน่งนั้นคือ 0

ขั้นตอนที่ 4

- การดำเนินการ: ดึงบิตถัดไปจากตัวตั้งเดิม (10110_2) นั่นคือ 1 มาต่อท้ายเศษที่เหลือจากขั้นตอนที่ 3
กลายเป็น 11_2
- ตัวตั้ง: ณ จุดนี้ ตัวตั้งของเราคือ 11_2
- เปรียบเทียบ: นำ 11_2 ไปเปรียบเทียบกับตัวหาร (10_2) เราพบว่า $11_2 \geq 10_2$ (3_{10} มากกว่า 2_{10})
- ผลหาร: ผลหารในตำแหน่งนี้จึงเป็น 1 ถ้าตัวตั้งมีค่า มากกว่าหรือเท่ากับ ตัวหาร: ผลหารในตำแหน่งนั้นคือ 1
- เศษเหลือ: เราทำการลบ $11_2 - 10_2$ ได้ 1 ถ้าตัวตั้งมีค่า น้อยกว่า ตัวหาร: ผลหารในตำแหน่งนั้นคือ 0

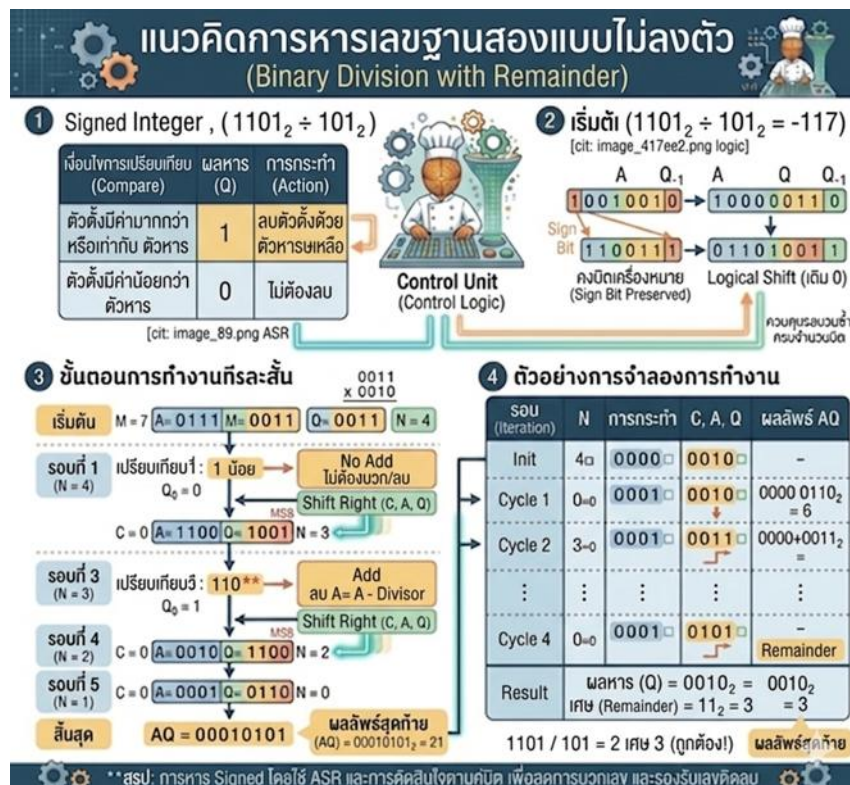
ขั้นตอนที่ 5

- การดำเนินการ: ดึงบิตสุดท้ายจากตัวตั้งเดิม ($1011\underline{0}_2$) นั่นคือ **0** มาต่อท้ายเศษที่เหลือจากขั้นตอนที่ 4 กลายเป็น **10**
- ตัวตั้ง: ณ จุดนี้ ตัวตั้งของเราคือ **10**
- เปรียบเทียบ: นำ **10** ไปเปรียบเทียบกับตัวหาร (10_2) เราพบว่า $10_2 \geq 10_2$ (2_{10} เท่ากับ 2_{10})
- ผลหาร: ผลหารในตำแหน่งนี้จึงเป็น **1** ถ้าตัวตั้งมีค่า มากกว่าหรือเท่ากับ ตัวหาร: ผลหารในตำแหน่งนั้นคือ 1
- เศษเหลือ: เราทำการลบ $10-10$ ได้ **0** ถ้าตัวตั้งมีค่า น้อยกว่า ตัวหาร: ผลหารในตำแหน่งนั้นคือ 0

สรุปผลลัพธ์: เมื่อสิ้นสุดกระบวนการ เราจะได้ผลหาร **01011₂** และเศษเหลือ **0₂** ซึ่งสอดคล้องกับการคำนวณในเลขฐานสิบ ($22 \div 2 = 11$ เศษ 0)

จากตัวอย่างนี้ จะเห็นว่าการทำงานของ CPU นั้นเป็นไปตามขั้นตอนที่ได้อธิบายไว้ โดยจะเปรียบเทียบและลบไปทีละบิต จนกว่าจะครบทุกหลักของตัวตั้ง

5.1 การหารเลขฐานสอง (Binary Division) แบบไม่ลงตัว เศษเหลือ (Remainder)



รูปที่ 6-30 ขั้นตอนการคำนวณการหารเลขฐานสองแบบมีเศษเหลือ (Binary Division with Remainder)

เพื่อให้เข้าใจกระบวนการหารเลขฐานสองที่ไม่ลงตัว ยกตัวอย่าง 13_{10} หารด้วย 5_{10} ซึ่งในเลขฐานสองคือ 1101_2 หารด้วย 101_2 หลักการยังคงเหมือนเดิม คือการเปรียบเทียบและลบไปทีละขั้น จนกระทั่งตัวตั้งสุดท้ายมีค่าน้อยกว่าตัวหาร ซึ่งตัวตั้งนั้นจะกลายเป็น **เศษเหลือ (Remainder)** ที่ไม่เป็นศูนย์

แบบที่ 1 ตัวอย่าง: $1101_2 \div 101_2$ ($13_{10} \div 5_{10}$)

1. เตรียมการ:

- ตัวตั้ง (Dividend): 1101_2
- ตัวหาร (Divisor): 101_2
- ผลหาร (Quotient): เว้นที่ว่างไว้สำหรับคำตอบ

2. ขั้นตอนการหาร:

ขั้นตอน	การดำเนินการ	ตัวตั้ง (Current Dividend)	เปรียบเทียบกับตัวหาร (101_2)	ผลหาร (Quotient) ที่ได้	เศษที่เหลือ (Remainder)
1	พิจารณาบิตแรก (1)	1	$1 < 101$	0	1
2	ดึงบิตถัดไป (1)	11	$11 < 101$	00	11
3	ดึงบิตถัดไป (0)	110	$110 \geq 101$	001	$110 - 101 = 1$
4	ดึงบิตถัดไป (1)	11	$11 < 101$	0010	11

ตารางที่ 6-5 ขั้นตอนตารางแสดงขั้นตอนการหารยาวเลขฐานสองแบบมีเศษเหลือ

3. คำอธิบายแบบละเอียดทีละขั้นตอนของ $1101_2 \div 101_2$ ($13_{10} \div 5_{10}$)

ขั้นตอนที่ 1

- การดำเนินการ: เริ่มต้นจากบิตแรกทางซ้ายสุดของตัวตั้ง **1**
- เปรียบเทียบ: นำ **1** ไปเปรียบเทียบกับตัวหาร 101_2 ($1_{10} < 5_{10}$) เนื่องจากตัวตั้งมีค่าน้อยกว่า ผลหารในตำแหน่งนี้จึงเป็น 0 และเศษที่เหลือคือ 1

ขั้นตอนที่ 2

- การดำเนินการ: ดึงบิตถัดไปคือ 1 มาต่อท้ายเศษเดิม กลายเป็น 11
- เปรียบเทียบ: นำ 11 ไปเปรียบเทียบกับตัวหาร 101_2 ($3 < 5$) เนื่องจากยังน้อยกว่า ผลหารในตำแหน่งนี้จึงเป็น 0 และเศษยังคงเป็น 11

ขั้นตอนที่ 3

- การดำเนินการ: ดึงบิตถัดไปคือ 0 มาต่อท้ายเศษเดิม กลายเป็น 110
- เปรียบเทียบ: นำ 110 ไปเปรียบเทียบกับตัวหาร 101_2 ($6 \geq 5$) เนื่องจากมากกว่า ผลหารในตำแหน่งนี้จึงเป็น 1 และทำการลบ $110 - 101$ ได้เศษเป็น 1

ขั้นตอนที่ 4

- การดำเนินการ: ดึงบิตสุดท้ายคือ 1 มาต่อท้ายเศษที่เหลือจากขั้นตอนที่ 3 กลายเป็น 11

- **เปรียบเทียบ:** นำ 11 ไปเปรียบเทียบกับตัวหาร 101 ($3 < 5$) เนื่องจากน้อยกว่า ผลหารในตำแหน่งนี้จึงเป็น 0 และกระบวนการสิ้นสุดลงเพราะไม่มีบิตให้ตั้งอีก

4. สรุปผลลัพธ์

- **ผลหาร (Quotient):** เรานำผลหารในแต่ละขั้นตอนมาเรียงต่อกันจะได้ 0010_2 หรือ 10_2 ซึ่งมีค่าเท่ากับ 2_{10}

- **เศษเหลือ (Remainder):** ตัวเลขสุดท้ายที่เราได้จากการเปรียบเทียบคือ 11_2 ซึ่งมีค่าเท่ากับ 3_{10} ดังนั้น $1101_2 \div 101_2$ จะได้ผลลัพธ์ 10_2 เศษ 11_2 ซึ่งตรงกับผลลัพธ์ในเลขฐานสิบ ($13 \div 5 = 2$ เศษ 3)

แบบที่ 2 ตัวอย่าง: $1101_2 \div 101_2$ ($13_{10} \div 5_{10}$) การหารเลขฐานสองที่ไม่ลงตัวให้หลักการเดิมของการหารยาว

ขั้นตอนที่ 1: การเตรียมการ

ในขั้นตอนนี้ เราจะวางตัวเลขให้อยู่ในรูปแบบการหารยาวแบบมาตรฐาน เพื่อให้ง่ายต่อการมองเห็นขั้นตอนต่างๆ:

- **ตัวตั้ง (Dividend):** 1101_2
- **ตัวหาร (Divisor):** 101_2

$$\begin{array}{r} \text{----} \\ 101 \overline{) 1101} \end{array}$$

เราจะเขียนโครงสร้างการหารยาวดังนี้:

ขั้นตอนที่ 2: การพิจารณาบิตแรก (จากซ้ายไปขวา)

เราจะเริ่มจากซ้ายมือสุดของตัวตั้ง (Dividend) แล้วพิจารณาจำนวนบิตที่เท่ากับความยาวของตัวหาร (3 บิต):

- **พิจารณา 3 บิตแรกของตัวตั้ง:** 110
- **เปรียบเทียบ:** 110_2 (6_{10}) มากกว่า 101_2 (5_{10})
- **ผลหาร:** เนื่องจากมีค่ามากกว่าหรือเท่ากับ ดังนั้นผลหารในตำแหน่งนี้คือ 1
- **ใส่ผลหารและลบ:** เขียน 1 ไว้บนตำแหน่งบิตสุดท้ายของกลุ่มที่พิจารณา (110) แล้วนำตัวหารมาลบ:
- **เศษเหลือ:** ผลลัพธ์จากการลบคือ 01_2 หรือ 1_{10}

$$\begin{array}{r} \text{--}1\text{--} \\ 101 \overline{) 1101} \\ \underline{-101} \\ \text{----} \\ 01 \end{array}$$

ขั้นตอนที่ 3: ดึงบิตถัดไป

เราจะดึงบิตถัดไปจากตัวตั้งลงมาต่อท้ายเศษที่เหลือจากการลบ เพื่อสร้างตัวเลขชุดใหม่สำหรับพิจารณาในขั้นตอนต่อไป:

- **เศษที่เหลือ:** 01_2
- **ดึงบิตถัดไป:** ดึงบิตสุดท้ายของตัวตั้งคือ 1 ลงมาต่อท้าย
- **ตัวตั้งใหม่:** กลายเป็น 011_2 หรือ 11_2

$$\begin{array}{r} \text{--}1\text{--} \\ 101 \overline{) 1101} \\ \underline{-101} \\ \text{----} \\ 011 \end{array}$$

โครงสร้างการหารยาวจะกลายเป็น:

ขั้นตอนที่ 4: ทำซ้ำขั้นตอนเดิม เราจะทำซ้ำกระบวนการเปรียบเทียบและลบกับตัวตั้งชุดใหม่นี้:

- พิจารณาตัวตั้งใหม่: 011_2
- เปรียบเทียบ: $011_2 (3_{10})$ น้อยกว่า $101_2 (5_{10})$
- ผลหาร: เนื่องจากมีค่าน้อยกว่า ผลหารในตำแหน่งนี้คือ 0
- ใส่ผลหาร: เขียน 0 ไว้บนตำแหน่งถัดไปของผลหาร:
- เศษเหลือ: เนื่องจากตัวตั้งน้อยกว่าตัวหาร จึงไม่มีการลบเกิดขึ้น และเศษสุดท้ายคือ 011_2

```

      _10_
101|1101
  -101
  ----
    011
    
```

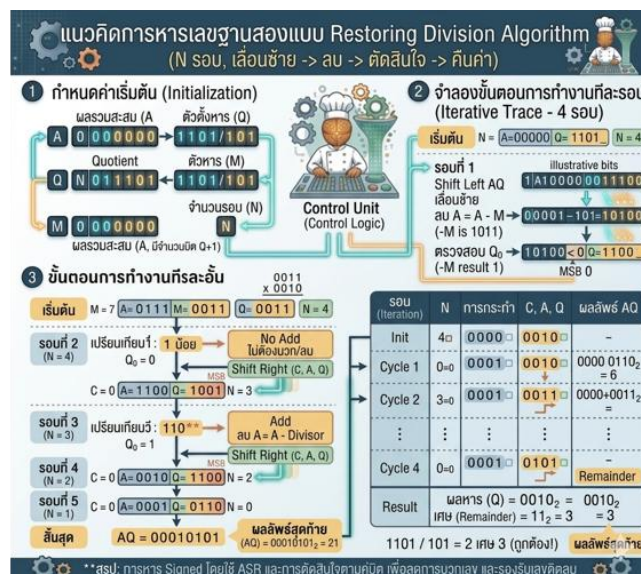
ขั้นตอนที่ 5: สรุปผลลัพธ์ เมื่อไม่มีบิตในตัวตั้งให้ดึงลงมาอีกแล้ว กระบวนการหารจะสิ้นสุดลง:

- ผลหาร (Quotient): ตัวเลขที่อยู่ด้านบนสุดคือ 10_2
- เศษเหลือ (Remainder): ตัวเลขสุดท้ายที่เหลืออยู่คือ 11_2

ดังนั้น $1101_2 \div 101_2$ เท่ากับ (ผลหาร 10_2 เศษเหลือ 11_2) ซึ่งเป็นคำตอบที่ถูกต้องตามหลักคณิตศาสตร์ ($13 \div 5 = 2$ เศษ 3)

5.2 หลักการหารเลขฐานสองด้วยแบบอัลกอริทึม (Restoring Division Algorithm)

Restoring Division Algorithm เป็นวิธีการหารเลขฐานสองแบบวนซ้ำ ซึ่งมีการตรวจสอบผลลัพธ์ของการลบในแต่ละรอบ หากผลลัพธ์ติดลบจะทำการ "คืนค่า" หรือ Restore ค่า A กลับไปก่อนหน้า



รูปที่ 6-31 ขั้นตอนการจำลองรีจิสเตอร์ของอัลกอริทึมการ

หารแบบคืนค่า สำหรับเลขฐานสองแบบไม่คิดเครื่องหมาย

อธิบายกระบวนการทำงานของ Restoring Division Algorithm สำหรับการหารเลขฐานสองแบบไม่คิดเครื่องหมาย (Unsigned Binary Division) แบบละเอียด โดยแบ่งเนื้อหาออกเป็น 4 ส่วนหลัก Conceptual Linked areas ดังนี้

1. **กำหนดค่าเริ่มต้น (Initialization) [บนซ้าย]:** แสดงผังการจัดวางรีจิสเตอร์ A (Accumulator), Q (Quotient), M (Divisor) พร้อมระบุเงื่อนไขขนาดบิตเริ่มต้นของ A ที่มีจำนวนบิต Q+1 เพื่อรองรับค่าติดลบชั่วคราว มีการแสดง data flow arrows ชัดเจนของกระบวนการ Initialization
2. **จำลองขั้นตอนการทำงานทีละขั้น (Operational Flow) [ส่วนกลาง]:** โดยแสดงการทำงานจริงแบบจำลองทีละ รอบ (รอบที่ 1-4) โดยระบุสถานะของบิตในรีจิสเตอร์ A และ Q ทั้งก่อนและหลังการดำเนินการทุกขั้นตอน (Shift, Sub, Restore) ตลอดจนการตรวจสอบบิตหน้าสุด (MSB) ของ A เพื่อตัดสินใจกำหนดค่า Q₀ (1 หรือ 0) มีการทำ color-coding ในขั้นตอน 'ตัดสินใจ/คืนค่า' เพื่อเน้นย้ำความแตกต่างระหว่าง "A<0" และ "A>=0"
3. **แนวคิดหลัก (Core Concept) [บนขวา]:** แสดงผังงาน (Flowchart) สรุปตรรกะการตัดสินใจในระดับวงจร ว่าฮาร์ดแวร์จะดูเพียงบิตหน้าสุดของ A เท่านั้น ถ้าเป็น 0 แสดงว่าหารได้ เราจะใส่ Q₀=1 และทำ steps ต่อไป แต่ถ้าเป็น 1 แสดงว่าหารไม่ได้ เราจะใส่ Q₀=0 และทำขั้นตอน Restore (A=A+M) เพื่อเอาค่าเดิมกลับคืนมา
4. **ผลลัพธ์สุดท้าย (Result Panel) [ล่างขวา]:** แสดงให้เห็นค่าสุดท้ายของรีจิสเตอร์ A และ Q ที่มาต่อกันได้ผลหาร 4 บิตและเศษเหลือ 4 บิต พร้อมแสดงสูตรการแปลงกลับเป็นเลขฐานสิบและตรวจสอบผลลัพธ์กับโจทย์ตัวอย่าง $1101_2 \div 101_2$ (ถูกต้อง!)

หลักการทำงานเบื้องต้น

1. **กำหนดค่าเริ่มต้น:**
 - A (Accumulator): เริ่มต้นที่ 0 มีจำนวนบิตเท่ากับจำนวนบิตของ Q บวก 1 (เพื่อรองรับค่าติดลบชั่วคราวจากการลบ) M (Divisor): ตัวหาร
 - Q (Quotient): ตัวตั้งหาร N: จำนวนบิตของ Q (คือจำนวนรอบที่ต้องทำซ้ำ)
2. **วนซ้ำ (N รอบ):** ในแต่ละรอบจะมีการทำ 3 ขั้นตอนหลัก:
 - Shift Left AQ: เลื่อนบิตของ A และ Q ไปทางซ้าย 1 บิต โดยบิตซ้ายสุดของ Q จะย้ายไปเป็นบิตขวาสุดของ A
 - A = A - M: นำค่าใน A ลบด้วย M
 - ตรวจสอบและกำหนดค่า Q₀:
 - ถ้าผลลัพธ์ A เป็นบวก (บิตซ้ายสุดของ A เป็น 0): กำหนดให้บิตขวาสุดของ Q (Q₀) เป็น 1
 - ถ้าผลลัพธ์ A เป็นลบ (บิตซ้ายสุดของ A เป็น 1): กำหนดให้บิตขวาสุดของ Q (Q₀) เป็น 0 และทำการ "Restore" ค่า A กลับมาเป็นค่าก่อนหน้าที่จะลบ (A = A + M)
3. **สิ้นสุด:** ทำซ้ำจนครบ N รอบ เมื่อ N เป็น 0
 - Q: คือผลหาร A: คือเศษ

ตัวอย่าง: หาร 1010_2 (10) ด้วย 0011_2 (3)

โดยใช้ตารางอธิบายเพื่อแสดงการเปลี่ยนแปลงค่าในแต่ละขั้นตอน

กำหนดค่าเริ่มต้น:

- Q (ตัวตั้ง) = 1010_2 (4 บิต)
- M (ตัวหาร) = 0011_2 (สำหรับ 4 บิต เราจะใช้ 5 บิตเพื่อรองรับการลบ เช่น 00011_2)
- $A = 00000_2$ (5 บิต)
- $N = 4$ (จำนวนบิตของตัวตั้ง)

รอบ (N)	การดำเนินการ	A (5 บิต)	Q (4 บิต)	Q_0	หมายเหตุ
เริ่มต้น		00000_2	1010_2		
---	---	---	---	---	---
1 (N=4)	1. Shift Left AQ	00000_2	1010_2 (บิตซ้ายสุดของ Q (1_2) เลื่อนไปต่อท้าย A)		$AQ = 000001010_2$
	2. $A = A - M$	$00000_2 - 00011_2$			ผลลัพธ์ติดลบ
		11101_2			(ถ้าเรากำหนดแบบ Two's Complement)
	3. ตรวจสอบ A	บิตซ้ายสุดของ A เป็น 1 (ติดลบ)		0	กำหนด $Q_0=0$
	4. Restore A	$11101_2 + 00011_2 = 00000_2$			คืนค่า A เดิม
	N ลดลง				$N = 3$
---	---	---	---	---	---
2 (N=3)	1. Shift Left AQ	00000_2	1010_2 (Q เลื่อนไป 1 บิต, Q_0 ที่ กำหนดไว้ 0_2 เป็นบิตขวาสุด) $\rightarrow$ $Q = 0100_2$		$AQ = 000001010_2$
		$00000 \leftarrow 1$	$0100 \leftarrow 0$		
	2. $A = A - M$	$00001_2 - 00011_2$			ผลลัพธ์ติดลบ
		11110_2			
	3. ตรวจสอบ A	บิตซ้ายสุดของ A เป็น 1 (ติดลบ)		0	กำหนด $Q_0=0$
	4. Restore A	$11110_2 + 00011_2 = 00001_2$			คืนค่า A เดิม

รอบ (N)	การดำเนินการ	A (5 บิต)	Q (4 บิต)	Q ₀	หมายเหตุ
	N ลดลง				N = 2
---	---	---	---	---	---
3 (N=2)	1. Shift Left AQ	00001 ₂	1000 ₂ (Q เลื่อนไป 1 บิต, Q ₀ ที่ กำหนดไว้ 0 ₂ เป็นบิตขวาสุด) → Q = 0000 ₂		AQ = 000101000 ₂
		00010←1	0000←0		
	2. A = A - M	00010 ₂ - 00011 ₂			ผลลัพธ์ติดลบ
		11111 ₂			
	3. ตรวจสอบ A	บิตซ้ายสุดของ A เป็น 1 (ติดลบ)		0	กำหนด Q ₀ =0
	4. Restore A	11111 ₂ +00011 ₂ =00010 ₂			คืนค่า A เดิม
	N ลดลง				N = 1
---	---	---	---	---	---
4 (N=1)	1. Shift Left AQ	00010 ₂	0000 ₂ (Q เลื่อนไป 1 บิต, Q ₀ ที่ กำหนดไว้ 0 ₂ เป็นบิตขวาสุด) → Q = 0000 ₂		AQ = 001010000 ₂
		00101←0	0000←0		
	2. A = A - M	00101 ₂ -00011 ₂			ผลลัพธ์เป็นบวก
		00010 ₂			
	3. ตรวจสอบ A	บิตซ้ายสุดของ A เป็น 0 (เป็นบวก)		1	กำหนด Q ₀ =1
	4. Restore A	ไม่มี (ไม่ต้อง Restore)			
	N ลดลง				N = 0

ตารางที่ 6-6 ตารางอธิบายเพื่อแสดงการเปลี่ยนแปลงค่าในแต่ละขั้นตอน

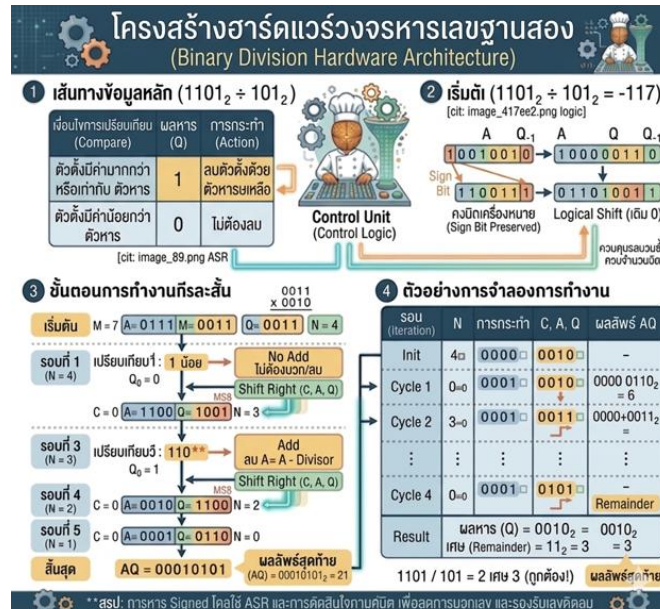
ผลลัพธ์สุดท้าย เมื่อ N = 0

- ผลหาร (Q) = 0010₂ (ซึ่งคือ 2 ในฐานสิบ)
- เศษ (A) = 00010₂ (ซึ่งคือ 2 ในฐานสิบ)

ดังนั้น 1010₂ หารด้วย 0011₂ ได้ผลลัพธ์ 0010₂ เศษ 00010₂ หรือ 10 หารด้วย 3 ได้ 2 เศษ 2 นั่นเอง

5.3 ส่วนประกอบหลักของวงจรหารเลขฐานสอง (Hardware for Binary Division):

การหารเลขฐานสองด้วยฮาร์ดแวร์ส่วนใหญ่จะอิงตามหลักการของ Restoring Division Algorithm หรือ Non-Restoring Division Algorithm ซึ่งทั้งสองวิธีมีโครงสร้างฮาร์ดแวร์พื้นฐานที่คล้ายคลึงกัน โดยมี ส่วนประกอบหลักดังนี้:



รูปที่ 6-32 โครงสร้างสถาปัตยกรรมฮาร์ดแวร์ของวงจรหารเลขฐานสอง (Binary Division Hardware Architecture) และความสัมพันธ์ของรีจิสเตอร์ในการประมวลผล

ส่วนประกอบหลักของวงจรหาร:

1. รีจิสเตอร์ Q (Quotient Register):

- หน้าที่: ใช้เก็บ ตัวตั้ง (Dividend) ในตอนเริ่มต้น และเมื่อกระบวนการหารเสร็จสิ้น จะใช้เก็บ ผลหาร (Quotient)
- ขนาด: มักจะมีขนาดเท่ากับจำนวนบิตของตัวตั้ง (เช่น ถ้าตัวตั้ง 4 บิต, Q ก็ 4 บิต)
- การทำงาน: บิตของ Q จะถูกเลื่อนไปทางซ้ายพร้อมกับ A ในทุกๆ รอบ (Shift Left AQ) และบิตขวาสุดของ Q (Q₀) จะถูกกำหนดค่าเป็น 0 หรือ 1 ตามผลลัพธ์ของการลบ

2. รีจิสเตอร์ M (Divisor Register):

- หน้าที่: ใช้เก็บ ตัวหาร (Divisor)
- ขนาด: ขนาดของรีจิสเตอร์ M มักจะถูกออกแบบให้มีขนาดใหญ่กว่าตัวหาร 1 บิตทางด้านซ้ายสุด (Sign Bit) เพื่อรองรับการดำเนินการลบที่อาจทำให้เกิดค่าติดลบ (เช่น ถ้าตัวหาร 4 บิต อาจใช้ M 5 บิต)
- การทำงาน: ค่าใน M จะคงที่ตลอดกระบวนการ และถูกใช้เป็นตัวลบ (subtrahend) ในการดำเนินการ A - M

3. รีจิสเตอร์ A (Accumulator / Remainder Register):

○ หน้าที่:

- เริ่มต้น: กำหนดค่าเป็นศูนย์ ($A = 0$) เสมอ
- ระหว่างการหาร: ทำหน้าที่เป็นตัวสะสม (Accumulator) ชั่วคราวในการคำนวณ และเก็บผลลัพธ์ของการลบ
- สิ้นสุด: เมื่อกระบวนการหารเสร็จสิ้น ค่าที่อยู่ในรีจิสเตอร์ A คือ เศษ (Remainder)

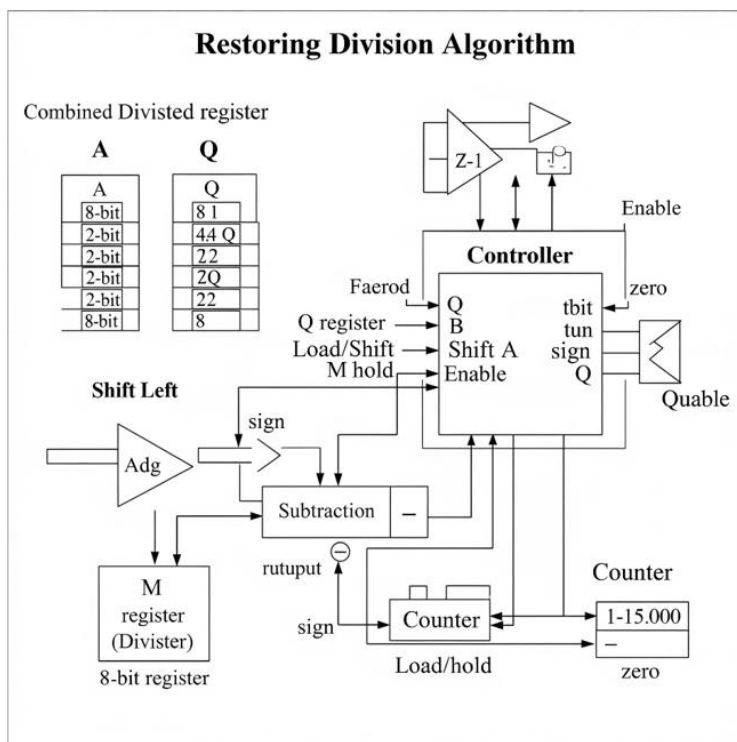
- ขนาด: มักจะมีขนาดเท่ากับรีจิสเตอร์ Q หรือเท่ากับขนาดของ M เพื่อรองรับการคำนวณ

4. วงจรลบ (Subtractor / Adder-Subtractor Unit):

- หน้าที่: ทำการคำนวณทางคณิตศาสตร์คือการลบ ($A - M$) โดยปกติแล้วจะถูกสร้างจากวงจรวก (Adder) โดยใช้หลักการ 2's complement ในการเปลี่ยนการลบเป็นการบวก
- การทำงาน: รับค่าจากรีจิสเตอร์ A และ M ผลลัพธ์การลบจะถูกป้อนกลับไปยังรีจิสเตอร์ A

5. วงจร Shift Left (Shifter):

- หน้าที่: ทำการเลื่อนบิตของรีจิสเตอร์ A และ Q ไปทางซ้ายพร้อมกัน 1 บิต ในแต่ละรอบ
- การทำงาน: บิตซ้ายสุดของ Q จะถูกเลื่อนเข้าสู่บิตขวาสุดของ A และบิตขวาสุดของ A จะถูกเลื่อนออกไป บิตขวาสุดของ Q จะถูกเติมด้วยค่า 0 หรือ 1 ตามผลลัพธ์การลบ



6. ตัวควบคุม (Control Unit):

- หน้าที่: เป็นสมองของวงจรรหาร ควบคุมลำดับขั้นตอนการทำงานทั้งหมด รวมถึงการสั่งให้ Shift, การสั่งให้ลบ, การตรวจสอบบิตเครื่องหมายของ A และการกำหนดบิต Q_0

- ส่วนประกอบ: มักจะประกอบด้วย Finite State Machine (FSM) และ Counter สำหรับนับจำนวนรอบการทำงาน

รูปที่ 6-33 ส่วนประกอบทางฮาร์ดแวร์และการเชื่อมต่อสัญญาณควบคุม (Logic Gate and Register Interconnects) สำหรับอัลกอริทึมการหารแบบคืนค่า (Restoring Division)

5.3.1 การเชื่อมโยงของส่วนประกอบตามโครงสร้างวงจร:

รีจิสเตอร์ Q และ รีจิสเตอร์ A ถูกจัดวางอยู่ติดกัน เสมือนเป็นรีจิสเตอร์รวมที่มีความยาวเป็นสองเท่าของตัวตั้ง (สมมติถ้าตัวตั้ง 4 บิต, Q มี 4 บิต, A มี 4 บิต, รวมเป็น 8 บิต)

- วงจร Shift Left จะรับอินพุตจากทั้ง A และ Q และส่งเอาต์พุตกลับไปทั้ง A และ Q ที่ถูกเลื่อนแล้ว
- รีจิสเตอร์ M เป็นรีจิสเตอร์ที่แยกต่างหาก และค่าภายใน M จะถูกส่งไปยัง วงจรลบ
- วงจรลบ รับอินพุตจาก A และ M ผลลัพธ์จากการลบจะถูกป้อนกลับเข้าไปยังรีจิสเตอร์ A และมีสัญญาณบอก "เครื่องหมาย" (บวก/ลบ) ส่งไปยัง ตัวควบคุม
- ตัวควบคุม จะรับสัญญาณจากวงจรลบ (บิตเครื่องหมาย) และจากตัวนับรอบ และจะส่งสัญญาณควบคุมไปยังรีจิสเตอร์ A, Q, วงจร Shift และวงจรลบ เพื่อดำเนินการในแต่ละขั้นตอน
- ตัวนับรอบ จะลดค่าลง 1 ในแต่ละครั้งที่ครบ 1 รอบการทำงาน และส่งสัญญาณไปที่ตัวควบคุมเมื่อถึง 0 เพื่อหยุดกระบวนการ

ตัวอย่างการทำงานจาก 1010_2 (เท่ากับ 10_{10}) หารด้วย 0011_2 (เท่ากับ 3_{10}) เพื่อแสดงให้เห็นการ

เปลี่ยนแปลงค่าในรีจิสเตอร์ต่างๆ ในแต่ละรอบการทำงาน

กำหนดค่าเริ่มต้น:

- Q (ตัวตั้ง) = 1010 (4 บิต)
- M (ตัวหาร) = 0011 (เราจะใช้ 5 บิตเพื่อรองรับการลบ: 00011)
- A (รีจิสเตอร์สะสม/เศษ) = 00000 (5 บิต, เริ่มต้นเป็นศูนย์)
- N (ตัวนับรอบ) = 4 (เท่ากับจำนวนบิตของตัวตั้ง)

เริ่มต้นการทำงาน:

รอบ (N)	การ ดำเนินการ	รีจิสเตอร์ A	รีจิสเตอร์ Q	ผลลัพธ์จากการลบ (A-M)	บิตซ้ายสุด ของ A	Q ₀
(เริ่มต้น)	-	00000	1010	-	-	-

ตารางที่ 6-7 ตารางอธิบายเพื่อแสดงการเปลี่ยนแปลงค่าในขั้นตอนเริ่มดำเนินการ

รอบที่ 1 (N = 4):

1. Shift Left AQ: เลื่อนบิตของ A และ Q ไปทางซ้าย 1 บิต
 - จาก A = 00000 , Q = 1010
 - กลายเป็น A = 00001 , Q = 0100 (บิตซ้ายสุดของ Q '1' ถูกเลื่อนไปขวาสุดของ A, ส่วนบิตขวาสุดของ Q จะถูกเว้นว่างไว้เพื่อรอผลลัพธ์)

2. $A = A - M$: นำค่าใน A ลบด้วยค่าใน M

- A (ปัจจุบัน) = 00001_2 (1_{10})
- $M = 00011_2$ (3_{10})
- $A - M = 00001_2 - 00011_2 = (1 - 3)_{10} = \text{ติดลบ}$ (ในเลขฐานสอง, จะเห็นว่าบิตซ้ายสุดของผลลัพธ์จะเป็น 1 ซึ่งแสดงถึงค่าลบในระบบ 2's complement หรือการติดลบในการลบแบบไม่คิดเครื่องหมาย)

3. ตรวจสอบบิตซ้ายสุดของ A:

- ผลลัพธ์จากการลบทำให้บิตซ้ายสุดของ A กลายเป็น 1 (ติดลบ)
- ทำ restoring: เนื่องจากผลลัพธ์ติดลบ จึงต้องดึงค่า A เดิมกลับมาใช้ คือ $A = 00001$ (บวก M กลับเข้าไปใน A หรือใช้ค่า A ก่อนการลบ)
- Set $Q_0 = 0$: กำหนดบิตขวาสุดของ Q เป็น 0

4. ลด N ลง: $N = 4 - 1 = 3$

รอบ (N)	การดำเนินการ	รีจิสเตอร์ A	รีจิสเตอร์ Q	ผลลัพธ์จากการลบ (A-M)	บิตซ้ายสุด ของ A	Q_0
4	Shift Left AQ	00001	0100			
	$A = A - M$	(ติดลบ)		$00001 - 00011$	1	0
	Restoring A, Set Q_0	00001	0100			

ตารางที่ 6-8 ตารางอธิบายเพื่อแสดงการเปลี่ยนแปลงค่าในขั้นตอนเริ่มดำเนินการ ($N = 4$)

รอบที่ 2 ($N = 3$):

1. Shift Left AQ:

- จาก $A = 00001$, $Q = 0100$
- กลายเป็น $A = 00010$, $Q = 1000$

2. $A = A - M$:

- A (ปัจจุบัน) = 00010_2 (2_{10})
- $M = 00011_2$ (3_{10})
- $A - M = 00010_2 - 00011_2 = (2 - 3)_{10} = \text{ติดลบ}$

3. ตรวจสอบบิตซ้ายสุดของ A:

- ผลลัพธ์จากการลบทำให้บิตซ้ายสุดของ A กลายเป็น 1 (ติดลบ)
- ทำ restoring: $A = 00010$

- Set $Q_0 = 0$

4. ลด N ลง: $N = 3 - 1 = 2$

รอบ (N)	การดำเนินการ	รีจิสเตอร์ A	รีจิสเตอร์ Q	ผลลัพธ์จากการลบ (A-M)	บิตซ้ายสุด ของ A	Q_0
3	Shift Left AQ	00010	1000			
	$A = A - M$	(ติดลบ)		00010 - 00011	1	0
	Restoring A, Set Q_0	00010	1000			

ตารางที่ 6-9 ตารางอธิบายเพื่อแสดงการเปลี่ยนแปลงค่าในขั้นตอนเริ่มดำเนินการ ($N = 3$)

รอบที่ 3 ($N = 2$):

1. Shift Left AQ:

- จาก $A = 00010$, $Q = 1000$
- กลายเป็น $A = 00101$, $Q = 0000$

2. $A = A - M$:

- A (ปัจจุบัน) = 00101_2 (5_{10})
- $M = 00011_2$ (3_{10})
- $A - M = 00101_2 - 00011_2 = 00010_2$ (2_{10})

3. ตรวจสอบบิตซ้ายสุดของ A:

- ผลลัพธ์จากการลบทำให้บิตซ้ายสุดของ A กลายเป็น 0 (เป็นบวก)
- ไม่ทำ restoring: A คงค่าเป็น 00010
- Set $Q_0 = 1$: กำหนดบิตขวาสุดของ Q เป็น 1 (เพราะหารได้)

4. ลด N ลง: $N = 2 - 1 = 1$

รอบ (N)	การดำเนินการ	รีจิสเตอร์ A	รีจิสเตอร์ Q	ผลลัพธ์จากการลบ (A-M)	บิตซ้ายสุด ของ A	Q_0
2	Shift Left AQ	00101	0000			
	$A = A - M$	00010		00101 - 00011	0	1
	Set Q_0	00010	0001			

ตารางที่ 6-10 ตารางอธิบายเพื่อแสดงการเปลี่ยนแปลงค่าในขั้นตอนเริ่มดำเนินการ ($N = 2$)

รอบที่ 4 ($N = 1$):

1. Shift Left AQ:

- จาก $A = 00010$, $Q = 0001$
- กลายเป็น $A = 00101$, $Q = 0010$

2. $A = A - M$:

- A (ปัจจุบัน) = 00101_2 (5_{10})
- $M = 00011_2$ (3_{10})
- $A - M = 00101_2 - 00011_2 = 00010_2$ (2_{10})

3. ตรวจสอบบิตซ้ายสุดของ A:

- ผลลัพธ์จากการลบทำให้บิตซ้ายสุดของ A กลายเป็น 0 (เป็นบวก)
- ไม่ทำ restoring: A คงค่าเป็น 00010
- Set $Q_0 = 1$: กำหนดบิตขวาสุดของ Q เป็น 1

4. ลด N ลง: $N = 1 - 1 = 0$

รอบ (N)	การ ดำเนินการ	รีจิสเตอร์ A	รีจิสเตอร์ Q	ผลลัพธ์จากการลบ (A-M)	บิตซ้ายสุด ของ A	Q_0
1	Shift Left AQ	00101	0010			
	$A = A - M$	00010		$00101 - 00011$	0	1
	Set Q_0	00010	0011			

ตารางที่ 6-11 ตารางอธิบายเพื่อแสดงการเปลี่ยนแปลงค่าในขั้นตอนเริ่มดำเนินการ ($N = 1$)

สิ้นสุดการทำงาน ($N = 0$):

- ผลหาร (Quotient) อยู่ในรีจิสเตอร์ Q: $Q = 0011_2$ (เท่ากับ 3_{10})
- เศษ (Remainder) อยู่ในรีจิสเตอร์ A: $A = 00010_2$ (เท่ากับ 2_{10})

ดังนั้น 1010_2 (10_{10}) หารด้วย 0011_2 (3_{10}) ได้ผลหารเท่ากับ 0011_2 (3_{10}) เศษ 00010_2 (2_{10}) ซึ่งถูกต้องตามหลักการหารเลขจำนวนเต็ม

ลำดับขั้นตอนใน Flowchart ของอัลกอริทึมซึ่งจะแสดงการวนซ้ำของการ Shift, Subtract, Restore (ถ้าจำเป็น) และการตั้งค่าบิตของผลหาร (ลำดับการทำงานจากไดอะแกรม:)

1.START: จุดเริ่มต้นของกระบวนการ

2.INITIALIZATION: กำหนดค่าเริ่มต้นของรีจิสเตอร์ A, Q, M และตัวนับ N

3.LOOP: $N > 0$?: ตรวจสอบว่า N ยังมากกว่า 0 หรือไม่

*ถ้าใช่ (YES): ดำเนินการขั้นตอนในลูปต่อ

***ถ้าไม่ใช่ (NO):** ออกจากลูปไปยังส่วนแสดง RESULTS

4. SHIFT LEFT AQ: เลื่อนบิตของ A และ Q ไปทางซ้าย 1 ตำแหน่ง

5. $A = A - M$: นำค่า A มาลบด้วย M

6. CHECK MSB OF A (Sign Bit): ตรวจสอบบิตซ้ายสุดของ A เพื่อดูเครื่องหมาย

*. ถ้า MSB = 0 (POSITIVE): หมายถึงผลลัพธ์เป็นบวก ไปยังขั้นตอน Set Q0 = 1

*. ถ้า MSB = 1 (NEGATIVE): หมายถึงผลลัพธ์เป็นลบ ไปยังขั้นตอน Set Q0 = 0

7. Set Q0 = 1 (กรณีบวก): กำหนดให้บิตขวาสุดของ Q เป็น 1

8. Set Q0 = 0 (กรณีลบ): กำหนดให้บิตขวาสุดของ Q เป็น 0

9. $A = A + M$ (Restore) (กรณีลบ): เนื่องจากผลลบติดลบ จึงต้องคืนค่า A กลับสู่สถานะเดิมก่อน
ลบ

10. $N = N - 1$: ลดค่า N ลง 1 ในทั้งสองกรณี (บวกและลบ)

11. เส้นทางกลับไปยัง LOOP: เมื่อลด N แล้ว กระบวนการจะกลับไปตรวจสอบเงื่อนไขของลูปอีกครั้ง

12. RESULTS: เมื่อลูปสิ้นสุด ($N = 0$) ค่า Quotient จะอยู่ใน Q และ Remainder จะอยู่ใน A

13. END: สิ้นสุดกระบวนการ

การอธิบายหลักการทำงานของ Restoring Division Algorithm ด้วยการเปรียบเทียบกับ
การแบ่งของให้เพื่อน เป็นวิธีที่ช่วยให้เห็นภาพและเข้าใจได้ง่าย

สมมติว่าเรามี ขนมเค้กอยู่ 10 ชิ้น (ตัวตั้ง - Dividend) และเราต้องการ แบ่งให้เพื่อนคนละ 3 ชิ้น (ตัวหาร - Divisor) เราอยากทราบว่าเพื่อนจะได้ขนมกี่คน (ผลหาร - Quotient) และจะเหลือขนมกี่ชิ้น (เศษ - Remainder) ลองดูว่า Restoring Division Algorithm ทำงานเหมือนกับการแบ่งขนมเค้กอย่างไร:

1. เตรียมพร้อม (Initialization):

- เรามีเค้ก 10 ชิ้น (ตัวตั้ง $Q = 1010_2$)
- เพื่อนแต่ละคนจะได้ 3 ชิ้น (ตัวหาร $M = 0011_2$)
- เริ่มแรก เรายังไม่มีใครได้เค้กเลย และเรายังไม่มีเศษเค้กที่แบ่งเกินมา (รีจิสเตอร์ $A = 00000$)
- เราจะแบ่งเค้กทั้งหมด 4 รอบ ($N = 4$) เพราะเค้กเรามี 4 บิต (หลัก)

2. เริ่มแบ่งรอบแรก (Shift Left AQ):

- เราจะหยิบเค้กมาพิจารณาจากซ้ายไปขวา (บิตจากซ้ายไปขวา) คล้ายกับการเลื่อนบิต Shift Left AQ
- สมมติว่าตอนนี้เรากำลังจะพิจารณาการแบ่งเค้ก "ชิ้นแรก"
- เราลองแบ่ง 1 ชิ้น
- เรามี 1 ชิ้น (ค่าใน A หลังจาก Shift) แต่เพื่อนต้องการ 3 ชิ้น (ค่าใน M)

3. ลองให้ (Subtract A - M):

- เราลอง "หยิบเค้ก 3 ชิ้น" ออกจากกองที่เรามี (A - M)
- เรามีแค่ 1 ชิ้น แต่เพื่อนต้องการ 3 ชิ้น (1 - 3)
- ผลลัพธ์คือ: เรามีไม่พอ! เราติดลบไป 2 ชิ้น! (บิตซ้ายสุดของ A เป็น 1 หรือติดลบ)

4. ถ้าให้เกินก็ดึงกลับ (Restore) และบันทึกผล:

- เนื่องจากเราลองแบ่งไปแล้วพบว่า "ให้เกินไป" (ผลลัพธ์ติดลบ)
 - เราก็ต้อง "ดึงเค้ก 3 ชิ้นนั้นกลับคืนมา" (Restore $A = A + M$) เพื่อให้เค้กกลับมาเท่าเดิม (00001_2 ใน A ก่อนลบ)
 - แล้วเราก็ก็นึกได้ว่า "เพื่อนคนนี้ยังไม่ได้เค้กเลย" (Set $Q_0 = 0$)
- ผ่านไป 1 รอบ: เราพิจารณาการแบ่งไป 1 บิตแล้ว

5. ทำซ้ำไปเรื่อยๆ (วนลูป N รอบ):

5.1 รอบที่ 2: Shift Left AQ: เราเลื่อนมาพิจารณาเค้กอีก 1 ชิ้น รวมกับของเดิมที่เหลือ

- เรามี 2 ชิ้น (00010_2 ใน A) แต่เพื่อนต้องการ 3 ชิ้น (M)
- ลองให้ (Subtract A - M): เรายังมีไม่พอ ($2 - 3 =$ ติดลบ)
- ดึงกลับ (Restore) และบันทึก: ก็ต้อง "ดึงเค้กกลับ" และบันทึกว่า "เพื่อนคนนี้ก็ยังไม่ได้" (Set $Q_0 = 0$)

5.2 รอบที่ 3: Shift Left AQ: เราเลื่อนมาพิจารณาอีก

- เรามี 5 ชิ้น (00101_2 ใน A) และเพื่อนต้องการ 3 ชิ้น (M)
- ลองให้ (Subtract A - M): เรามีพอ! ($5 - 3 = 2$ ชิ้น)
- ไม่ดึงกลับ (No Restore) และบันทึก: เนื่องจากมีพอและเหลือ เราก็ "ไม่ดึงเค้กกลับ" (ค่า $A = 00010_2$ เก็บไว้) และบันทึกว่า "เพื่อนคนนี้ได้เค้กแล้วนะ" (Set $Q_0 = 1$)

5.3 รอบที่ 4: Shift Left AQ: เราเลื่อนมาพิจารณาอีก

- เรามี 5 ชิ้น (00101_2 ใน A) และเพื่อนต้องการ 3 ชิ้น (M)
- ลองให้ (Subtract A - M): เรามีพอ! ($5 - 3 = 2$ ชิ้น)
- ไม่ดึงกลับ (No Restore) และบันทึก: "ไม่ดึงเค้กกลับ" (ค่า $A = 00010_2$ เก็บไว้) และบันทึกว่า "เพื่อนคนนี้ก็ได้อีกแล้วนะ" (Set $Q_0 = 1$)

6. จบการแบ่ง (N=0):

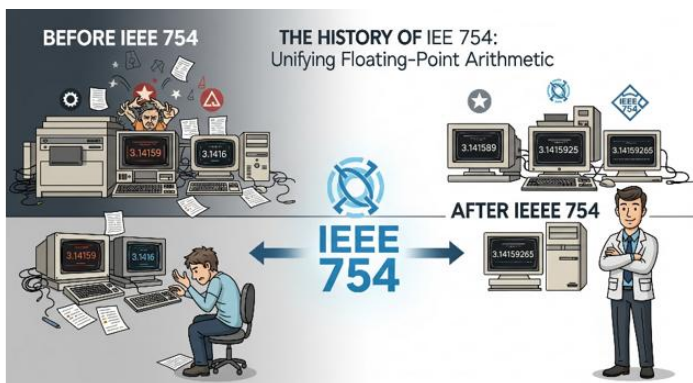
- เมื่อเราแบ่งครบทุก "ชิ้น" ของเค้กแล้ว (ครบ N รอบ)
- ผลหาร (Q): จะบอกว่ามีเพื่อนกี่คนที่ได้เค้ก (ในตัวอย่างของเราคือ 0011_2 หรือ 3 คน)
- เศษ (A): จะบอกว่าเราเหลือเค้กกี่ชิ้น (ในตัวอย่างของเราคือ 00010_2 หรือ 2 ชิ้น)

สรุปง่ายๆ คือ:

Restoring Division Algorithm ก็เหมือนกับการ "ลองให้" ไปก่อน ถ้า "ให้เกิน" (ผลลัพธ์ของการลบติดลบ) ก็ให้ "ดึงกลับคืนมา" (Restore) แล้วบันทึกว่ายังไม่ได้ จากนั้นก็ลองใหม่ แต่ถ้า "ให้พอดีหรือเหลือ" (ผลลัพธ์ของการลบเป็นบวก) ก็ "ไม่ต้องดึงกลับ" แล้วบันทึกว่าได้ไปแล้ว ทำแบบนี้ไปเรื่อยๆ จนกว่าจะแบ่งครบทุกส่วนของตัวตั้ง

6.การจัดเก็บและเลขจำนวนจริงตามมาตรฐาน IEEE754 (Floating Point Representation)

ประวัติและความเป็นมาของมาตรฐาน IEEE 754:



รูปที่ 6-34 วิวัฒนาการและความสำคัญของการกำหนดมาตรฐาน IEEE 754 เพื่อสร้างความเป็นเอกภาพในการคำนวณเลขทศนิยม

ก่อนที่จะมีมาตรฐาน IEEE

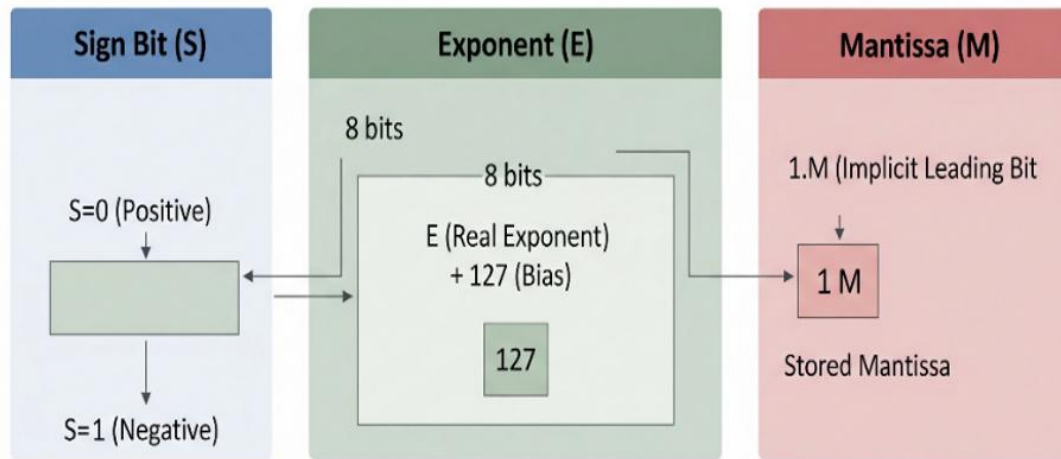
754 การจัดการกับตัวเลขทศนิยมในคอมพิวเตอร์นั้นมีความหลากหลายอย่างมาก แต่ละบริษัทผู้ผลิตฮาร์ดแวร์หรือผู้ออกแบบระบบต่างก็มีวิธีการเก็บและคำนวณเลขทศนิยมเป็นของตัวเอง สิ่งนี้ทำให้เกิดปัญหาใหญ่ขึ้นมาหลายประการ:

1. **ความไม่เข้ากัน (Incompatibility):** โปรแกรมที่เขียนบนคอมพิวเตอร์เครื่องหนึ่ง อาจไม่สามารถทำงานได้อย่างถูกต้องหรือให้ผลลัพธ์ที่แตกต่างกันเมื่อนำไปรันบนคอมพิวเตอร์อีกเครื่องหนึ่งได้ เนื่องจากวิธีการจัดเก็บและการคำนวณไม่เหมือนกัน
2. **ความซับซ้อนในการพัฒนาซอฟต์แวร์:** นักพัฒนาซอฟต์แวร์ต้องคอยคำนึงถึงความแตกต่างของระบบทศนิยมในแต่ละแพลตฟอร์ม ทำให้การเขียนโปรแกรมที่ต้องทำงานกับตัวเลขทศนิยมเป็นเรื่องที่ยุ่งยากและมีโอกาสเกิดข้อผิดพลาดสูง
3. **ความแม่นยำ (Precision) และช่วงของตัวเลข (Range) ที่แตกต่างกัน:** การออกแบบแต่ละแบบอาจให้ความแม่นยำหรือสามารถแทนตัวเลขได้ในช่วงที่ต่างกัน ทำให้ผลลัพธ์การคำนวณทางวิทยาศาสตร์และวิศวกรรมที่ต้องการความแม่นยำสูงมีปัญหา

ด้วยเหตุนี้ ในปี ค.ศ. 1985 (พ.ศ. 2528) Institute of Electrical and Electronics Engineers (IEEE) ซึ่งเป็นองค์กรวิชาชีพทางวิศวกรรมไฟฟ้าและอิเล็กทรอนิกส์ จึงได้ประกาศมาตรฐาน IEEE 754 ขึ้นมา เพื่อกำหนดวิธีการจัดเก็บและคำนวณตัวเลขทศนิยมในคอมพิวเตอร์ให้เป็นมาตรฐานสากลเดียวกัน ซึ่งช่วยแก้ปัญหาข้างต้นได้เป็นอย่างมาก ทำให้ซอฟต์แวร์สามารถทำงานได้ถูกต้องและให้ผลลัพธ์ที่สอดคล้องกันบนฮาร์ดแวร์ที่แตกต่างกัน

6.1 หลักการจัดเก็บเลขจำนวนจริงตามมาตรฐาน IEEE 754 (Floating Point Representation):

มาตรฐาน IEEE 754 จัดเก็บเลขทศนิยมในรูปแบบที่เรียกว่า "เลขชี้กำลัง" หรือ "Floating Point" ซึ่งคล้ายกับการเขียนตัวเลขในรูปแบบวิทยาศาสตร์ (Scientific Notation) โดยจะแบ่งตัวเลขออกเป็น 3 ส่วนหลักๆ



รูปที่ 6-35 องค์ประกอบและโครงสร้างการจัดเก็บรูปแบบบิตของหมายเลขทศนิยม

มาตรฐาน IEEE 754 (Single Precision)

6.1.1 บิตเครื่องหมาย (Sign Bit: S):

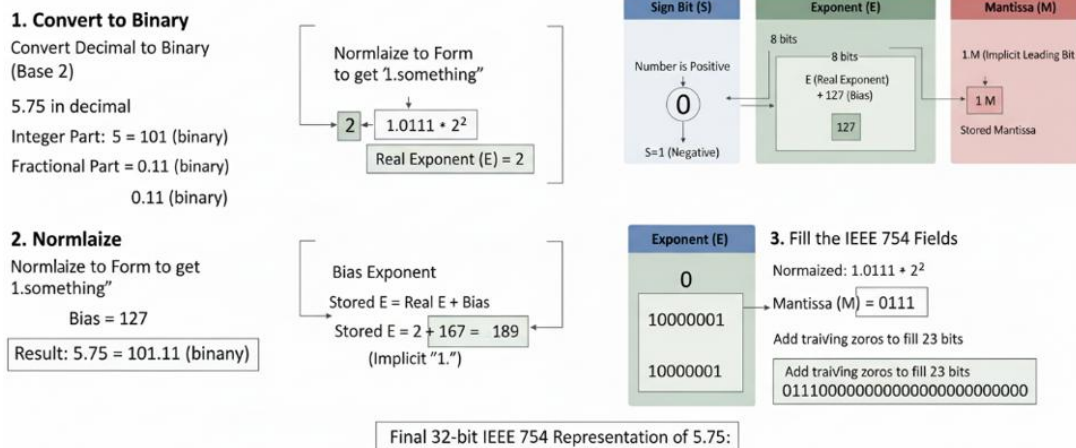
- เป็น 1 บิตแรกที่อยู่ซ้ายสุดของตัวเลข
- ใช้ระบุเครื่องหมายของตัวเลข:
 - ถ้า $S = 0$ หมายถึง ตัวเลขนั้นเป็น “ค่าบวก”
 - ถ้า $S = 1$ หมายถึง ตัวเลขนั้นเป็น “ค่าลบ”

6.1.2 ส่วนเลขชี้กำลัง (Exponent: E):

- เป็นส่วนที่บ่งบอกถึง "ตำแหน่งของจุดทศนิยม" หรือ "ขนาดของตัวเลข"
- มีการใช้เทคนิคที่เรียกว่า "Bias Exponent" โดยจะมีการบวกค่าคงที่ (Bias) เข้าไปกับเลขชี้กำลังจริง เพื่อให้เลขชี้กำลังทั้งหมดเป็นค่าบวกเสมอ ซึ่งช่วยให้การเปรียบเทียบขนาดของเลข Floating Point ทำได้ง่ายขึ้น
- สำหรับรูปแบบ Single-precision (32 บิต) ค่า Bias คือ 127 ดังนั้นค่า Exponent ที่จัดเก็บในคอมพิวเตอร์คือ $E + 127$
- สำหรับรูปแบบ Double-precision (64 บิต) ค่า Bias คือ “1023”

IEEE 754 Single-Precision Example: Converting 5.75

Step-step breakdown of how the the number 5.75 in a 32-bit floating format, following structure from <IMAGE.0.



รูปที่ 6-36 ขั้นตอนการแปลงค่าเลขทศนิยมฐานสิบ (5.75) เป็นรูปแบบมาตรฐาน IEEE 754 ขนาด 32 บิต ผ่านกระบวนการหาค่า Normalization, Biased Exponent และ Mantissa

6.1.3 ส่วน Mantissa (M) หรือ Significand:

- เป็นส่วนที่เก็บ "ค่าตัวเลขหลักๆ" ของทศนิยม จะอยู่ในรูปแบบ Normal Form เสมอ นั่นคือ จะมีเลข 1 อยู่หน้าจุดทศนิยมเสมอ (ยกเว้นกรณีพิเศษ เช่น ค่าศูนย์ หรือ Infinity) โดยเลข 1 ตัวนี้จะไม่ได้ถูกจัดเก็บจริงในหน่วยความจำ เพื่อประหยัดพื้นที่ (Implicit Leading Bit) ดังนั้น ค่า Mantissa จริงๆ ที่แทนคือ "1.M"

ตัวอย่าง เพื่อให้รูปภาพและแนวคิดของ IEEE 754 เข้าใจง่ายขึ้น ลองดูตัวอย่างการแปลงตัวเลขทศนิยมง่ายๆ เช่น 5.75 ให้เป็นรูปแบบ IEEE 754 แบบ Single-precision (32 บิต) จากภาพจะแสดงตัวเลข 5.75 และขั้นตอนการแปลงเป็นรูปแบบ IEEE 754 โดยเน้นที่แต่ละส่วนประกอบ: Sign Bit, Exponent และ Mantissa

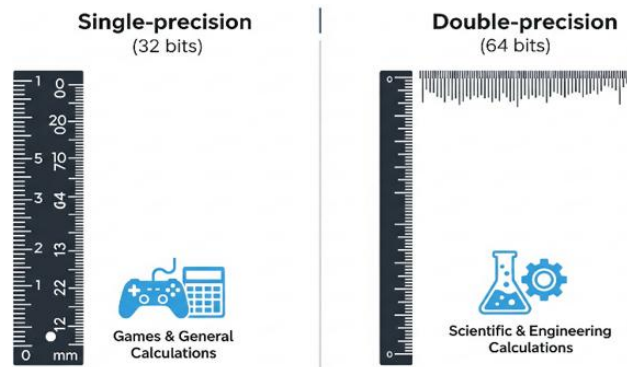
6.2. หลักการจัดเก็บรูปแบบการจัดเก็บเลขทศนิยมตามมาตรฐาน IEEE 754:

หลักการสำคัญของ IEEE 754 คือการจัดเก็บตัวเลขทศนิยมให้อยู่ในรูปแบบคล้ายสัญกรณ์วิทยาศาสตร์ (Scientific Notation) คือ $\text{Sign} \times \text{Mantissa} \times \text{Base}^{\text{Exponent}}$ ซึ่งในระบบคอมพิวเตอร์ที่ใช้เลขฐานสอง จะเป็น $(-1)^S \times 1.M \times 2^E$

มาตรฐานนี้แบ่งออกเป็น 2 รูปแบบหลักๆ ที่ใช้บ่อย ดังนี้:

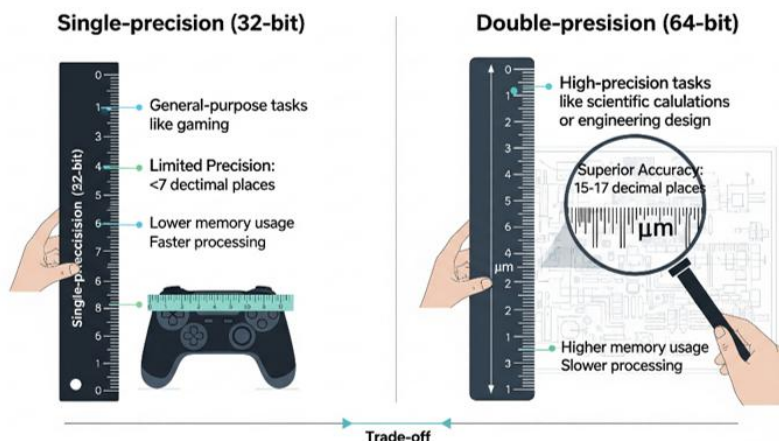
6.2.1. Single-precision (32 บิต) และ Double-precision (64 บิต): ขนาดและระดับความแม่นยำ ลองนึกภาพว่าเรากำลังใช้ **ไม้บรรทัด** ในการวัดสิ่งของ:

- Single-precision (32 บิต):** เปรียบเสมือน **ไม้บรรทัดสั้นๆ (32 บิต)** ที่มีขีดแบ่งละเอียดระดับ "มิลลิเมตร" (ประมาณ 7 หลักทศนิยม)



รูปที่ 6-37 ระดับความแม่นยำระหว่างมาตรฐาน Single-precision (32 บิต) และ Double-precision (64 บิต)

- **เหมาะสำหรับ:** ใช้ในงานทั่วไป ที่ไม่ต้องการความละเอียดสูงมาก เช่น เกม (กราฟิกไม่ต้อ่งเป๊ะขนาดมิลลิเมตรย่อยๆ), การคำนวณง่ายๆ ที่ไม่ซับซ้อน
 - **ข้อจำกัด:** เพราะมันสั้นและขีดจำกัด การวัดค่าที่ยาวมากๆ หรือต้องการความละเอียดระดับ "ไมโครเมตร" จะทำไม่ได้ หรือทำได้แต่ไม่แม่นยำ
- **Double-precision (64 บิต):** เปรียบเสมือน ไม้บรรทัดยาวๆ (64 บิต) ที่มีขีดแบ่งละเอียดระดับ "ไมโครเมตร" (ประมาณ 15-17 หลักทศนิยม)



รูปที่ 6-38 การเลือกใช้งาน (Trade-off Analysis) ระหว่าง

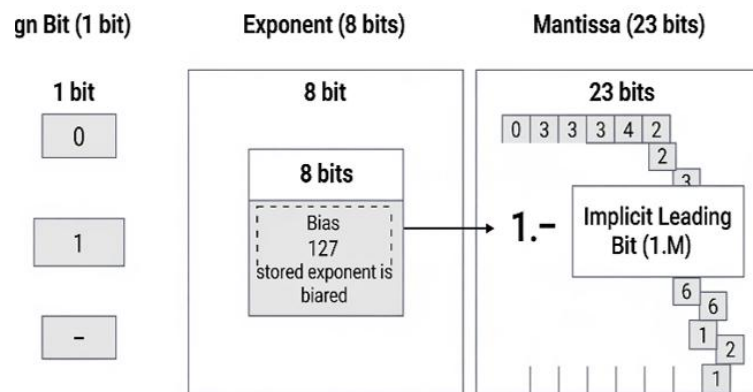
มาตรฐาน Single-precision และ Double-precision

- **เหมาะสำหรับ:** ใช้ในงานที่ต้องการความละเอียดสูงมากๆ เช่น การคำนวณทางวิทยาศาสตร์ (วงโคจรดวงดาว), วิศวกรรม (การออกแบบเครื่องบิน), การเงิน (คำนวณดอกเบี้ยที่ละเอียด)
 - **ข้อดี:** มันยาวและมีขีดแบ่งที่ละเอียดกว่ามาก ทำให้วัดค่าได้ทั้งที่ยาวมากๆ และวัดค่าได้ละเอียดสุดๆ
 - **ข้อเสีย (เล็กน้อย):** ก็คือมันใช้พื้นที่เก็บเยอะกว่า (64 บิต เทียบกับ 32 บิต) และใช้เวลาในการประมวลผลนานกว่าเล็กน้อย

สรุปเลขทศนิยม ยิ่งบิตเยอะ (64 บิต) ก็เหมือนไม้บรรทัดที่ยาวและขีดละเอียดยิบ ทำให้เก็บตัวเลขได้กว้างและแม่นยำกว่า

6.3 ส่วนประกอบของเลขทศนิยมตามมาตรฐาน IEEE 754: แยกส่วนประกอบของ "บ้านเลขที่"

การเปรียบเทียบตัวเลขทศนิยมกับการบอก "บ้านเลขที่" ที่มีรายละเอียด ตัวเลขทศนิยม (เช่น 123.45) เมื่อถูกจัดเก็บตามมาตรฐาน IEEE 754 จะถูกแบ่งออกเป็น 3 ส่วน เหมือนกับที่เราบอกข้อมูลของบ้าน:



รูปที่ 6-39 สรุปจำนวนบิตและรายละเอียดการจัดเก็บข้อมูลในส่วน

Sign Bit, Exponent และ Mantissa สำหรับตัวเลขทศนิยม

มาตรฐาน IEEE 754 แบบ 32 บิต

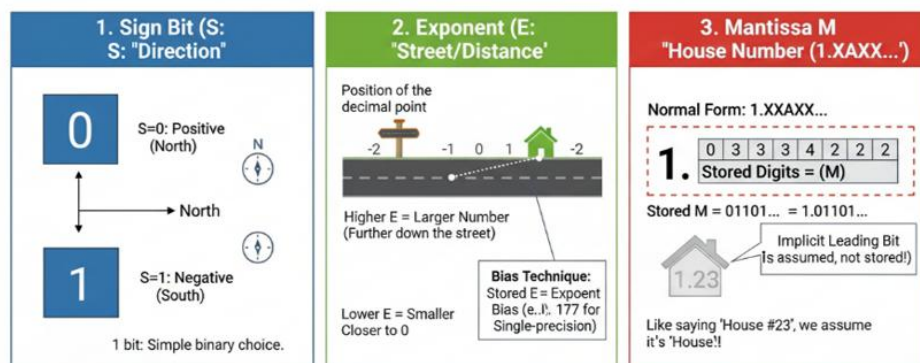
- Sign bit (S): บิตเครื่องหมาย (0 = บวก, 1 = ลบ)
 - เปรียบเหมือน: "ทิศทางของบ้าน"
 - ถ้า S = 0 (บวก) ก็คือ "บ้านอยู่ทางทิศเหนือ"
 - ถ้า S = 1 (ลบ) ก็คือ "บ้านอยู่ทางทิศใต้"
 - มีแค่ 1 บิต ง่ายๆ บอกแค่สองทางเลือก
- Exponent (E): ส่วนเลขชี้กำลัง (ตำแหน่งของจุดทศนิยม)
 - เปรียบเหมือน: "ซอยย่อย" หรือ "ระยะห่างจากถนนใหญ่"
 - บอกว่าบ้านหลังนี้อยู่ลึกเข้าไปในซอยแค่ไหน หรือห่างจากถนนใหญ่ไปที่ช่วงตึก
 - ในเลขทศนิยม ส่วน Exponent จะบอกว่า "จุดทศนิยม" ของตัวเลขจริงนั้น ควรจะอยู่ตรงไหน เมื่อเอา Mantissa มาแปลงกลับ
 - ยิ่ง Exponent มีค่ามาก ตัวเลขก็จะยิ่งใหญ่ขึ้น
 - ยิ่ง Exponent มีค่าน้อยลง ตัวเลขก็จะยิ่งเล็กลง (เข้าใกล้ 0 มากขึ้น)
- Mantissa (M) หรือ Significand: ส่วนตัวเลขหลักๆ หรือ "บ้านเลขที่จริงๆ" (ที่อยู่หลังจุดทศนิยม)
 - ส่วนนี้สำคัญเปรียบเหมือน: "ตัวเลขบ้านจริง ๆ (เช่น เลข 23, 45)" แต่มีเทคนิคพิเศษ
 - หลักการ Normal Form (รูปแบบปกติ):

ในทางคณิตศาสตร์ เลขทศนิยมใดๆ สามารถเขียนในรูปแบบวิทยาศาสตร์ได้หลายแบบ เช่น 123.45 ก็คือ 1.2345×10^2 หรือ 12.345×10^1 ก็ได้ แต่เพื่อให้คอมพิวเตอร์เก็บได้

"ประหยัดและเป็นมาตรฐานเดียวกัน" จึงบังคับว่า ทุกตัวเลข (ยกเว้น 0) จะต้องถูกจัดให้อยู่ในรูปแบบที่ขึ้นต้นด้วย 1.xxxxxxx เสมอ

- เช่น 1011.01_2 จะถูกจัดรูปเป็น $1.01101_2 \times 2^3$ (จุดทศนิยมเลื่อนไป 3 ตำแหน่ง)
- หรือ 0.00101_2 จะถูกจัดรูปเป็น $1.01_2 \times 2^{-3}$ (จุดทศนิยมเลื่อนไป -3 ตำแหน่ง)
- Implicit Leading Bit ('1' ที่ซ่อนอยู่): เนื่องจาก ทุกตัวเลขในรูปแบบ Normal Form จะขึ้นต้นด้วยเลข 1. เสมอ คอมพิวเตอร์จึงฉลาด! มันรู้ว่า "เลข 1 ตัวหน้าจุดทศนิยมนั้น มีอยู่แล้วเสมอ ไม่จำเป็นต้องเก็บให้เปลืองพื้นที่"
- มันก็เลย "ซ่อน" เลข 1 ตัวนั้นไว้ (Implicit 1)
- สิ่งที่ถูกเก็บจริงๆ ในส่วน Mantissa คือเฉพาะ ตัวเลขที่อยู่ "หลังจุดทศนิยม" เท่านั้น
- เปรียบเทียบ: เหมือนกับเวลาเราบอก "บ้านเลขที่ 123 หมู่ 5" คนฟังก็รู้ว่ามันเป็น "บ้าน" ไม่ต้องพูดคำว่า "บ้าน" ซ้ำอีกรอบ ตัวเลข "1" หน้าจุดทศนิยมก็เหมือนคำว่า "บ้าน" ที่ถูกซ่อนไว้ เพราะมันเป็น Default อยู่แล้ว

IEEE 754 Floating-Point Standard: Deconstructing the "House Address"



$$\text{Floating-Point Value} = (-1)^S \times 1.M \times 2^E$$

More bits = Longer ruler + Finer marks = Wider Range & Higher Precision!

รูปที่ 6-40 โครงสร้างเลขทศนิยมมาตรฐาน IEEE 754 กับองค์ประกอบของที่อยู่บ้าน (House Address Analogy)

- ตัวอย่าง: ถ้า Mantissa ที่คอมพิวเตอร์เก็บคือ 01101 (23 บิตใน Single-precision)
 - เวลาเรานำมาใช้จริง เราต้อง "เติม 1." เข้าไปข้างหน้า กลายเป็น 1.01101
 - นี่คือความฉลาดในการประหยัดพื้นที่ของมาตรฐาน IEEE 754

จากรูปที่ 6-40 นี้แสดงโครงสร้างของเลขทศนิยมแบบ Single-precision (32 บิต) ซึ่งประกอบด้วย 3 ส่วนหลัก ดังนี้:

- **Sign Bit (S):** บิตแรกที่อยู่ซ้ายสุด (1 บิต) เพื่อบอกเครื่องหมายของตัวเลข โดย 0 คือค่าบวก และ 1 คือค่าลบ
- **Exponent (E):** ส่วนถัดมา (8 บิต) ที่บอกตำแหน่งของจุดทศนิยมหรือขนาดของตัวเลข โดยค่าที่เก็บจะเป็น เลขชี้กำลังจริง + ค่า Bias (127) เพื่อให้ตัวเลขเป็นบวกเสมอ
- **Mantissa (M):** ส่วนสุดท้าย (23 บิต) ที่เก็บค่าตัวเลขหลักๆ ของทศนิยม และมีหลักการสำคัญคือ Implicit Leading Bit ซึ่งหมายถึงการซ่อนเลข 1 หน้าจุดทศนิยมไว้เสมอ เพื่อประหยัดพื้นที่ในการจัดเก็บ

ตัวอย่างท้ายบทที่ 1 จงแสดงวิธีการหารเลขฐานสองขนาด 4 บิต (แบบไม่คิดเครื่องหมาย) โดยกำหนดให้ ตัวตั้ง (Dividend) = 13_{10} และ ตัวหาร (Divisor) = 2_{10} โดยใช้วิธีการหารแบบ Restoring Division

ขั้นตอนการแปลงค่าเริ่มต้น

1. แปลงค่าจากเลขฐานสิบเป็นเลขฐานสองขนาด 4 บิต
 - ตัวตั้ง (Q) = $13_{10} = 1101_2$
 - ตัวหาร (M) = $2_{10} = 0010_2$
2. กำหนดค่าเริ่มต้นของรีจิสเตอร์ที่เกี่ยวข้อง
 - รีจิสเตอร์ A (Accumulator): กำหนดค่าเริ่มต้นเป็น 0000
 - รีจิสเตอร์ Q (Quotient): กำหนดค่าเริ่มต้นเป็น 1101 (ตัวตั้ง)
 - รีจิสเตอร์ M (Divisor): กำหนดค่าเริ่มต้นเป็น 0010 (ตัวหาร)
 - จำนวนรอบการทำงาน (N): เท่ากับจำนวนบิตของตัวตั้ง ซึ่งคือ 4 รอบ

การแสดงวิธีการหารแบบ Restoring Division

รอบที่ (N)	การกระทำ (Action)	A (Accumulator)	Q (Quotient)	เหตุผล
เริ่มต้น	Initial State	0000	1101	กำหนดค่าเริ่มต้น
---	---	---	---	---
1	Shift Left AQ	0001	1010	เลื่อน A และ Q ไปทางซ้าย 1 บิต
	$A = A - M$	1111	1010	$0001 - 0010 = 1111$ (ติดลบ)
	Restore A	0001	1010	ผลลัพธ์ติดลบ จึงต้องกู้คืนค่า A และกำหนด $Q_0 = 0$
	กำหนด Q_0	0001	1010	(จาก 1010)
---	---	---	---	---
2	Shift Left AQ	0011	0100	เลื่อน A และ Q ไปทางซ้าย 1 บิต
	$A = A - M$	0001	0100	$0011 - 0010 = 0001$ (ไม่ติดลบ)
	กำหนด Q_0	0001	0101	ผลลัพธ์ไม่ติดลบ จึงกำหนด $Q_0 = 1$
---	---	---	---	---
3	Shift Left AQ	0010	1010	เลื่อน A และ Q ไปทางซ้าย 1 บิต
	$A = A - M$	0000	1010	$0010 - 0010 = 0000$ (ไม่ติดลบ)
	กำหนด Q_0	0000	1011	ผลลัพธ์ไม่ติดลบ จึงกำหนด $Q_0 = 1$
---	---	---	---	---
4	Shift Left AQ	0001	0110	เลื่อน A และ Q ไปทางซ้าย 1 บิต
	$A = A - M$	1111	0110	$0001 - 0010 = 1111$ (ติดลบ)
	Restore A	0001	0110	ผลลัพธ์ติดลบ จึงต้องกู้คืนค่า A และกำหนด $Q_0 = 0$
	กำหนด Q_0	0001	0110	(จาก 0110)
---	---	---	---	---
สิ้นสุด	Final State	0001	0110	สิ้นสุดการทำงานครบ 4 รอบ

ตารางที่ 6-12 ตารางอธิบายเพื่อแสดงการเปลี่ยนแปลงค่าในขั้นตอน

อธิบายผังการทำงาน (Flowchart) การหารแบบ Restoring Division

กระบวนการจะเริ่มต้นจากการกำหนดค่ารีจิสเตอร์ต่างๆ และจะวนลูปตามจำนวนบิตของตัวตั้ง (N) โดยในแต่ละรอบจะมีเงื่อนไขการตรวจสอบและปรับปรุงค่า เพื่อหาผลลัพธ์ที่ถูกต้อง

ขั้นตอนที่ 1: การเริ่มต้น (Initialization)

- **Start:** เริ่มต้นกระบวนการ
- **กำหนดค่าเริ่มต้น:**
 - กำหนดค่าในรีจิสเตอร์ A (Accumulator) = **0000**
 - กำหนดค่าในรีจิสเตอร์ Q (Quotient) = **1101** (ค่าตัวตั้ง)
 - กำหนดค่าในรีจิสเตอร์ M (Divisor) = **0010** (ค่าตัวหาร)
 - กำหนดค่าตัวนับ N = **4** (จำนวนบิต)

ขั้นตอนที่ 2: การวนลูป (Loop) Loop N ครั้ง: ทำซ้ำขั้นตอนถัดไปจนกว่า N จะมีค่าเป็น 0

ขั้นตอนที่ 3: การประมวลผลในแต่ละรอบ

- **Shift Left AQ:** เลื่อนบิตในรีจิสเตอร์ A และ Q ไปทางซ้าย 1 บิต
- **A = A - M:** ทำการลบค่า M ออกจาก A (ในทางปฏิบัติคือการบวกค่า 2's complement ของ M)
- **ตรวจสอบบิตซ้ายสุดของ A (MSB):**
 - ถ้าบิตซ้ายสุดของ A = 1 (ผลลัพธ์ติดลบ):
 - **กู้คืนค่า A (Restore A):** ทำ $A = A + M$ (นำค่า M กลับมาบวกคืน)
 - กำหนดค่าบิตสุดท้ายของ Q (Q_0): ตั้งค่า $Q_0 = 0$
 - ถ้าบิตซ้ายสุดของ A = 0 (ผลลัพธ์เป็นบวกหรือศูนย์):
 - **ไม่ต้องกู้คืนค่า A:** ค่า A ที่ได้จากการลบเป็นค่าที่ถูกต้อง
 - กำหนดค่าบิตสุดท้ายของ Q (Q_0): ตั้งค่า $Q_0 = 1$
- **ลดค่าตัวนับ:** $N = N - 1$

ขั้นตอนที่ 4: การสิ้นสุด (Termination)

- **ตรวจสอบ N:** ถ้า $N = 0$ ให้หยุดการทำงาน
- **End:** สิ้นสุดกระบวนการ
- **ผลลัพธ์:**
 - ค่าในรีจิสเตอร์ Q คือ **ผลหาร (Quotient)**
 - ค่าในรีจิสเตอร์ A คือ **เศษเหลือ (Remainder)**

จากผังการทำงานนี้ จะเห็นได้ว่าหลักการสำคัญของ Restoring Division คือการตรวจสอบผลลัพธ์การลบในแต่ละรอบ หากลบแล้วติดลบ ก็จะต้อง "กู้คืน" ค่าเดิมกลับมา และกำหนดบิตผลหารเป็น 0 แต่หากไม่ติดลบ ก็จะเก็บค่าที่ลบได้นั้นไว้และกำหนดบิตผลหารเป็น 1 และผลลัพธ์สุดท้ายที่ได้คือ:

- **ผลหาร (Quotient):** ค่าในรีจิสเตอร์ Q คือ **0110₂** ซึ่งเท่ากับ **6₁₀**
- **เศษเหลือ (Remainder):** ค่าในรีจิสเตอร์ A คือ **0001₂** ซึ่งเท่ากับ **1₁₀**

จะเห็นว่าผลลัพธ์ที่ได้จากการคำนวณด้วยวิธี Restoring Division คือ $13 / 2 = 6$ เศษ 1

ตัวอย่างที่ 2 จงแสดงวิธีการหารเลขฐานสองขนาด 4 บิต (แบบไม่คืนเครื่องหมาย) โดยกำหนดให้ตัวตั้งของ

การหาร = 13_{10} และ ตัวหาร = 2_{10} โดยใช้วิธี การหารแบบ NON- Restoring Division

การหารแบบ Non-Restoring Division เป็นอีกหนึ่งวิธีที่ใช้ในการหารเลขฐานสอง ซึ่งมีความแตกต่างจากวิธี Restoring Division ที่ไม่จำเป็นต้อง "กู้คืน" ค่าในรีจิสเตอร์ A หากผลลัพธ์การลบออกมาติดลบ ทำให้การทำงานมีประสิทธิภาพมากขึ้น

ขั้นตอนการแปลงค่าเริ่มต้น

- ตัวตั้ง (Q): $13_{10} = 1101_2$
- ตัวหาร (M): $2_{10} = 0010_2$
- รีจิสเตอร์ A (Accumulator): กำหนดค่าเริ่มต้นเป็น 0000
- จำนวนรอบ (N): เท่ากับจำนวนบิตของตัวตั้ง ซึ่งคือ 4 รอบ

การแสดงวิธีการหารแบบ Non-Restoring Division หลักการสำคัญของวิธีนี้คือ:

- ถ้าผลลัพธ์การลบในรอบก่อนหน้าติดลบ (MSB ของ A = 1) ในรอบถัดไปให้ทำการ บวก (A + M)
- ถ้าผลลัพธ์การลบในรอบก่อนหน้าไม่ติดลบ (MSB ของ A = 0) ในรอบถัดไปให้ทำการ ลบ (A - M)

รอบที่	การกระทำ (Action)	A (Accumulator)	Q (Quotient)	เหตุผลและหมายเหตุ
เริ่มต้น	Initial State	0000	1101	กำหนดค่าเริ่มต้น
---	---	---	---	---
1	Shift Left AQ	0001	1010	เลื่อน A และ Q ไปทางซ้าย 1 บิต
	A = A - M	1111	1010	$0001 - 0010 = 1111$ (ติดลบ)
	กำหนด $Q_0 = 0$	1111	1010	ผลลัพธ์ติดลบ จึงกำหนด Q_0 เป็น 0
---	---	---	---	---
2	Shift Left AQ	1110	0100	เลื่อน A และ Q ไปทางซ้าย 1 บิต
	A = A + M	0000	0100	เนื่องจากรอบก่อนหน้าติดลบ รอบนี้จึงทำการบวก: $1110 + 0010 = 0000$ (ไม่ติดลบ)
	กำหนด $Q_0 = 1$	0000	0101	ผลลัพธ์ไม่ติดลบ จึงกำหนด Q_0 เป็น 1
---	---	---	---	---
3	Shift Left AQ	0001	0100	เลื่อน A และ Q ไปทางซ้าย 1 บิต
	A = A - M	1111	0100	เนื่องจากรอบก่อนหน้าไม่ติดลบ รอบนี้จึงทำการลบ: $0001 - 0010 = 1111$ (ติดลบ)
	กำหนด $Q_0 = 0$	1111	0100	ผลลัพธ์ติดลบ จึงกำหนด Q_0 เป็น 0

รอบที่	การกระทำ (Action)	A (Accumulator)	Q (Quotient)	เหตุผลและหมายเหตุ
---	---	---	---	---
4	Shift Left AQ	1110	1000	เลื่อน A และ Q ไปทางซ้าย 1 บิต
	$A = A + M$	0000	1000	เนื่องจากรอบก่อนหน้าติดลบ รอบนี้จึงทำการบวก: $1110 + 0010 = 0000$ (ไม่ติดลบ)
	กำหนด $Q_0 = 1$	0000	1001	ผลลัพธ์ไม่ติดลบ จึงกำหนด Q_0 เป็น 1
---	---	---	---	---
ขั้นสุดท้าย	Restore A	0001	1001	ตรวจสอบครั้งสุดท้าย: หากผลลัพธ์ในรอบสุดท้ายติดลบ ให้ทำการกู้คืน A โดยการบวก $A + M$ หนึ่งครั้ง

ตารางที่ 6-13 ตารางอธิบายเพื่อแสดงการเปลี่ยนแปลงค่าในขั้นตอน

จากตารางข้างต้น ผลลัพธ์สุดท้ายที่ได้คือ:

- ผลหาร (Quotient): ค่าในรีจิสเตอร์ Q คือ **0110₂** ซึ่งเท่ากับ **6₁₀**
- เศษเหลือ (Remainder): ค่าในรีจิสเตอร์ A คือ **0001₂** ซึ่งเท่ากับ **1₁₀**

จะเห็นว่าผลลัพธ์ที่ได้จากการคำนวณคือ $13 / 2 = 6$ เศษ 1 ซึ่งถูกต้องตรงกันกับวิธี Restoring Division

ชุดข้อสอบเพื่อทบทวนความเข้าใจบทที่ 6 หน่วยคำนวณทางคณิตศาสตร์และตรรกะ (ALU)

1. หน้าที่หลักของ ALU (Arithmetic and Logic Unit) คืออะไร?

- ดึงคำสั่ง (Fetch) จากหน่วยความจำหลัก
- ควบคุมการทำงานของหน่วยประมวลผลกลาง
- ประมวลผลทางคณิตศาสตร์และตรรกะ
- จัดเก็บข้อมูลและผลลัพธ์ชั่วคราว

เฉลย: ค.

2. การประมวลผลทางคณิตศาสตร์พื้นฐานที่ ALU ดำเนินการคืออะไร?

- sin, cos, tan
- บวก, ลบ, คูณ, หาร
- การจัดการอินพุต/เอาต์พุต
- การอ้างอิงตำแหน่งหน่วยความจำ

เฉลย: ข.

3. การดำเนินการทางตรรกะและการเปรียบเทียบพื้นฐานที่ ALU ดำเนินการคืออะไร?

ก. LOAD, STORE

ข. JUMP, CALL

ค. AND, OR, NOT, COMPLEMENT, การเปรียบเทียบค่า

ง. SHIFT LEFT, SHIFT RIGHT (ทางคณิตศาสตร์)

เฉลย: ค.

4. ส่วนประกอบใดที่ทำหน้าที่สั่งการให้ ALU ทำการประมวลผลแบบใดแบบหนึ่ง?

ก. Memory Unit

ข. Input/Output Unit

ค. Control Unit

ง. Registers

เฉลย: ค.

5. หน้าที่หลักของ แฟล็ก (Flags) ในการประมวลผลของ ALU คืออะไร?

ก. เก็บตัวถูกดำเนินการ (Operand)

ข. เก็บที่อยู่ (Address) ของคำสั่งถัดไป

ค. จัดเก็บสถานะที่ได้จากการประมวลผล (เช่น ตัวทด, ค่าเป็นลบ, ผลลัพธ์เป็นศูนย์)

ง. จัดเก็บคำสั่งที่กำลังถูกประมวลผล

เฉลย: ค.

6. การแทนค่าตัวเลขจำนวนเต็มแบบ Unsigned Representation ต่างจากแบบ Sign-Magnitude Representation อย่างไร?

ก. Unsigned ใช้บิตซ้ายสุดเป็นเครื่องหมายเสมอ

ข. Sign-Magnitude ใช้บิตทั้งหมดแทนขนาดของตัวเลข

ค. Unsigned ใช้บิตทั้งหมดแทนขนาดของตัวเลข ไม่มีการระบุเครื่องหมาย

ง. Sign-Magnitude ไม่สามารถแทนค่าลบได้

เฉลย: ค.

7. 1's Complement ของเลขฐานสอง 10110 คืออะไร?

ก. 10111

ข. 01000

ค. 01001

ง. 11001

เฉลย: ค. (สลับ 0 เป็น 1 และ 1 เป็น 0)

8. 2's Complement ของเลขฐานสอง 01101 คืออะไร?

ก. 10010

ข. 10011

ค. 11111

ง. 01101

เฉลย: ข. (1's Complement ของ 01101 คือ 10010 แล้วบวก 1 ได้ 10011)

9. ผลลัพธ์ของการบวกเลขฐานสอง $1011_2 + 0101_2$ คืออะไร?

ก. 10000_2

ข. 11011_2

ค. 10000_2

ง. 01111_2

เฉลย: ค. ($1011_2 = 11_{10}$, $0101_2 = 5_{10}$, $11 + 5 = 16_{10} = 10000_2$)

10. ในการลบเลขฐานสองโดยใช้วิธี 2's Complement หากผลลัพธ์ของการบวกมีตัวทด (Carry-out) เกิดขึ้น แสดงว่าผลลัพธ์สุดท้ายมีลักษณะอย่างไร?

ก. ต้องทำ 2's Complement อีกครั้ง และเป็นค่าลบ

ข. ตัดตัวทดทิ้ง คำตอบจะเป็นค่าบวก

ค. ตัดตัวทดทิ้ง คำตอบจะเป็นค่าลบ

ง. เก็บตัวทดไว้เป็นส่วนหนึ่งของคำตอบ

เฉลย: ข.

11. วงจรตรรกะพื้นฐานที่ใช้ในการบวกเลขฐานสองในคอมพิวเตอร์คืออะไร?

ก. Decoder และ Encoder

ข. Multiplexer และ Demultiplexer

ค. Half Adder และ Full Adder

ง. Register และ Flag

เฉลย: ค.

12. ผลลัพธ์ของการคูณเลขฐานสอง $101_2 \times 11_2$ คืออะไร?

ก. 101_2

ข. 1101_2

ค. 1111_2

ง. 1001_2

เฉลย: ค. ($101_2 = 5_{10}$, $11_2 = 3_{10}$, $5 \times 3 = 15_{10} = 1111_2$)

13. การคูณเลขจำนวนเต็ม ไม่คิดเครื่องหมาย (Unsigned) การดำเนินการใดในการสร้างผลคูณย่อย (Partial Products)?

ก. การลบ (Subtraction)

ข. การทำ 2's Complement

ค. การบวก (Addition) และการเลื่อนบิต (Shift)

ง. การหาร (Division)

เฉลย: ค.

14. ในวงจรคูณพื้นฐานของฮาร์ดแวร์ ส่วนประกอบหลักที่ใช้ในการเก็บตัวตั้ง (Multiplicand), ตัวคูณ (Multiplier), และผลคูณ (Product) คืออะไร?

ก. MAR, MBR, IR

ข. PC, SP, MBR

ค. M (Multiplicand Register), Q (Multiplier Register), A (Accumulator)

ง. ALU, Control Unit, Memory

เฉลย: ค.

15. ในการคูณเลขจำนวนเต็มแบบไม่คิดเครื่องหมายด้วยการทำงานฮาร์ดแวร์ บิตใดของ ตัวคูณ (Q) ที่ถูกตรวจสอบเพื่อตัดสินใจว่าจะมีการบวกตัวตั้ง (M) เข้าไปใน A หรือไม่?

- ก. บิตซ้ายสุด (Q_{n-1}) ข. บิตขวาถัดไป (Q_1) ค. บิตขวาสุด (Q_0) ง. บิตเครื่องหมาย (S)

เฉลย: ค.

16. การคูณเลขจำนวนเต็ม คิดเครื่องหมาย (Signed Integer Multiplication) ต่างจากการคูณ ไม่คิดเครื่องหมายอย่างไร?

- ก. ใช้การดำเนินการบวกและการเลื่อนบิตเท่านั้น
ข. ต้องคำนึงถึงและจัดการกับเครื่องหมายของตัวเลข
ค. ใช้เฉพาะวงจร Full Adder เท่านั้น
ง. ต้องแปลงเป็นเลขฐานสิบก่อนทำการคูณ

เฉลย: ข.

17. การขยายบิตเครื่องหมาย (Sign Extension) มีวัตถุประสงค์หลักเพื่ออะไร?

- ก. เพื่อลดจำนวนบิตที่ใช้ในการจัดเก็บตัวเลข
ข. เพื่อเปลี่ยนค่าบวกให้เป็นค่าลบ
ค. เพื่อรักษาค่าและเครื่องหมายของตัวเลข เมื่อเพิ่มขนาดบิตให้ยาวขึ้น
ง. เพื่อใช้ในขั้นตอนการทำ 1's Complement

เฉลย: ค.

18. ในการคูณเลขฐานสองแบบฮาร์ดแวร์สำหรับการคูณธรรมดา สิ่งที่ทำเป็นประจำทุกรอบการทำงาน (นอกเหนือจากการบวกตามเงื่อนไข) คืออะไร?

- ก. การทำ 2's Complement ให้กับ A ข. การเลื่อนบิตขวา (Shift Right) ของ (A, Q)
ค. การเลื่อนบิตซ้าย (Shift Left) ของ Q ง. การกำหนดค่า A ให้เป็นศูนย์ใหม่

เฉลย: ข.

19. หากพิจารณา Flowchart การคูณเลขฐานสองแบบฮาร์ดแวร์สำหรับการคูณธรรมดาแบบตัวเลขขนาด 4 บิต จำนวนรอบการทำงาน (N) ทั้งหมดจะต้องทำกี่รอบ?

- ก. 2 รอบ ข. 3 รอบ ค. 4 รอบ ง. 8 รอบ

เฉลย: ค. (เท่ากับจำนวนบิตของตัวคูณ)

20. (อัลกอริทึมของบูธ (Booth's Algorithm) ถูกออกแบบมาเพื่อคำนวณการคูณเลขฐานสองที่มีลักษณะใด?

- ก. เลขฐานสองแบบไม่คิดเครื่องหมายเท่านั้น ข. เลขฐานสองแบบทศนิยม
ค. เลขฐานสองแบบมีเครื่องหมาย (2's Complement) ง. เลขฐานแปด

เฉลย: ค.

21. หากใช้ Booth's Algorithm และพบว่า $Q_0 Q_{-1}$ มีค่าเท่ากับ 01 ควรดำเนินการอย่างไร?

- ก. $A = A - M$ แล้ว Arithmetic Shift Right (ASR)
- ข. Arithmetic Shift Right (ASR)
- ค. $A = A + M$ แล้ว Arithmetic Shift Right (ASR)
- ง. $A = A + M + Q$

เฉลย: ค. (01 คือการเปลี่ยนจาก 0 เป็น 1 ให้บวก M)

22. จาก Flowchart การคูณเลขฐานสอง แบบ Booth's Algorithm ในขั้นตอน Arithmetic Shift Right (A, Q, Q_{-1}) บิตเครื่องหมายในรีจิสเตอร์ A จะถูกจัดการอย่างไร?

- ก. บิตเครื่องหมายจะถูกสลับค่า (Complement)
- ข. บิตเครื่องหมายจะถูกกำหนดให้เป็น 0 เสมอ
- ค. บิตเครื่องหมายจะถูกคัดลอก (Preserved) ไปที่ตำแหน่งถัดไปทางขวา
- ง. บิตเครื่องหมายจะถูกละทิ้งไป

เฉลย: ค.

23. ในการหารเลขฐานสองแบบ Integer Division ผลลัพธ์ที่ได้จากการหารจะประกอบด้วยอะไรบ้าง?

- ก. ตัวตั้ง (Dividend) และ ตัวหาร (Divisor)
- ข. ผลหาร (Quotient) และ เศษเหลือ (Remainder)
- ค. เครื่องหมาย (Sign) และ เลขชี้กำลัง (Exponent)
- ง. ตัวทด (Carry) และ ตัวยืม (Borrow)

เฉลย: ข.

24. หากการหารเลขฐานสองแบบไม่ลงตัว ผลลัพธ์ที่ไม่ใช่ผลหารจะถูกเรียกว่าอะไร?

- ก. ตัวตั้ง (Dividend)
- ข. ตัวหาร (Divisor)
- ค. เศษเหลือ (Remainder)
- ง. ตัวทด (Carry)

เฉลย: ค.

25. Restoring Division Algorithm มีลักษณะเด่นอย่างไรในกรณีที่ผลลัพธ์ของการลบเป็นค่าลบ?

- ก. ทำการเปลี่ยนเครื่องหมายเป็นบวกทันที
- ข. ทำการยุติการหารทันที
- ค. ต้องดึงค่าเดิมของตัวตั้งกลับมา (Restoring) ก่อนดำเนินการต่อ
- ง. ทำการ Arithmetic Shift Right

เฉลย: ค.

26. ส่วนประกอบหลักของวงจรหารเลขฐานสอง (Hardware for Binary Division) ที่ใช้ในการเก็บ ตัวตั้ง (Dividend) เริ่มต้นคือรีจิสเตอร์ใด?

ก. M (Divisor Register)

ข. A (Accumulator)

ค. Q (Dividend/Quotient Register)

ง. N (Counter)

เฉลย: ค. (ตัวตั้งเริ่มต้นเก็บใน Q)

27. ในการทำงานของ Hardware Restoring Division Algorithm เมื่อมีการ Shift Left AQ แล้ว ขั้นตอนถัดไปคืออะไร?

ก. Set $Q_0=1$

ข. Set $Q_0=0$

ค. $A = A - M$ (ลบตัวหารออกจาก A)

ง. ดึงค่า A เดิมกลับมาใช้

เฉลย: ค.

28. จาก Flowchart การหารเลขฐานสองแบบ Restoring Division Algorithm หากบิตซ้ายสุดของ A เท่ากับ 1 แสดงว่าเกิดอะไรขึ้น และต้องดำเนินการอย่างไรต่อไป?

ก. การลบประสบความสำเร็จ Set $Q_0=1$

ข. การลบประสบความสำเร็จ Set $Q_0=0$

ค. การลบล้มเหลว (ผลเป็นลบ) Set $Q_0=0$ และดึงค่า A เดิมกลับมา (Restoring)

ง. การลบล้มเหลว (ผลเป็นลบ) Set $Q_0=1$ และทำ 2's Complement

เฉลย: ค.

29. มาตรฐานที่คอมพิวเตอร์ส่วนใหญ่ใช้ในการจัดเก็บเลขจำนวนจริง (Floating Point Number) คือมาตรฐานใด?

ก. ASCII Code

ข. BCD Code

ค. IEEE 754

ง. EBCDIC

เฉลย: ค.

30. หลักการจัดเก็บเลขจำนวนจริงตามมาตรฐาน IEEE 754 จะอยู่ในรูปทั่วไปอย่างไร?

ก. $(-1) \times 1.M \times 2^E$

ข. $(-1)^S \times M \times 2^E$

ค. $(-1)^S \times 1.M \times 2^{E-Bias}$

ง. $S \times M \times 2^E$

เฉลย: ค.

31. รูปแบบเลขชี้กำลังแบบ Single-precision และ Double-precision ตามมาตรฐาน IEEE 754 มีขนาดกี่บิตตามลำดับ?

ก. 16 บิต และ 32 บิต

ข. 32 บิต และ 80 บิต

ค. 64 บิต และ 128 บิต

ง. 32 บิต และ 64 บิต

เฉลย: ง.

32. Sign bit (S) ซึ่งเป็นส่วนประกอบของเลขทศนิยมตามมาตรฐาน IEEE 754 ทำหน้าที่อะไร?

- ก. ระบุตำแหน่งของจุดทศนิยม
- ข. ระบุเครื่องหมายของตัวเลข (0=บวก, 1=ลบ)
- ค. ระบุค่าสัมบูรณ์ของตัวเลข
- ง. ระบุค่า Bias ของ Exponent

เฉลย: ข.

33. ส่วน Exponent (E) ในมาตรฐาน IEEE 754 ใช้ในการระบุค่าใด?

- ก. เครื่องหมายของตัวเลข
- ข. เลขชี้กำลัง (ตำแหน่งของจุดทศนิยม) ในรูปของ Biased Exponent
- ค. ส่วนที่เป็นเศษส่วน (Fraction) ของตัวเลข
- ง. ค่าคงที่ 1 หน้าจุดทศนิยม

เฉลย: ข.

34. ในการจัดเก็บ Mantissa (M) ตามมาตรฐาน IEEE 754 ในรูปแบบ Normalized Form (รูปแบบปกติ)

ทำไมจึง ไม่ต้องจัดเก็บ เลข 1 ที่อยู่หน้าจุดทศนิยม (e.g., $1.xxxxx_2$)?

- ก. เพราะเลข 1 ถูกนำไปรวมกับ Sign bit แล้ว
- ข. เพราะเลข 1 ถูกเข้ารหัสอยู่ในส่วน Exponent
- ค. เพราะเลข 1 ตัวนี้เป็นค่าคงที่ (Implicit Leading 1) จึงละไว้เพื่อประหยัดบิต
- ง. เพราะเลข 1 จะถูกใช้เมื่อเป็น Double-precision เท่านั้น

เฉลย: ค.